



DETEKSI ANOMALI MENGGUNAKAN LSTM-AUTOENCODER PADA DATA LOG EVENT WAZUH

I Gede Arya Restyandika Pramana Putra
NIM. 2201010325

Dosen Pembimbing I
Ir. I Made Dwi Putra Asana, S.Kom., MT.

Dosen Pembimbing II
Bagus Kusuma Wijaya, S.E., M.A.B.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI DAN INFORMATIKA
INSTITUT BISNIS DAN TEKNOLOGI INDONESIA
DENPASAR 2026

HALAMAN PENGESAHAN

Judul : **DETEKSI ANOMALI MENGGUNAKAN LSTM-AUTOENCODER PADA DATA LOG EVENT WAZUH**
Nama : **I Gede Arya Restyandika Pramana Putra**
NIM : **2201010325**
Program Studi : **Informatika**

Telah diperiksa dan disetujui sebagai Skripsi mahasiswa Program Studi Informatika Institut Bisnis dan Teknologi Indonesia.

Menyetujui:

Pembimbing I

Pembimbing II

Ir. I Made Dwi Putra Asana,
S.Kom., M.T.,

NUPTK. 7241769670130373

Bagus Kusuma Wijaya, S.E.,
M.A.B.,

NUPTK. 2654761662130202

Mengetahui:

Koordinator Program Studi Informatika

I Gusti Agung Indrawan, S.T., M.T.
NUPTK. 3151763664130233

HALAMAN PERSETUJUAN

Judul : **DETEKSI ANOMALI MENGGUNAKAN LSTM-AUTOENCODER PADA DATA LOG EVENT WAZUH**
Nama : **I Gede Arya Restyandika Pramana Putra**
NIM : **2201010325**
Program Studi : **Informatika**

Telah berhasil dipertahankan dihadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Komputer pada Program Studi Informatika, Institut Bisnis dan Teknologi Indonesia.

Ditetapkan di : Denpasar
Tanggal : 13 Juni 2026

DEWAN PENGUJI

Pembimbing I :

Ir. I Made Dwi Putra Asana, S.Kom., M.T., ()

Penguji :

I Gusti Ayu Agung Mas Aristamy, S.TI., M.Kom.. ()

Penguji :

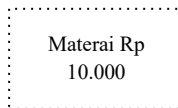
Dr. Ir. Komang Kurniawan Widiartha, S.Kom., M.Cs., ()

HALAMAN PERNYATAAN ORISINALITAS

Judul : **DETEKSI ANOMALI MENGGUNAKAN LSTM-AUTOENCODER PADA DATA LOG EVENT WAZUH**
Nama : **I Gede Arya Restyandika Pramana Putra**
NIM : **2201010325**
Program Studi : **Informatika**

Dengan ini saya menyatakan bahwa Skripsi ini merupakan karya saya sendiri, semua sumber baik yang dikutip maupun dirujuk telah saya cantumkan dengan benar dan sepanjang pengetahuan saya juga tidak terdapat karya yang sama pernah ditulis atau diterbitkan orang lain. Apabila terdapat permasalahan terhadap isi dari Skripsi ini maka akan menjadi tanggung jawab saya sepenuhnya.

Denpasar, 13 Juni 2026



(I Gede Arya Restyandika Pramana Putra)

HALAMAN PERNYATAAN PERSETUJUAN

PUBLIKASI SKRIPSI UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Institut Bisnis dan Teknologi Indonesia,
saya yang bertanda tangan di bawah ini:

Nama : I Gede Arya Restyandika Pramana Putra
NIM : 2201010325
Program Studi : Informatika
Jenis Karya : Skripsi

Demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Bisnis dan Teknologi Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty-Free Right*) atas Skripsi saya yang berjudul:

DETEKSI ANOMALI MENGGUNAKAN LSTM- AUTOENCODER PADA DATA LOG EVENT WAZUH

Beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Non-ekskutif ini Institut Bisnis dan Teknologi Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan mempublikasikan Skripsi saya tanpa meminta izin dari saya selama tetap mencantumkan saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Denpasar
Tanggal : 13 Juni 2026
Yang menyatakan

Materai Rp
10.000

(I Gede Arya Restyandika Pramana Putra)

ABSTRAK

DETEKSI ANOMALI MENGGUNAKAN LSTM-AUTOENCODER PADA DATA LOG EVENT WAZUH

Nama Mahasiswa : I Gede Arya Restyandika Pramana Putra
NIM : 2201010325
Program Studi : Informatika
Dosen Pembimbing : Ir. I Made Dwi Putra Asana, S.Kom., M.T.
Bagus Kusuma Wijaya, S.E., M.A.B.

Seiring dengan meningkatnya volume data log pada sistem *Security Information and Event Management* (SIEM) Wazuh, administrator keamanan siber sering kali menghadapi tantangan berupa *alert fatigue* akibat tingginya angka alarm palsu (*false positive*) dari metode deteksi konvensional berbasis aturan (*rule-based*). Penelitian ini mengusulkan penerapan arsitektur *Deep Learning* berupa *Long Short-Term Memory* (LSTM) Autoencoder dengan paradigma *unsupervised learning* untuk mendeteksi anomali secara otomatis. Model ini dilatih secara eksklusif menggunakan sekuens data log operasional normal untuk mengenali pola rutinitas *server* kampus di Institut Bisnis dan Teknologi Indonesia (INSTIKI). Guna mengatasi kelemahan metrik evaluasi yang rawan mengalami lonjakan ekstrem akibat pembagian dengan nilai nol, penelitian ini memperkenalkan inovasi penyaringan berupa Gerbang Logika *Mean Absolute Error* (MAE) dengan batas toleransi 0.3 sebelum kalkulasi persentase *error* (MAPE). Hasil pengujian menunjukkan bahwa model mampu merumuskan nilai *adaptive threshold* yang presisi di angka 60%. Evaluasi kinerja sistem membuktikan efektivitas metode ini dengan raihan *Precision* sebesar 96.56%, *Recall* 77.97%, dan *F1-Score* mencapai 86.28%. Implementasi gerbang logika terbukti sukses menekan angka alarm palsu secara drastis, sehingga arsitektur yang dibangun tidak hanya cerdas dalam mengidentifikasi jejak serangan siber, tetapi juga siap digunakan sebagai lapisan perlindungan proaktif bagi infrastruktur jaringan kampus.

Kata kunci: Deteksi Anomali, LSTM-Autoencoder, SIEM Wazuh, Keamanan Siber.

ABSTRACT

ANOMALY DETECTION USING LSTM-AUTOENCODER ON WAZUH EVENT LOG DATA

Author : I Gede Arya Restyandika Pramana Putra
Student ID : 2201010325
Study Program : Informatics
Supervisor : Ir. I Made Dwi Putra Asana, S.Kom., M.T.
Bagus Kusuma Wijaya, S.E., M.A.B.

Along with the increasing volume of log data in the Wazuh Security Information and Event Management (SIEM) system, cybersecurity administrators frequently face alert fatigue due to the high rate of false positives generated by conventional rule-based detection methods. This study proposes the implementation of a Deep Learning architecture, specifically the Long Short-Term Memory (LSTM) Autoencoder, utilizing an unsupervised learning paradigm for automated anomaly detection. The model was exclusively trained using sequences of normal operational log data to recognize the routine server patterns at the Institut Bisnis dan Teknologi Indonesia (INSTIKI). To address the vulnerability of evaluation metrics to asymptotic explosions caused by zero-division, this research introduces an innovative filtering mechanism known as the Mean Absolute Error (MAE) Logic Gate with a 0.3 tolerance limit prior to calculating the percentage error (MAPE). The testing results demonstrate that the model successfully established a precise adaptive threshold at 60%. The system's performance evaluation validates the effectiveness of this method, achieving a Precision of 96.56%, a Recall of 77.97%, and an F1-Score of 86.28%. The application of the logic gate significantly reduced false alarms, proving that the developed architecture is not only intelligent in identifying cyberattack footprints but also ready to be implemented as a proactive protection layer for the campus network infrastructure.

Keywords: Anomaly Detection, LSTM-Autoencoder, Wazuh SIEM, Cybersecurity.

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa, karena berkat rahmat-Nya Skripsi dengan judul “Deteksi Anomali Menggunakan LSTM-Autoencoder Pada Data Log Event Wazuh” dapat diselesaikan.

Penulis menyadari dalam menyelesaikan laporan Skripsi ini tidak terlepas dari berbagai pihak yang telah membantu dalam membimbing dan membantu baik secara moral, materi, dan ilmu pengetahuan. Untuk itu penulis menyampaikan terimakasih kepada:

1. Bapak Dr. I Dewa Made Krishna Muku, S.T., M.T., sebagai Rektor Institut Bisnis dan Teknologi Indonesia.
2. Bapak Dr. Ir. Komang Kurniawan Widiartha, M.Cs. sebagai Dekan Fakultas Teknologi dan Informatika.
3. Bapak I Gusti Agung Indrawan, S.T., M.T. sebagai Koordinator Program Studi Informatika Institut Bisnis dan Teknologi Indonesia.
4. Bapak Ir. I Made Dwi Putra Asana, S.Kom., MT., sebagai Dosen Pembimbing I.
5. Bapak Bagus Kusuma Wijaya, S.E., M.A.B., sebagai Dosen Pembimbing II.

Untuk itu kritik dan saran dari semua pihak untuk penyempurnaan sangat penulis hargai.

Denpasar, 13 Juni 2026

Penulis

DAFTAR ISI

Judul.....	i
HALAMAN PENGESAHAN.....	ii
HALAMAN PERSETUJUAN.....	iii
HALAMAN PERNYATAAN ORISINALITAS.....	iv
HALAMAN PERNYATAAN PERSETUJUAN.....	v
ABSTRAK.....	vi
ABSTRACT.....	vii
KATA PENGANTAR.....	viii
DAFTAR ISI.....	ix
DAFTAR TABEL.....	xiii
DAFTAR GAMBAR.....	xiv
DAFTAR LAMPIRAN.....	xv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	3
1.3 Batasan Masalah.....	3
1.4 Tujuan Penelitian.....	3
1.5 Manfaat Penelitian.....	4
1.6 Sistematika Penulisan.....	5
BAB II TINJAUAN PUSTAKA.....	7
2.1 Penelitian Terdahulu.....	7
2.2 <i>Cyber Security & Intrusion Detection System</i>	13
2.3 <i>Security Information and Event Management (SIEM)</i>	14
2.4 Wazuh.....	15
2.5 <i>Flowchart</i>	15
2.6 <i>Deep Learning</i>	17
2.7 <i>Recurrent Neural Network (RNN) dan LSTM</i>	18
2.8 <i>Auto Encoder & Unsupervised Learning</i>	20
2.9 <i>Arsitektur LSTM-Autoencoder</i>	21
2.10 <i>Metode Pemodelan CRISP-DM</i>	23
2.11 <i>Preprocessing Data</i>	25
2.12 <i>Metode Evaluasi</i>	28
2.12.1 <i>Confusion Matrix</i>	28
2.12.2 <i>Measurement Metrics</i>	28

2.13	<i>Python</i>	30
2.14	<i>Google Colab</i>	31
BAB III ANALISIS DAN PERANCANGAN		33
3.1	Tempat dan Waktu Penelitian	33
3.2	Teknik Pengumpulan Data	33
3.2.1	Data Primer	33
3.2.2	Data Sekunder	34
3.3	Analisis Sistem	34
3.3.1	Kebutuhan Fungsional	35
3.3.2	Kebutuhan <i>Non Fungsional</i>	35
3.4	Tahapan Penelitian	35
3.5	Gambaran Umum Sistem	36
3.6	Metode Perancangan Sistem	37
3.7	<i>Business Understanding</i>	38
3.8	<i>Data Understanding</i>	38
3.9	<i>Data Preparation</i>	40
3.9.1	<i>Feature Selection</i>	42
3.9.2	<i>Sorting</i>	43
3.9.3	<i>Encoding</i>	43
3.9.4	<i>Scaling</i>	44
3.9.5	<i>Agent Based Sequentialization</i>	45
3.10	<i>Modeling</i>	46
3.11	<i>Evaluation</i>	53
3.11.1	Penentuan <i>Threshold</i>	53
3.11.2	<i>Confusion Matrix</i>	54
3.11.3	<i>Performance Measurement Metrics</i>	54
3.12	<i>Deployment</i>	56
BAB IV IMPLEMENTASI DAN PEMBAHASAN		57
4.1	Data Preprocessing	57
4.1.1	<i>Feature Selection</i>	57
4.1.2	<i>Encoding</i>	58
4.1.3	<i>Agent Based Sequentialization</i>	59
4.1.4	<i>Data Cleansing & Splitting</i>	60
4.1.5	<i>Scaling</i>	61
4.2	Arsitektur dan Pelatihan Model LSTM-Autoencoder	62
4.2.1	Arsitektur Model	62

4.2.2	Skenario Pelatihan	63
4.3	Evaluasi Model dan Penentuan <i>Threshold</i>	65
4.3.1	Pengukuran <i>Reconstruction Error</i> (MAPE).....	66
4.3.2	Penentuan <i>Adaptive Threshold</i>	68
4.4	Pengujian pada Lingkungan Nyata (<i>Real-World Inference</i>)....	70
4.4.1	Skenario Fase Deployment	70
4.4.2	Evaluasi Kinerja Model pada Aktivitas Normal dan Anomali	72
4.4.3	Evaluasi Kinerja Model pada Data Observasi.....	74
4.5	Kontribusi Model terhadap Keamanan Infrastruktur.....	78
BAB V	PENUTUP	81
5.1	Simpulan	81
5.2	Saran.....	83
DAFTAR PUSTAKA		86
LAMPIRAN.....		90
BIODATA PENULIS		92

DAFTAR TABEL

Tabel 2.1 Perbandingan Penelitian.....	10
Tabel 2.2 Simbol <i>flowchart</i>	16
Tabel 3.1 Contoh Data Hasil Pembersihan dan Seleksi Fitur	42
Tabel 3.2 Sampel Data Hasil Pengurutan Kronologis.....	43
Tabel 3.3 Hasil Transformasi Data Menjadi Numerik (<i>Encoded</i>).....	44
Tabel 3.4 Hasil Normalisasi Data dengan Min-Max Scaling.....	45
Tabel 3.5 Contoh Sampel <i>Input Sequence</i> (<i>Window Size</i> = 5)	46
Tabel 3.6 Tabel perbandingan data Normal & Serangan	55

DAFTAR GAMBAR

Gambar 2.1 Alur CRISP-DM	24
Gambar 2.2 Logo <i>Python</i>	30
Gambar 3.1 Dokumentasi <i>Raw Data</i>	34
Gambar 3.2 Alur penelitian.	36
Gambar 3.3 Gambaran umum sistem.....	37
Gambar 3.4 Metode CRISP-DM	38
Gambar 3.5 Proses seleksi fitur data.....	39
Gambar 3.6 Alur <i>pre-processing</i> data.....	41
Gambar 3.7 Alur Kerja Arsitektur LSTM-AE	47
Gambar 3.8 Alur modeling & Spliting data.....	52
Gambar 4.1 Output <i>feature selection</i>	58
Gambar 4.2 Output Proses <i>Encoding</i>	58
Gambar 4.3 Output Proses <i>Sequentialization</i>	59
Gambar 4.4 Output Proses <i>Data Cleansing & Spliting</i>	60
Gambar 4.5 Output Proses <i>Scaling</i>	62
Gambar 4.6 Arsitektur Model LSTM-Autoencoder (<i>ModelSummary</i>)	63
Gambar 4.7 Grafik <i>loss history</i>	64
Gambar 4.8 Breakdown Kalkulasi <i>Sequence</i>	67
Gambar 4.9 Adaptive Threshold Model.	69
Gambar 4.10 Data Hasil <i>Preprocessing Pipeline</i>	71
Gambar 4.11 Grafik Distribusi Error (MAPE) pada Evaluasi Sekuens Log Normal	73
Gambar 4.12 Lonjakan Ekstrem Distribusi Error (MAPE) pada Evaluasi Sekuens Anomali	74
Gambar 4.13 Data JSON Observasi Format JSON Lines	75
Gambar 4.14 Laporan Hasil Deteksi dan Grafik Distribusi Skor MAPE pada Data Observasi.	76
Gambar 4.15 <i>Confusion Matrix</i> pada Data Observasi.	77

DAFTAR LAMPIRAN

Lampiran 1. Lampiran Bukti Dokumentasi

HALAMAN INI SENGAJA DIKOSONGKAN

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era transformasi digital saat ini, keamanan infrastruktur teknologi informasi telah menjadi prioritas utama bagi setiap institusi pendidikan maupun organisasi bisnis. Untuk memitigasi ancaman siber yang semakin kompleks, banyak organisasi menerapkan sistem *Security Information and Event Management* (SIEM) seperti Wazuh sebagai standar pemantauan keamanan. Sistem ini bertugas merekam seluruh jejak aktivitas sistem dan perilaku pengguna dalam bentuk data log yang terpusat. Namun, penerapan teknologi ini memunculkan tantangan baru berupa ledakan volume data atau yang dikenal sebagai fenomena *Big Data*. Menurut penelitian survei oleh (Landauer dkk., 2023) volume data log yang masif seringkali melampaui kemampuan manusia untuk menganalisisnya secara manual. Akibatnya, banyak informasi keamanan krusial yang mungkin terlewatkan karena tertimbun oleh jutaan baris data yang masuk setiap harinya. Keterbatasan kapasitas analisis manusia inilah yang mendorong kebutuhan akan metode otomatis cerdas dalam pemantauan aktivitas sistem.

Selama ini, metode deteksi ancaman pada sistem SIEM umumnya masih bergantung pada pendekatan *rule-based* atau pencocokan tanda tangan (*signature matching*). Dalam metode ini, administrator harus mendefinisikan aturan statis secara manual, seperti contohnya memberikan peringatan jika terjadi kegagalan login sebanyak lima kali. Meskipun sederhana, pendekatan konvensional ini memiliki kelemahan yang signifikan dalam menghadapi serangan modern yang dinamis. Sebagaimana dijelaskan oleh (Thakkar dan Lohiya, 2023) dalam surveinya mengenai sistem deteksi intrusi, metode berbasis *signature* cenderung menghasilkan tingkat peringatan palsu (*false positive*) yang tinggi dan gagal mengenali varian serangan yang dimodifikasi. Selain itu, sistem ini tidak mampu mendeteksi jenis serangan baru (*zero-day attacks*) yang polanya belum terdaftar dalam *database* tanda tangan (Wang dkk., 2024). Keterbatasan ini menegaskan bahwa pendekatan statis tidak lagi memadai untuk menangani variasi serangan siber yang terus berkembang.

Permasalahan umum tersebut tercermin secara nyata pada kondisi operasional di lingkungan Institut Bisnis dan Teknologi Indonesia (INSTIKI). Berdasarkan hasil observasi langsung ke bagian SIEM, sistem SIEM yang berjalan saat ini menghadapi kendala beban kerja yang signifikan. Rata-rata volume log harian yang dianalisis tercatat sebesar 385 kejadian per hari, dengan fluktuasi aktivitas yang bervariasi antara

minimum 42 hingga maksimum 1.189 kejadian per hari. Banyaknya data yang masuk menyebabkan administrator kewalahan dan kesulitan membedakan antara aktivitas operasional rutin dengan indikasi serangan yang sebenarnya. Tingginya *noise* atau data sampah yang tidak relevan mengakibatkan proses deteksi anomali menjadi sangat lambat dan tidak efisien. Kondisi ini berpotensi meningkatkan risiko keamanan, di mana serangan siber yang sebenarnya bisa saja lolos dari pengawasan karena tertutup oleh ribuan log aktivitas harian.

Untuk mengatasi keterbatasan analisis manual dan metode berbasis aturan, pendekatan *Deep Learning* menawarkan solusi yang menjanjikan melalui analisis pola data. Data log event pada dasarnya memiliki karakteristik sekuensial, di mana suatu kejadian log saling berkaitan dengan kejadian sebelumnya dalam urutan waktu tertentu. Oleh karena itu, arsitektur *Long Short-Term Memory* (LSTM) dinilai sangat tepat untuk diterapkan karena kemampuannya dalam mempelajari ketergantungan jangka panjang (*long-term dependencies*) pada data deret waktu. Sebagaimana dibuktikan dalam penelitian *DeepLog* oleh (Altunay dan Albayrak, 2023) LSTM mampu memodelkan pola perilaku normal sistem dengan mempelajari urutan kejadian log secara otomatis dan terbukti mengungguli metode tradisional seperti *Principal Component Analysis* (PCA). Dengan kemampuan ini, sistem dapat mengenali konteks dari suatu aktivitas, bukan hanya melihat log sebagai baris teks yang berdiri sendiri. Hal ini memungkinkan deteksi yang lebih akurat terhadap anomali yang menyimpang dari pola kebiasaan sistem.

Tantangan teknis lain yang dihadapi dalam penelitian ini adalah ketiadaan label data serangan (*unlabeled data*) pada log historis di INSTIKI. Melabeli ribuan data log secara manual sebagai "serangan" atau "normal" merupakan pekerjaan yang tidak efisien dan rentan terhadap kesalahan manusia. Oleh karena itu, penelitian ini mengusulkan penerapan metode *Unsupervised Learning* menggunakan arsitektur gabungan LSTM dan Autoencoder (LSTM-AE). Konsep ini mengacu pada penelitian (Kim dan Lee, 2024), yang membuktikan bahwa LSTM-Autoencoder efektif digunakan untuk deteksi anomali pada data *time-series* tanpa memerlukan label. Metode ini bekerja dengan cara melatih model hanya menggunakan data aktivitas normal untuk mempelajari karakteristik dasar sistem (*baseline*). Nantinya, setiap aktivitas baru yang gagal direkonstruksi dengan baik oleh model akan dianggap sebagai anomali atau potensi serangan. Dengan pendekatan ini, diharapkan dapat terbangun sebuah sistem deteksi otomatis yang mampu membantu administrator di INSTIKI dalam memantau keamanan server dan sistem secara lebih efektif dan efisien.

1.2 Rumusan Masalah

Berdasarkan latar belakang penelitian yang sudah disampaikan penulis, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

- 1) Bagaimana menerapkan metode *LSTM-Autoencoder* untuk mendeteksi anomali pada data log SIEM Wazuh di INSTIKI?
- 2) Bagaimana kinerja model *LSTM-Autoencoder* dalam mendeteksi serangan berdasarkan hasil evaluasi *Precision*, *Recall*, dan *F1-Score*?

1.3 Batasan Masalah

Demi menjaga alur penelitian agar tetap terarah, penulis telah menetapkan batasan masalah yang dituliskan pada poin berikut ini:

- 1) Penelitian ini menggunakan data log dari SIEM Wazuh di INSTIKI terhitung mulai tanggal 19 Juni 2025 hingga 30 Juli 2025 dengan jumlah keseluruhan 10.000 baris data. Nantinya diproses menggunakan algoritma *Deep Learning LSTM-Autoencoder*
- 2) Sistem yang dibangun berupa model yang hanya memberikan status deteksi, tanpa disertai antarmuka pengguna maupun fitur pencegahan serangan otomatis.

1.4 Tujuan Penelitian

Mengacu pada rumusan masalah yang telah ditetapkan, maka tujuan utama yang ingin dicapai dalam penelitian ini adalah:

- 1) Mengimplementasikan arsitektur *Deep Learning* berbasis *Long Short-Term Memory* (LSTM) *Autoencoder* dengan pendekatan *Unsupervised Learning* untuk mendeteksi anomali pada data log event SIEM Wazuh di Institut Bisnis dan Teknologi Indonesia (INSTIKI).
- 2) Mengukur dan mengevaluasi kinerja model deteksi anomali yang dibangun dalam membedakan aktivitas normal dan serangan, berdasarkan analisis metrik evaluasi *Precision*, *Recall*, dan *F1-Score*.

1.5 Manfaat Penelitian

Adapun manfaat yang diharapkan dari hasil penelitian ini adalah sebagai berikut:

- 1) Tersedianya model AI deteksi anomali menggunakan metode LSTM-Autoencoder pada data log Wazuh yang diharapkan dapat membantu administrator sistem di Institut Bisnis dan Teknologi Indonesia (INSTIKI) dalam melakukan pemantauan aktivitas sistem secara otomatis dan efisien.
- 2) Memberikan informasi terukur mengenai kinerja model berdasarkan hasil evaluasi *Precision*, *Recall*, dan *F1-Score*, sehingga dapat menjadi tolok ukur keandalan metode *Unsupervised Learning* dalam mendeteksi ancaman tanpa bergantung pada pelabelan data manual.
- 3) Menjadi referensi ilmiah bagi pengembangan sistem deteksi intrusi berbasis *Deep Learning* pada kasus data log keamanan riil di lingkungan institusi pendidikan.

1.6 Sistematika Penulisan

Penulisan laporan penelitian ini dibagi menjadi lima bab, yaitu:

BAB I PENDAHULUAN

Mengulas tentang latar belakang, masalah, batasan masalah, tujuan, manfaat, dan sistematika penulisan.

BAB II TINJAUAN PUSTAKA

Pada bab ini mengulas tentang penelitian terdahulu yang pernah dilakukan serta teori-teori terkait *Deep Learning* dan metode LSTM-Autoencoder.

BAB III ANALISIS DAN PERANCANGAN

Pada bab ini mengulas mengenai metode yang dilakukan selama proses penelitian, mulai dari pengumpulan data, perancangan.

BAB IV IMPLEMENTASI DAN PEMBAHASAN

Bab ini membahas implementasi rancangan ulang ke dalam *prototype*. Analisis validasi *usability* juga disajikan.

BAB V PENUTUP

Bab ini berisi simpulan dan saran selama penelitian dilakukan

HALAMAN INI SENGAJA DIKOSONGKAN

BAB II

TINJAUAN PUSTAKA

2.1 Penelitian Terdahulu

Untuk memberikan landasan teori yang kokoh, tinjauan pustaka ini menyajikan analisis mendalam terhadap lima jurnal penelitian utama yang diterbitkan dalam kurun waktu lima tahun terakhir. Pemilihan literatur ini dilakukan secara selektif oleh penulis dengan memprioritaskan studi yang relevan dengan topik deteksi anomali pada sistem keamanan jaringan dan analisis data *log*. Berbagai spektrum metodologi dikaji dalam ulasan ini, mulai dari pendekatan konvensional berbasis *Machine Learning* klasik hingga teknik mutakhir berbasis *Deep Learning*. Ulasan ini tidak hanya menyoroti keberhasilan metode *Supervised Learning* dan model *Hybrid* yang telah ada, tetapi juga mengidentifikasi keterbatasan mendasar terkait ketergantungan pada data berlabel yang masif. Selain itu, analisis ini bertujuan untuk memetakan pergeseran tren teknologi keamanan siber dari deteksi berbasis tanda tangan manual menuju mekanisme deteksi perilaku otomatis. Dengan memahami kelebihan dan kekurangan dari penelitian-penelitian terdahulu tersebut, posisi dan kontribusi kebaruan (*novelty*) dari penelitian ini dapat dirumuskan secara lebih objektif. Oleh karena itu, berikut adalah uraian rinci mengenai metode, hasil, dan celah penelitian dari masing-masing studi yang menjadi acuan utama dalam penyusunan laporan penelitian ini.

Penelitian pertama yang menjadi acuan utama dilakukan oleh (Zhao dkk., 2023) dengan judul '*Robust Log-Based Anomaly Detection with Hierarchical Contrastive Learning*', yang mengusulkan pendekatan *unsupervised* mutakhir untuk mengatasi tantangan variabilitas data *log* yang tidak berlabel. Dalam studi ini, peneliti mengembangkan model berbasis Hierarchical Contrastive Learning yang dirancang untuk menangkap representasi semantik *log* secara mendalam dan tahan (*robust*) terhadap perubahan format data. Metode ini bekerja dengan mempelajari kemiripan pola (*similarity*) dari data *log* operasional normal dan membedakannya dari pola yang menyimpang, tanpa bergantung pada label serangan. Berdasarkan evaluasi eksperimental, model ini terbukti menghasilkan kinerja yang sangat tinggi dengan mencatatkan rata-rata F1-Score mencapai 99% pada dataset publik seperti HDFS dan BGL. Hasil ini secara signifikan mengungguli metode statistik tradisional seperti PCA (*Principal Component Analysis*) dan *Invariant Mining*, sekaligus menegaskan efektivitas penggunaan arsitektur *Deep Learning*

modern dalam memisahkan pola data normal dan serangan secara otomatis tanpa memerlukan pelabelan manual yang memakan biaya.

Selanjutnya, kajian komprehensif oleh (Landauer dkk., 2023) melalui survei terhadap 62 studi literatur memetakan dominasi arsitektur LSTM serta urgensi pergeseran ke metode *Unsupervised Learning* guna mengatasi kendala ketiadaan label pada data log modern. Penelitian ini tidak hanya menyoroti keterbatasan metode konvensional, tetapi juga memberikan kritik tajam terhadap penggunaan dataset standar seperti HDFS yang dianggap terlalu sederhana untuk memvalidasi model. Untuk membuktikan klaim tersebut, mereka melakukan eksperimen menggunakan algoritma heuristik sederhana pada dataset HDFS yang terbukti mampu mencapai F1-Score sebesar 98,86% dan akurasi 99,93%, menyaingi kinerja model *Deep Learning* yang kompleks. Temuan empiris ini menegaskan bahwa metrik akurasi tinggi pada dataset publik tidak selalu mencerminkan kehandalan sistem di lingkungan nyata yang dinamis. Oleh karena itu, studi ini memberikan legitimasi teoretis yang kuat bagi penelitian ini untuk menerapkan pendekatan tanpa pengawasan pada data operasional riil, bukan sekadar bergantung pada *benchmark* standar.

Sebagai perbandingan dari segi kerumitan model gabungan, penelitian yang dilakukan oleh (Altunay dan Albayrak, 2023) dengan judul '*A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks*' mengusulkan penggabungan dua arsitektur *Deep Learning*, yaitu *Convolutional Neural Network* (CNN) dan *Long Short-Term Memory* (LSTM). Dalam pendekatan ini, lapisan CNN digunakan untuk mengekstraksi fitur-fitur spasial yang kompleks dari trafik jaringan industri, sedangkan lapisan LSTM dimanfaatkan untuk menangkap dependensi waktu (*temporal dependencies*). Berdasarkan evaluasi menggunakan dataset benchmark UNSW-NB15, model hibrida ini terbukti menghasilkan akurasi yang luar biasa hingga 99,56%. Meskipun mampu menekan tingkat kesalahan secara signifikan, implementasi metode ini memiliki kompleksitas komputasi yang tinggi dan sepenuhnya bersifat *supervised learning*, yang menuntut ketersediaan dataset berlabel lengkap dalam jumlah besar. Hal ini berbeda dengan karakteristik data log di lapangan (seperti di tempat studi kasus ini) yang seringkali belum memiliki label serangan.

Sebagai studi evaluasi kritis terhadap klaim performa tinggi pada penelitian-penelitian sebelumnya, (Le dan Zhang, 2022) dalam artikelnya '*Log-based anomaly detection with deep learning: how far are we?*' melakukan pengujian ulang secara mendalam terhadap lima model *Deep Learning* populer, termasuk *DeepLog* dan *LogRobust*. Penelitian ini

membongkar fakta bahwa akurasi tinggi (F-measure di atas 0,9) yang sering dilaporkan pada dataset HDFS sebagian besar disebabkan oleh kesalahan metodologis dalam pemilihan data latih, yaitu penggunaan strategi pengacakan (*random selection*) yang memicu kebocoran data (*data leakage*) di mana informasi masa depan bocor ke fase pelatihan. Ketika dievaluasi ulang menggunakan pemisahan data secara kronologis yang ketat sesuai kondisi riil, kinerja model-model tersebut terbukti menurun drastis, di mana F-measure *DeepLog* tercatat jatuh dari 0,927 menjadi 0,426 pada dataset BGL. Temuan ini menekankan bahwa tantangan deteksi anomali pada data *log* belum sepenuhnya terpecahkan dan menuntut metode evaluasi yang lebih realistis dengan mempertimbangkan ketidakseimbangan kelas (*class imbalance*) serta sensitivitas terhadap kesalahan *parsing* (*noise*).

Sebagai bukti relevansi arsitektur yang diusulkan pada penelitian modern, studi terbaru oleh (Kim dan Lee, 2024) dengan judul '*LSTM Autoencoder-Based Insider Abnormal Behavior Detection Using De-Identified Data*' menerapkan arsitektur LSTM-Autoencoder untuk mendeteksi anomali perilaku pengguna yang berpotensi menjadi ancaman keamanan (*insider threat*). Penelitian ini mengonfirmasi bahwa penggunaan lapisan LSTM dalam blok *Encoder-Decoder* mampu mempelajari pola sekuensial dari aktivitas normal pengguna dan mendeteksi penyimpangan perilaku dengan akurasi mencapai 94% tanpa memerlukan dataset serangan berlabel. Hasil eksperimen mereka menunjukkan bahwa metode ini jauh lebih unggul dalam menangani data yang bersifat deret waktu (*time-series*) dibandingkan metode deteksi statis, sekaligus memvalidasi efektivitas pendekatan *Unsupervised Learning* dalam lingkungan keamanan siber yang sensitif terhadap privasi data.

Berdasarkan Berdasarkan pemetaan terhadap kelima studi literatur di atas, penelitian ini menempatkan posisinya untuk mengisi celah metodologis antara kinerja tinggi model *Deep Learning* dan kebutuhan operasional di lapangan yang minim label. Penelitian oleh (Altunay dan Albayrak, 2023) menunjukkan bahwa model berbasis *Supervised Learning* mampu mencapai akurasi yang sangat tinggi, namun ketergantungannya pada ketersediaan dataset berlabel lengkap sulit dipenuhi di lingkungan dinamis seperti INSTIKI. Di sisi lain, studi terbaru oleh (Kim dan Lee, 2024) serta (Zhao dkk., 2023) telah memvalidasi bahwa pendekatan *Unsupervised* dengan arsitektur *Autoencoder* sangat efektif untuk menangani data tanpa label, namun efektivitasnya seringkali baru teruji pada dataset publik yang bersih. Hal ini menjadi perhatian serius mengingat temuan kritis dari (Le dan Zhang,

2022) serta (Landauer dkk., 2023) yang memberikan peringatan bahwa penggunaan dataset publik standar (seperti HDFS) sering kali bias dan tidak mencerminkan kompleksitas serangan nyata. Oleh karena itu, penelitian ini mengadopsi ketangguhan arsitektur LSTM-Autoencoder (sebagaimana divalidasi oleh Kim dan Lee, 2024) untuk diterapkan secara spesifik pada data log operasional riil dari Wazuh. Kebaruan (*novelty*) penelitian ini terletak pada penerapan standar evaluasi kronologis yang ketat guna menghindari kebocoran data (*data leakage*) serta pembuktian ketahanan model terhadap *noise* pada lingkungan produksi nyata, yang bertujuan untuk menjamin validitas sistem saat diimplementasikan di dunia nyata.

Perbandingan penelitian yang akan dilakukan dengan penelitian terdahulu dapat dilihat pada tabel 2.1

Tabel 2.1 Perbandingan Penelitian

Indikator	Penelitian		
	Penelitian I	Penelitian II.	Penelitian III
Topik	Deteksi anomali log yang tangguh (<i>Robust</i>) menggunakan <i>Contrastive Learning</i> .	Survei metode <i>Deep Learning</i> pada data log & validasi dataset	Sistem Deteksi Intrusi (IDS) Hibrida untuk jaringan IoT Industri.
Dataset	Dataset HDFS dan BGL.	HDFS (untuk studi kasus validasi)	<i>UNSW-NB15</i> dan <i>X-IIoTID</i> .
Metode Penelitian	<i>Hierarchical Contrastive Learning</i> : Pendekatan <i>unsupervised</i> yang mempelajari representasi semantik log secara hierarkis untuk menangani variabilitas data	Studi Literatur Sistematis (SLR) & Eksperimen Validasi menggunakan metode Heuristik sederhana.	<i>Hybrid CNN + LSTM</i> : Menggabungkan <i>CNN</i> untuk ekstraksi fitur spasial trafik dan <i>LSTM</i> untuk menangkap urutan waktu serangan (<i>Supervised</i>).

	log yang kompleks.		
Metode Evaluasi	<i>Precision, Recall, F1-Score.</i>	F1-Score, Accuracy	<i>Accuracy, Precision, Recall, F1-Score.</i>
Hasil	Mencapai F1-Score 99% pada dataset HDFS. Metode ini terbukti jauh lebih tangguh terhadap <i>noise</i> dan perubahan format log dibandingkan metode <i>Deep Learning</i> konvensional.	Penelitian ini membuktikan bahwa algoritma sederhana bisa mencapai <i>F1-Score</i> 98.86% pada dataset HDFS,	Mencapai akurasi 99.56% pada dataset UNSW-NB15. Sangat efektif menekan <i>False Positive</i> , namun memiliki beban komputasi tinggi dan bergantung penuh pada data berlabel.
Indikator	Penelitian		
	Penelitian IV	Penelitian V	Yang akan dilakukan
Topik	Evaluasi kritis kinerja <i>Deep Learning</i> pada deteksi anomali log	Deteksi perilaku abnormal (<i>Insider Threat</i>) dengan tetap menjaga privasi pengguna melalui teknik de-identifikasi data.	Analisis Anomali <i>Log Event</i> pada SIEM Wazuh (Studi Kasus: INSTIKI)
Dataset	HDFS, BGL	Data aktivitas pengguna internal (organisasi) yang telah melalui proses (<i>De-identified Data</i>) menggunakan	Data Log Riil (Privat): Bersumber langsung dari <i>server</i> SIEM Wazuh di Institut Bisnis dan Teknologi

		alat ARX Data Anonymization.	Indonesia (INSTIKI)
Metode Penelitian	Re-evaluasi Model (<i>Benchmarking</i>): Menguji ulang DeepLog & LogRobust dengan pemisahan data kronologis ketat.	LSTM-Autoencoder (<i>Unsupervised Learning</i>). menggunakan arsitektur LSTM-AE untuk mempelajari pola normal perilaku pengguna dari data yang sudah disamarkan.	LSTM-Autoencoder (<i>Unsupervised</i>): Menggunakan LSTM sebagai <i>Encoder-Decoder</i> untuk merekonstruksi pola normal data log sekuensial.
Metode Evaluasi	<i>F1-Measure, AUC</i>	Metode Privasi: Mengukur <i>Re-identification Probability</i> dan <i>Attack Success Probability</i> (menggunakan 3 model serangan). Metode Performansi: Akurasi (<i>Accuracy</i>) dan <i>F1-Score</i> .	<i>Reconstruction Error (MAE/MSE)</i> untuk penentuan <i>Threshold</i> , serta <i>Precision, Recall, F1-Score</i> untuk validasi deteksi.
Hasil	Menemukan bahwa metode pemisahan data acak menyebabkan bias (<i>data leakage</i>); Kinerja DeepLog pada BGL turun	Model berhasil mencapai Akurasi 94% dan F1-Score 97% dalam mendeteksi perilaku abnormal, dengan	Sistem deteksi otomatis yang mampu mengenali serangan baru (<i>zero-day</i>) atau anomali perilaku tanpa memerlukan

	drastis dari F1 >0.9 menjadi ~0.4 saat dievaluasi dengan benar.	probabilitas re-identifikasi data yang sangat rendah (~1.2%). Hal ini membuktikan bahwa metode LSTM-AE tetap efektif mendeteksi anomali meskipun data telah disamarkan demi privasi.	pelabelan data manual, serta tervalidasi pada trafik jaringan kampus yang dinamis.

2.2 Cyber Security & Intrusion Detection System

Keamanan siber merupakan upaya fundamental untuk melindungi kerahasiaan, integritas, dan ketersediaan data dari ancaman akses ilegal. Sebagai komponen pertahanan utama, Sistem Deteksi Intrusi (*Intrusion Detection System* atau IDS) berfungsi memantau lalu lintas jaringan secara terus-menerus guna mengidentifikasi aktivitas mencurigakan yang berhasil menembus pertahanan preventif seperti *firewall* (Liao dkk., 2013; Sarker dkk., 2020). Berdasarkan area pengawasannya, IDS diklasifikasikan menjadi *Network-based IDS* (NIDS) yang menganalisis paket data di seluruh segmen jaringan, dan *Host-based IDS* (HIDS) yang memantau aktivitas internal pada perangkat server (Thakkar dan Lohiya, 2020).

Dalam operasionalnya, metode deteksi IDS terbagi menjadi dua pendekatan utama: *Misuse Detection* dan *Anomaly Detection*. *Misuse Detection* bekerja dengan mencocokkan pola serangan yang sudah dikenal (*signature-based*), sehingga efektif untuk ancaman lama namun lemah terhadap serangan baru. Sebaliknya, *Anomaly Detection* berfokus pada pemetaan perilaku normal sistem dan menandai penyimpangan aktivitas sebagai potensi ancaman. Pendekatan berbasis anomali inilah yang kini menjadi fokus pengembangan riset keamanan siber modern karena kemampuannya mendeteksi serangan jenis baru (*zero-day attack*) yang belum teridentifikasi sebelumnya (Ahmad dkk., 2021).

Cyber Security menjadi landasan fundamental dalam penelitian ini, mengingat tujuan utama dari pengembangan model deteksi anomali adalah untuk memitigasi risiko serangan terhadap infrastruktur teknologi informasi. Pemahaman mengenai prinsip kerahasiaan, integritas, dan ketersediaan (CIA Triad) serta vektor serangan modern sangat diperlukan untuk mengidentifikasi pola ancaman yang mungkin muncul dalam data log, sehingga sistem yang dibangun benar-benar relevan dalam konteks pertahanan jaringan institusi.

2.3 *Security Information and Event Management (SIEM)*

Security Information and Event Management (SIEM) merupakan solusi keamanan terpadu yang menggabungkan dua fungsi utama manajemen keamanan, yaitu *Security Information Management (SIM)* dan *Security Event Management (SEM)*. Menurut studi terbaru oleh (Islam dan Rafique, 2024), SIEM bekerja dengan cara mengumpulkan data log dari berbagai sumber infrastruktur TI seperti perangkat jaringan, server, dan aplikasi untuk kemudian dianalisis secara terpusat guna mendeteksi ancaman siber, memastikan kepatuhan regulasi, serta memfasilitasi respons insiden yang cepat. Dalam arsitektur ini, komponen SIM bertanggung jawab atas pengumpulan, pengarsipan, dan pelaporan data log historis, sedangkan komponen SEM berfokus pada pemantauan *real-time*, korelasi kejadian (*event correlation*), dan pengiriman peringatan (*alerting*) kepada administrator. Proses operasional SIEM dimulai dari tahap agregasi data (*data aggregation*), di mana log mentah dari berbagai format dikonversi ke dalam format standar melalui proses normalisasi. (Hidayasari dan Efendi, 2025) menjelaskan bahwa setelah data ternormalisasi, mesin SIEM akan melakukan korelasi peristiwa untuk menghubungkan serangkaian aktivitas yang tampak terisolasi menjadi satu indikasi serangan yang utuh. Kemampuan korelasi inilah yang membedakan SIEM dari alat manajemen log biasa, karena memungkinkan sistem untuk mendeteksi pola serangan kompleks yang tersebar di berbagai titik jaringan.

Salah satu implementasi SIEM modern yang bersifat *open-source* adalah Wazuh. Berbeda dengan SIEM tradisional yang hanya mengandalkan log jaringan, Wazuh mengintegrasikan kemampuan *Host-based Intrusion Detection System (HIDS)* melalui pemasangan agen pada setiap *endpoint*. (S & Mohankumar, 2024) dalam penelitiannya menunjukkan bahwa arsitektur berbasis agen ini memungkinkan Wazuh untuk melakukan pemantauan integritas file (*File Integrity Monitoring*), deteksi kerentanan, serta respons aktif terhadap serangan secara *real-time*. Pendekatan ini dinilai sangat efektif untuk lingkungan infrastruktur yang

dinamis karena memberikan visibilitas mendalam hingga ke level sistem operasi, bukan hanya pada lalu lintas jaringan semata. Konsep SIEM digunakan sebagai kerangka kerja operasional dalam penelitian ini karena fungsinya yang krusial dalam mengumpulkan dan menyentralisasi data log dari berbagai sumber agen yang tersebar. Tanpa penerapan sistem SIEM, analisis log akan terfragmentasi dan sulit dilakukan; oleh karena itu, teori ini mendasari proses pengumpulan dataset terpusat yang menjadi bahan bakar utama bagi model *Deep Learning* untuk mempelajari perilaku sistem secara menyeluruh

2.4 Wazuh

Wazuh merupakan platform keamanan siber berbasis sumber terbuka (*open-source*) yang mengintegrasikan kemampuan *Security Information and Event Management* (SIEM) dan *Extended Detection and Response* (XDR) dalam satu arsitektur terpadu. Menurut (RANGGA dkk., 2024) sistem ini dirancang untuk melakukan pemantauan keamanan secara *real-time* melalui mekanisme berbasis agen yang dipasang pada *endpoint*, yang berfungsi mengumpulkan data log, mendeteksi kerentanan, memantau integritas file, serta merespons insiden secara otomatis. Lebih lanjut, arsitektur Wazuh terdiri dari komponen utama berupa *Wazuh Agent* untuk koleksi data, *Wazuh Server* untuk analisis ancaman, *Wazuh Indexer* untuk pengindeksan data log, dan *Wazuh Dashboard* untuk visualisasi, menjadikannya solusi komprehensif yang tidak hanya mendeteksi serangan jaringan tetapi juga memberikan visibilitas mendalam terhadap anomali pada level sistem operasi (Hidayasari dan Efendi, 2025).

Platform Wazuh menjadi objek studi kasus utama sekaligus penyedia data (*data source*) dalam penelitian ini, mengingat penggunaannya yang telah berjalan di lingkungan operasional INSTIKI. Pemahaman mendalam mengenai arsitektur Wazuh, format log JSON yang dihasilkan, serta mekanisme agen-nya sangat penting untuk melakukan tahapan pra-pemrosesan data yang tepat, memastikan bahwa model LSTM-Autoencoder dilatih menggunakan atribut log yang valid dan representatif dari kondisi nyata di lapangan.

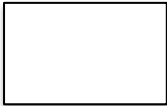
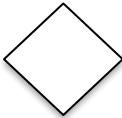
2.5 Flowchart

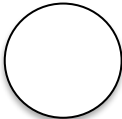
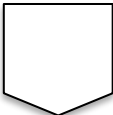
Flowchart adalah representasi grafis dari langkah-langkah prosedural suatu program yang disusun secara berurutan untuk menggambarkan logika penyelesaian masalah. Menurut (As dan Shalahudin, 2021), *flowchart* berfungsi sebagai alat bantu komunikasi antara analis sistem dan pengembang perangkat lunak untuk

memvisualisasikan alur kerja (*workflow*) sistem, mulai dari input, proses, hingga output yang dihasilkan. Penggunaan *flowchart* sangat krusial dalam tahap perancangan sistem karena memungkinkan pendeteksian kesalahan logika (*logic error*) dan ketidakefisienan alur sebelum proses pengodean (*coding*) dimulai.

Dalam standar penulisan teknis, simbol-simbol yang digunakan dalam *flowchart* telah dibakukan secara internasional (standar ANSI/ISO) untuk memastikan keseragaman pemahaman. Setiap simbol memiliki makna spesifik yang merepresentasikan jenis instruksi tertentu. Berikut adalah simbol-simbol utama yang digunakan dalam perancangan sistem ini, sebagaimana dijelaskan oleh (Indrajani, 2013):

Tabel 2.2 Simbol *flowchart*

Simbol Flowchart	Nama	Fungsi Simbol
	<i>Process</i>	Melambangkan operasi pemrosesan data, perhitungan aritmatika, atau instruksi penugasan komputer.
	<i>Terminator</i>	Menandakan awal (<i>Start</i>) atau akhir (<i>End</i>) dari suatu aliran proses dalam program.
	<i>Decision</i>	Menunjukkan titik pengambilan keputusan berdasarkan kondisi logika (Benar/Salah atau Ya/Tidak) yang akan memecah alur program.

	<i>Data</i>	Merepresentasikan proses pemasukan data (<i>input</i>) atau pengeluaran informasi (<i>output</i>) tanpa memandang perangkat fisiknya.
	<i>Document</i>	Menunjukkan sebuah dokumen ataupun laporan yang dihasilkan dari suatu proses.
	<i>On-page reference/connector</i>	Penghubung alur dalam halaman yang sama.
	<i>Flow</i>	Simbol penghubung antar <i>process</i> yang menunjukkan alur proses.
	<i>Off-page connector</i>	Penghubung alur <i>flowchart</i> pada halaman berbeda.

Penggunaan bagan alir (*flowchart*) diterapkan dalam penelitian ini sebagai alat visualisasi logika untuk memetakan tahapan metodologi yang kompleks, mulai dari pengambilan data mentah, proses pelatihan model, hingga mekanisme evaluasi. Visualisasi ini krusial untuk memastikan bahwa desain sistem deteksi anomali yang dirancang memiliki alur algoritma yang runtut, logis, dan dapat dipahami secara sistematis baik untuk keperluan implementasi kode maupun audit penelitian.

2.6 *Deep Learning*

Deep Learning merupakan evolusi mutakhir dari bidang *Machine Learning* yang dibangun berdasarkan arsitektur *Artificial Neural*

Networks (ANN) dengan struktur yang lebih kompleks dan mendalam. Menurut (LeCun dkk., 2015) (dalam Farzad & Gulliver, 2020), *Deep Learning* memungkinkan model komputasi yang terdiri dari banyak lapisan pemrosesan (*multiple processing layers*) untuk mempelajari representasi data dengan berbagai tingkat abstraksi. Hal ini berbeda dengan metode *Machine Learning* konvensional yang memiliki kemampuan terbatas dalam memproses data mentah secara langsung dan sangat bergantung pada keahlian manusia untuk merancang fitur (*feature engineering*) secara manual.

Keunggulan utama *Deep Learning* terletak pada kemampuannya untuk melakukan *automatic feature extraction* atau pembelajaran representasi. Sebagaimana dijelaskan dalam survei komprehensif oleh (Landauer dkk., 2023) algoritma *Deep Learning* secara otomatis menemukan pola rumit dan struktur fitur yang diperlukan untuk deteksi-seperti dalam kasus analisis data log-tanpa memerlukan campur tangan manusia untuk menentukan variabel mana yang penting. Karakteristik ini menjadikan *Deep Learning* sangat andal dalam menangani data berdimensi tinggi dan tidak terstruktur yang sering kali menjadi kendala bagi algoritma pembelajaran mesin tradisional. Pendekatan *Deep Learning* dipilih sebagai metode inti analisis karena kemampuannya yang superior dalam menangani data berskala besar dan berdimensi tinggi dibandingkan metode statistik konvensional. Dalam konteks log Wazuh yang memiliki pola non-linear dan kompleks, arsitektur jaringan saraf tiruan yang mendalam memungkinkan sistem untuk mengekstraksi fitur-fitur abstrak secara otomatis, memberikan akurasi deteksi anomali yang lebih tinggi tanpa bergantung pada rekayasa fitur manual yang kaku.

2.7 Recurrent Neural Network (RNN) dan LSTM

Recurrent Neural Network (RNN) adalah merupakan kelas khusus dari *Artificial Neural Network* (ANN) yang dirancang secara khusus untuk memproses data yang bersifat sekuensial atau berurutan, seperti deret waktu (*time-series*) dan data log aktivitas sistem. Menurut (Sherstinsky, 2020), pada penelitiannya dikatakan bahwa karakteristik utama yang membedakan RNN dari jaringan saraf konvensional adalah adanya mekanisme umpan balik (*feedback loops*) dalam arsitekturnya, yang memungkinkan informasi dari langkah waktu sebelumnya dipertahankan untuk memengaruhi *output* saat ini. Namun, RNN standar memiliki keterbatasan fundamental yang dikenal sebagai masalah gradien menghilang (*vanishing gradient problem*), di mana jaringan kesulitan mempelajari ketergantungan informasi jarak jauh (*long-term dependencies*) pada urutan data yang panjang.

Untuk mengatasi kelemahan tersebut, (Hochreiter dan Schmidhuber, 1997) memperkenalkan arsitektur *Long Short-Term Memory* (LSTM). LSTM dirancang dengan struktur sel memori kompleks yang mampu mengatur aliran informasi—menyimpan yang penting dan membuang yang tidak relevan—melalui mekanisme gerbang (*gating mechanism*). Secara matematis, proses pengolahan informasi pada satu unit sel LSTM pada waktu “ t ” didefinisikan melalui serangkaian persamaan berikut (Sherstinsky, 2020):

- 1) *Forget Gate*, tahap pertama adalah memutuskan informasi mana dari cell state sebelumnya (C_{t-1}) yang akan dibuang atau dilupakan. Keputusan ini diambil oleh fungsi sigmoid (σ) berdasarkan input saat ini (x_t) dan hidden state sebelumnya (h_{t-1}):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- 2) *Input Gate*, tahap selanjutnya adalah menentukan informasi baru yang akan disimpan ke dalam cell state. Proses ini melibatkan dua bagian: lapisan sigmoid (i_t) yang menentukan nilai mana yang akan diperbarui, dan lapisan tanh ($\tilde{C}_t$) yang membuat vektor kandidat nilai baru:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- 3) *Cell State Update*, kondisi memori lama (C_{t-1}) kemudian diperbarui menjadi kondisi baru (C_t) dengan cara mengalikan kondisi lama dengan *forget factor* (f_t), lalu menambahkannya dengan informasi baru yang telah diskalakan ($i_t \cdot \tilde{C}_t$):

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- 4) *Output Gate*, bagian terakhir, *output* dari sel (h_t) ditentukan berdasarkan *cell state* yang telah diperbarui. Bagian ini memutuskan seberapa banyak informasi internal yang akan diteruskan ke *layer* berikutnya:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Keterangan: σ adalah fungsi aktivasi sigmoid, W adalah matriks bobot (weight), dan b adalah vektor bias.

Adanya seluruh mekanisme gerbang matematis tersebut, LSTM mampu menangkap pola sintaksis dan semantik yang rumit dari urutan log yang panjang. Hal ini dikonfirmasi oleh (Landauer dkk., 2023) dalam

surveinya yang menempatkan LSTM sebagai arsitektur *Deep Learning* paling dominan untuk deteksi anomali pada data log sistem karena stabilitasnya dalam memodelkan urutan kejadian yang dinamis. Dalam konteks analisis log sistem yang bersifat kronologis, teori *Recurrent Neural Network* (RNN) dan pengembangannya, *Long Short-Term Memory* (LSTM), menjadi landasan komputasi utama penelitian ini. Kemampuan spesifik LSTM dalam mengelola memori jangka panjang (*cell state*) sangat krusial untuk mengatasi kelemahan RNN standar dalam menangkap ketergantungan temporal pada urutan log yang panjang, sehingga pola serangan yang tersebar dalam rentang waktu lama dapat dikenali dengan akurat

2.8 Auto Encoder & Unsupervised Learning

Unsupervised Learning atau pembelajaran tanpa pengawasan adalah paradigma dalam *Machine Learning* di mana model dilatih menggunakan data yang tidak memiliki label atau target keluaran yang eksplisit. Berbeda dengan *Supervised Learning* yang membutuhkan pasangan *input-output* yang telah dianotasi oleh manusia, *Unsupervised Learning* bertujuan untuk menemukan struktur tersembunyi, pola intrinsik, atau distribusi probabilitas dari data masukan itu sendiri. Menurut (Goodfellow, 2016), pendekatan ini sangat relevan untuk kasus pengolahan *Big Data* seperti log keamanan jaringan, di mana pelabelan manual terhadap jutaan rekaman data adalah hal yang tidak praktis dan memakan biaya tinggi.

Salah satu arsitektur *Neural Network* yang paling populer dalam domain *Unsupervised Learning* adalah Autoencoder. Secara konseptual, Autoencoder adalah jaringan saraf tiruan yang dilatih untuk menyalin *input* menjadi *output*. Arsitektur ini terdiri dari dua komponen utama: *Encoder* yang berfungsi mengompresi data masukan x menjadi representasi laten berdimensi rendah h , dan *Decoder* yang bertugas merekonstruksi kembali data dari representasi laten tersebut menjadi keluaran $\hat{x}$ yang semirip mungkin dengan *input* aslinya (Bank dkk., 2023).

Dalam konteks deteksi anomali, Autoencoder bekerja berdasarkan hipotesis bahwa anomali adalah kejadian langka yang sulit direkonstruksi. Model dilatih hanya menggunakan data normal untuk meminimalkan selisih antara *input* dan *output*. Ketika model dihadapkan pada data anomali yang belum pernah dilihat sebelumnya, model akan gagal merekonstruksinya dengan baik sehingga menghasilkan nilai galat (*error*) yang membludak. Secara matematis, proses pemetaan pada Autoencoder dapat dirumuskan sebagai berikut (Farzad dan Gulliver, 2020):

- 1) Fungsi *Encoder*, Memetakan input x ke dalam *hidden state* h .

$$h = f(W \cdot x + b)$$

- 2) Fungsi *Decoder*, Merekonstruksi h kembali menjadi $\hat{x}$.

$$\hat{x} = g(W' \cdot h + b')$$

- 3) Fungsi Evaluasi Galat Rekonstruksi (*Reconstruction Error*), Berbeda dengan arsitektur konvensional yang umumnya menggunakan *Mean Squared Error* (MSE), penelitian ini merancang mekanisme evaluasi khusus menggunakan kombinasi *Mean Absolute Error* (MAE) dan *Mean Absolute Percentage Error* (MAPE). Tahap pertama adalah menghitung selisih absolut antara data asli dan hasil rekonstruksi:

$$MAE = |x - \hat{x}|$$

- 4) MAE (*MAE Logic Gate*), Nilai absolut MAE ini kemudian disaring melalui Gerbang Logika MAE (*MAE Logic Gate*). Jika nilai selisih berada di bawah batas toleransi tertentu, nilai tersebut dinolkan untuk menyaring *noise* komputasi. Hanya selisih yang lolos dari gerbang logika ini yang kemudian dikonversi menjadi persentase galat akhir menggunakan MAPE, dengan penambahan nilai *Epsilon* (ϵ) pada penyebut untuk mencegah *error* pembagian dengan nol:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{x_i - \hat{x}_i}{x_i + \epsilon} \right| \times 100$$

Nilai MAPE inilah yang ditetapkan sebagai Skor Anomali. Jika persentase galat melebihi ambang batas (*threshold*) adaptif yang telah dikalkulasi dari fase pelatihan, maka sekuens data log tersebut secara resmi diklasifikasikan sebagai serangan.

Penerapan konsep *Unsupervised Learning* melalui arsitektur Autoencoder ini dipilih sebagai solusi metodologis untuk mengatasi ketiadaan label serangan (*unlabeled data*) pada dataset log historis di INSTIKI. Dengan mekanisme pelatihan yang hanya mempelajari pola rekonstruksi data normal, teori ini memungkinkan sistem untuk

mendeteksi segala bentuk penyimpangan atau serangan varian baru (*zero-day attacks*) secara otomatis. Penggunaan filter gerbang logika MAE dan kalkulasi MAPE dalam mengukur tingkat kesalahan memastikan bahwa sistem memiliki kepekaan analitis yang tajam, mampu meredam alarm palsu dari komputasi desimal berskala kecil, dan tidak bergantung pada definisi aturan *firewall* yang kaku.

Arsitektur LSTM-Autoencoder

LSTM-Autoencoder adalah arsitektur hibrida dalam *Deep Learning* yang menggabungkan kemampuan *Long Short-Term Memory* (LSTM) dalam menangkap pola data sekuensial dengan kemampuan *Autoencoder* dalam mempelajari representasi fitur tanpa pengawasan (*unsupervised*). Menurut (Srivastava dkk., 2015), model ini dirancang khusus untuk menangani data deret waktu atau urutan (*sequence*) dengan cara memetakan urutan input ke dalam representasi vektor berdimensi tetap, dan kemudian mendekodekannya kembali menjadi urutan output. Struktur dasar LSTM-Autoencoder terdiri dari dua komponen utama yang bekerja secara berkesinambungan:

- 5) *Encoder*, adalah bagian yang terdiri dari lapisan sel LSTM yang bertugas membaca urutan input data log ($\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$) langkah demi langkah. Keadaan tersembunyi (*hidden state*) terakhir dari Encoder dianggap sebagai ringkasan atau representasi laten ($\mathbf{Z}$) yang mengandung informasi konteks paling penting dari seluruh urutan input.
- 6) *Decoder*, bagian ini merupakan bagian yang terdiri dari lapisan LSTM yang menerima representasi laten ($\mathbf{Z}$) dari Encoder dan mencoba merekonstruksi kembali urutan input aslinya menjadi output ($\mathbf{X}' = \{\mathbf{x}'_1, \mathbf{x}'_2, \dots, \mathbf{x}'_T\}$) secara terbalik atau berurutan.

Dalam konteks deteksi anomali, (Malhotra, 2016) menjelaskan bahwa LSTM-Autoencoder dilatih hanya menggunakan dataset normal. Tujuannya adalah meminimalkan fungsi kerugian (*loss function*) yang berupa kesalahan rekonstruksi (*reconstruction error*), yaitu selisih antara data input asli ($\mathbf{X}$) dengan data hasil rekonstruksi ($\mathbf{X}'$).

$$L = \sum_{t=1}^T ||x_t - x'_t||^2$$

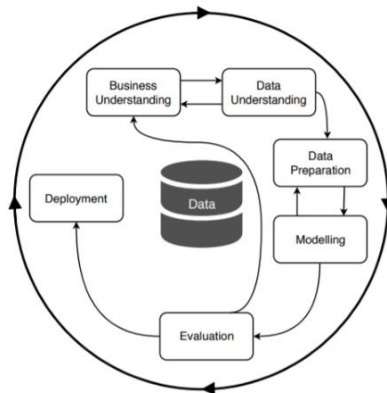
Hipotesis utama dari pendekatan ini adalah bahwa model akan belajar merekonstruksi pola normal dengan sangat baik (*error* rendah).

Namun, ketika model dihadapkan pada data anomali atau serangan yang memiliki pola asing, model akan kesulitan merekonstruksinya sehingga menghasilkan nilai *reconstruction error* yang tinggi. Lonjakan nilai error inilah yang kemudian digunakan sebagai indikator untuk mendeteksi adanya intrusi atau anomali pada sistem. Penggabungan unit LSTM ke dalam struktur *Encoder-Decoder* (LSTM-Autoencoder) diterapkan untuk mensinergikan keunggulan ekstraksi fitur waktu (*temporal*) dengan kemampuan deteksi anomali tanpa pengawasan. Arsitektur hibrida ini dirancang secara spesifik untuk memodelkan representasi terkompresi dari urutan normal log Wazuh, di mana model akan belajar mengenali 'irama' normal sistem dan secara sensitif menolak pola input asing yang mengindikasikan adanya intrusi keamanan.

2.9 Metode Pemodelan CRISP-DM

Cross-Industry Standard Process for Data Mining (CRISP-DM) merupakan standar metodologi yang paling banyak digunakan dalam industri dan penelitian untuk mengelola proyek penambangan data (*data mining*) dan analitik. Menurut (Chapman dkk., 2000) dan (Wirth dan Hipp, t.t.), CRISP-DM menyediakan kerangka kerja yang terstruktur namun fleksibel, yang membagi siklus hidup proyek menjadi enam tahapan hierarkis yang saling terhubung dan bersifat iteratif (berulang).

- 1) *Business Understanding*: Memahami tujuan proyek dan kebutuhan dari perspektif bisnis atau organisasi, serta mendefinisikan masalah data mining.
- 2) *Data Understanding*: Mengumpulkan data awal, mendeskripsikan, mengeksplorasi, dan memverifikasi kualitas data.
- 3) *Data Preparation*: Membangun dataset final (tabel, record, dan atribut) yang akan digunakan dalam pemodelan.
- 4) *Modeling*: Memilih dan menerapkan teknik pemodelan (seperti LSTM-Autoencoder) serta mengkalibrasi parameternya.
- 5) *Evaluation*: Mengevaluasi model secara menyeluruh untuk memastikan model mencapai tujuan bisnis yang telah ditetapkan.
- 6) *Deployment*: Menerapkan model ke dalam sistem operasional atau membuat laporan akhir.



Gambar 2.1 Alur CRISP-DM

Kekuatan utama CRISP-DM terletak pada sifat siklusnya, di mana output dari satu tahap dapat memicu revisi pada tahap sebelumnya, memastikan bahwa solusi yang dihasilkan benar-benar relevan dengan masalah yang dihadapi. Metodologi CRISP-DM (*Cross-Industry Standard Process for Data Mining*) diadopsi sebagai kerangka kerja standar untuk memetakan alur eksperimen yang sistematis seperti pada Gambar 2.1, mulai dari pemahaman karakteristik data log hingga evaluasi model akhir. Penggunaan siklus hidup CRISP-DM menjamin bahwa setiap tahapan teknis penelitian memiliki arah yang jelas dan terstruktur,

serta memungkinkan proses perbaikan iteratif (*feedback loop*) apabila hasil evaluasi model belum memenuhi target deteksi yang diharapkan.

2.10 Preprocessing Data

Pra-pemrosesan data adalah tahapan krusial dalam *Deep Learning* untuk mengubah data log mentah menjadi format numerik yang terstruktur dan optimal bagi model. Berdasarkan karakteristik data *Log Event* pada platform Wazuh, tahapan pra-pemrosesan dalam penelitian ini meliputi seleksi fitur, *encoding*, penskalaan, pengurutan kronologis, dan pembentukan sekuens (*sequentialization*).

- 1) *Feature Selection*, seleksi fitur adalah proses mereduksi dimensi data dengan memilih subset atribut yang paling relevan dan membuang atribut yang redundan atau tidak memiliki nilai prediktif. Menurut (Han dkk., 2006), reduksi dimensi sangat penting untuk mencegah fenomena *curse of dimensionality*, di mana jumlah fitur yang terlalu banyak dapat menyebabkan model menjadi *overfitting* dan meningkatkan beban komputasi secara eksponensial. Dalam konteks log keamanan, fitur esensial biasanya mencakup penanda waktu (*timestamp*) untuk urutan, deskripsi aktivitas (*semantik*), tingkat keparahan (*severity level*), dan identitas agen (*entity*).
- 2) *Label Encoding*, Algoritma *Deep Learning* bekerja berdasarkan operasi matematis matriks dan vektor, sehingga data kategorikal berbasis teks (seperti *rule.description* dan *agent.name*) tidak dapat diproses secara langsung. Untuk mengatasi hal ini, diterapkan teknik *Label Encoding*. *Label Encoding* adalah metode transformasi yang mengonversi setiap nilai unik (kategori) dalam fitur teks menjadi representasi numerik (integer) yang unik. Jika sebuah fitur kategorikal F memiliki himpunan nilai unik $C = \{c_1, c_2, \dots, c_n\}$, maka fungsi pemetaan E akan memberikan nilai integer k dimana $0 \leq k < n$ untuk setiap kategori:

$$E(c_i) = k_i$$

Metode ini memastikan bahwa informasi kualitatif dari log tetap terjaga dan dapat diterjemahkan ke dalam bahasa mesin tanpa mengubah makna aslinya, serta lebih hemat memori dibandingkan teknik *One-Hot Encoding* untuk fitur dengan kardinalitas tinggi.

- 3) *Min-Max Scaling* (Normalisasi), data log seringkali memiliki rentang nilai yang sangat bervariasi antar fiturnya (contohnya, fitur `rule.level` memiliki skala 0-15, sedangkan hasil *encoding* bisa mencapai ribuan). Perbedaan skala ini dapat menyebabkan fitur dengan nilai nominal besar mendominasi proses pembelajaran bobot (*weight updates*). Untuk mengatasi hal tersebut, diterapkan metode *Min-Max Scaling*. Metode ini memetakan seluruh nilai fitur ke dalam rentang seragam $[0, 1]$. Secara matematis, untuk setiap nilai x pada fitur X , nilai ternormalisasi x' dihitung dengan rumus:

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

Menurut (Patro dan Sahu, 2015), normalisasi ini sangat krusial dalam *Deep Learning* untuk menjaga stabilitas gradien, mencegah bias numerik, dan mempercepat proses konvergensi model menuju titik optimal (global minimum) saat pelatihan.

- 4) *Sorting Data*, sebelum dilakukan transformasi menjadi urutan, dataset wajib diurutkan secara kronologis berdasarkan atribut waktu (*timestamp*). (Le dan Zhang, 2022) dalam penelitiannya menekankan bahwa penggunaan strategi acak (*random selection*) pada data log dapat menyebabkan masalah kebocoran data (*data leakage*), di mana informasi masa depan bocor ke masa lalu. Oleh karena itu, pengurutan data secara *ascending* menjamin integritas kausalitas, memastikan bahwa model LSTM hanya mempelajari pola berdasarkan kejadian yang telah terjadi sebelumnya, mensimulasikan kondisi deteksi ancaman di dunia nyata.
- 5) *Sliding Window Sequentialization*, karena arsitektur LSTM membutuhkan input berupa data deret waktu, data tabular 2 dimensi harus direstrukturisasi menjadi format tensor 3 dimensi: (*Samples, Timesteps, Features*). Transformasi ini dilakukan menggunakan metode Jendela Geser (*Sliding Window*). Sebagaimana dijelaskan dalam alur kerja deteksi anomali log oleh (Le dan Zhang, 2022), metode ini mengelompokkan log menjadi sekuens dengan panjang tetap (*fixed window size*). Jika ditetapkan panjang jendela W , maka pada setiap langkah waktu t , model tidak melihat log sebagai kejadian tunggal yang terisolasi, melainkan sebagai satu kesatuan vektor urutan S_t :

$$S_t = \{x_t, x_{t+1}, \dots, x_{t+W-1}\}$$

Hal ini memungkinkan arsitektur LSTM untuk menangkap konteks temporal dan mempelajari pola ketergantungan jangka panjang (*long-term dependencies*) antar aktivitas sistem.

Mengingat data mentah yang dihasilkan oleh platform Wazuh berupa teks semi-terstruktur yang mengandung banyak variasi dan *noise*, penerapan teori pra-pemrosesan data menjadi syarat mutlak sebelum tahap pemodelan. Rangkaian proses ini, yang meliputi pembersihan data, *parsing* log, hingga transformasi numerik, berfungsi untuk menjamin kualitas input (*data quality*) sehingga model *Deep Learning* dapat memproses informasi dengan optimal dan terhindar dari bias akibat format data yang buruk

2.11 Metode Evaluasi

Evaluasi kinerja adalah tahapan fundamental dalam pengembangan model pembelajaran mesin untuk mengukur seberapa efektif model dalam memetakan input ke output yang diharapkan. Dalam konteks deteksi anomali atau intrusi, evaluasi tidak hanya bertujuan mengukur ketepatan prediksi, tetapi juga memverifikasi kemampuan model dalam meminimalkan kesalahan fatal seperti kegagalan mendeteksi serangan (*False Negative*) atau kesalahan memberikan peringatan palsu (*False Positive*). Menurut (Han dkk., 2012) metode evaluasi standar untuk masalah klasifikasi biner didasarkan pada perbandingan antara kelas prediksi model dengan kelas sebenarnya (*ground truth*) yang direpresentasikan dalam bentuk *Confusion Matrix*.

2.11.1 *Confusion Matrix*

Confusion Matrix adalah tabel visualisasi yang merinci kinerja algoritma klasifikasi. Tabel ini membagi hasil prediksi ke dalam empat kuadran utama berdasarkan kesesuaiannya dengan label aktual data. Komponen-komponen dalam *Confusion Matrix* didefinisikan sebagai berikut (Sammut dan Webb, 2011):

- 1) *True Positive (TP)*: Jumlah data anomali (serangan) yang benar dideteksi oleh sistem sebagai anomali.
- 2) *True Negative (TN)*: Jumlah data normal yang benar dikenali oleh sistem sebagai normal.
- 3) *False Positive (FP)*: Jumlah data normal yang salah dideteksi sebagai anomali (dikenal sebagai *False Alarm*). Tingginya nilai FP dapat menyebabkan kelelahan pada administrator (*alert fatigue*).
- 4) *False Negative (FN)*: Jumlah data anomali (serangan) yang gagal dideteksi dan dianggap sebagai normal. Dalam keamanan siber, ini adalah jenis kesalahan yang paling berbahaya karena serangan lolos tanpa terdeteksi.

2.11.2 *Measurement Metrics*

Berdasarkan nilai-nilai pada *Confusion Matrix*, kinerja model diquantifikasi menggunakan beberapa metrik statistik. Mengingat karakteristik data log keamanan yang seringkali tidak seimbang (*imbalanced*) di mana jumlah aktivitas normal jauh lebih banyak daripada serangan metrik Akurasi saja seringkali bias dan tidak cukup untuk menggambarkan kinerja model yang sebenarnya (Davis dan Goadrich,

2006). Oleh karena itu, digunakan metrik tambahan seperti *Precision*, *Recall*, dan *F1-Score*.

- 1) Akurasi (*Accuracy*), akurasi mengukur rasio total prediksi yang benar (baik positif maupun negatif) terhadap keseluruhan sampel data.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- 2) Presisi (*Precision*), presisi mengukur tingkat kepercayaan dari prediksi positif yang dihasilkan model. Metrik ini menjawab pertanyaan: "Dari semua log yang dideteksi sebagai serangan, berapa persen yang benar-benar serangan?".

$$Precision = \frac{TP}{TP + FP}$$

- 3) Sensitivitas (*Recall*) *Recall* atau *Detection Rate* mengukur kemampuan model dalam menemukan kembali seluruh kejadian positif yang ada di dalam dataset. Metrik ini menjawab pertanyaan: "Dari semua serangan yang benar-benar terjadi, berapa persen yang berhasil ditangkap?".

$$Recall = \frac{TP}{TP + FN}$$

- 4) F1-Score F1-Score adalah rata-rata harmonik (harmonic mean) antara Presisi dan Recall. Menurut (Goutte dan Gaussier, 2005), F1-Score memberikan gambaran kinerja yang lebih seimbang dibandingkan rata-rata aritmatika biasa, terutama ketika terdapat *trade-off* antara *precision* dan *recall*. Nilai F1-Score yang tinggi (mendekati 1) mengindikasikan bahwa model memiliki tingkat deteksi yang baik dengan tingkat alarm palsu yang rendah.

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

2.12 *Python*

Python adalah bahasa pemrograman tingkat tinggi (*high-level programming language*) yang bersifat interpretatif, interaktif, dan berorientasi objek. Diciptakan pertama kali oleh Guido van Rossum pada tahun 1991, *Python* dirancang dengan filosofi desain yang menekankan pada keterbacaan kode (*code readability*) melalui penggunaan indentasi spasi yang signifikan. Menurut (Van Rossum dan Drake, 1995), *Python* menggabungkan kapabilitas yang luar biasa dengan sintaksis kode yang sangat jelas, menjadikannya salah satu bahasa pemrograman yang paling efisien untuk pengembangan aplikasi berskala besar maupun skrip sederhana.



Gambar 2.2 Logo *Python*

Dalam konteks sains data dan kecerdasan buatan, *Python* telah menjadi standar industri karena dukungan ekosistem pustaka (*libraries*) yang luas dan komunitas pengembang yang aktif. (Raschka dan Mirjalili, 2019) dalam bukunya menjelaskan bahwa dominasi *Python* dalam pembelajaran mesin (*Machine Learning*) didorong oleh ketersediaan pustaka *open-source* yang kuat, antara lain:

- 1) Pandas, untuk manipulasi dan analisis data terstruktur (tabular).
- 2) NumPy, untuk komputasi numerik dan operasi matriks berkinerja tinggi.
- 3) Scikit-learn, untuk algoritma pembelajaran mesin standar dan evaluasi model.
- 4) TensorFlow dan Keras, framework utama untuk pengembangan *Deep Learning* dan jaringan saraf tiruan (*Neural Networks*).
- 5) Matplotlib/Seaborn, untuk visualisasi data grafis.

Dalam penelitian ini, Python dipilih sebagai lingkungan pengembangan utama karena ketersediaan pustaka khusus yang sangat relevan dengan tahapan eksperimen, mulai dari pra-pemrosesan hingga evaluasi model. Pustaka Pandas digunakan secara ekstensif untuk mengolah ribuan baris data log mentah dari Wazuh yang berformat CSV, sementara kerangka kerja Keras berbasis TensorFlow dimanfaatkan untuk membangun dan melatih arsitektur LSTM-Autoencoder. Kemampuan *Python* dalam menangani operasi vektor melalui NumPy juga sangat krusial untuk mempercepat komputasi matriks pada proses pembelajaran *Deep Learning*, memastikan sistem deteksi anomali dapat dibangun secara efisien dan akurat.

2.13 Google Colab

Google Colaboratory atau yang lebih dikenal sebagai *Google Colab* adalah layanan lingkungan pemrograman berbasis komputasi awan (*cloud computing*) yang disediakan oleh *Google Research*. Layanan ini dirancang khusus untuk memfasilitasi penelitian di bidang pembelajaran mesin (*Machine Learning*) dan sains data dengan menyediakan antarmuka *Jupyter Notebook* yang dapat dijalankan langsung melalui peramban web (*web browser*) tanpa memerlukan konfigurasi instalasi lokal (Bisong, 2019).

Keunggulan utama *Google Colab* terletak pada penyediaan akses gratis terhadap sumber daya komputasi kinerja tinggi, termasuk *Graphics Processing Unit* (GPU) seperti NVIDIA Tesla K80 atau T4, dan *Tensor Processing Unit* (TPU). Menurut (Carneiro dkk., 2018), ketersediaan akselerator perangkat keras ini sangat krusial dalam mempercepat proses pelatihan model *Deep Learning* yang kompleks, yang jika dilakukan pada komputer konvensional (CPU standar) akan memakan waktu yang sangat lama.

Selain keunggulan perangkat keras, Google Colab terintegrasi secara *native* dengan ekosistem Google lainnya, khususnya Google Drive. Fitur ini memungkinkan penyimpanan dan pengambilan dataset dalam skala besar dilakukan secara efisien, serta mendukung kolaborasi *real-time* antar peneliti selayaknya menyunting dokumen di Google Docs. Lingkungan ini juga telah dilengkapi (*pre-installed*) dengan berbagai pustaka populer seperti TensorFlow, Keras, NumPy, dan Pandas, sehingga peneliti dapat langsung fokus pada pengembangan algoritma. Dalam penelitian ini, *Google Colab* dipilih sebagai platform utama untuk implementasi dan pelatihan model LSTM-Autoencoder. Pemilihan platform ini didasarkan pada kebutuhan komputasi intensif yang diperlukan saat melatih arsitektur jaringan saraf tiruan berulang (*Recurrent Neural Network*) pada dataset log Wazuh yang bervolume besar. Dengan memanfaatkan fasilitas GPU akselerator yang disediakan oleh *Google Colab*, durasi pelatihan (*training time*) model untuk mempelajari pola normal log dapat dipersingkat secara signifikan. Selain itu, fitur integrasi *Google Drive* dimanfaatkan untuk menyimpan file log wazuh.csv secara terpusat, memastikan data dapat diakses dan diolah secara konsisten selama proses eksperimen berlangsung.

BAB III

ANALISIS DAN PERANCANGAN

3.1 Tempat dan Waktu Penelitian

Penelitian ini dilaksanakan di lingkungan Institut Bisnis dan Teknologi Indonesia (INSTIKI), dengan fokus utama pada analisis data komputasi. Objek penelitian yang digunakan adalah data log sekunder yang diperoleh langsung dari sistem *Security Information and Event Management* (SIEM) Wazuh yang beroperasi di jaringan internal kampus. Rangkaian penelitian, yang mencakup studi literatur, pengumpulan data, *preprocessing*, pemodelan *deep learning* dengan LSTM-Autoencoder, hingga tahap evaluasi, dilaksanakan mulai tanggal 25 Juli 2025.

3.2 Teknik Pengumpulan Data

Teknik pengumpulan data untuk penelitian ini menggunakan metode kuantitatif. Observasi langsung dilakukan di lingkungan jaringan Institut Bisnis dan Teknologi Indonesia (INSTIKI) untuk mengidentifikasi arsitektur dan operasional sistem keamanan yang sedang berjalan. Untuk memperkuat pemahaman mengenai data dan proses bisnis, penulis juga melakukan wawancara dengan administrator dari bagian SIEM (*Security Information and Event Management*).

3.2.1 Data Primer

Metode Metode yang digunakan oleh penulis untuk mendapatkan data primer adalah dengan Observasi langsung ke tempat penelitian, yaitu bagian SIEM INSTIKI. Metode Observasi adalah teknik pengumpulan data yang dilakukan dengan mengamati objek penelitian secara langsung di lapangan. Dalam hal ini, penulis meninjau langsung kondisi sistem pengelolaan log keamanan saat ini serta mengamati fluktuasi volume log harian untuk menentukan sampel data yang representatif.

Berdasarkan observasi tersebut, didapatkanlah data primer berupa data *log event* yang diekspor dari platform Wazuh dengan total 10.000 baris data. Sampel data ini mencakup rentang waktu selama 26 hari, terhitung mulai tanggal 19 Juni 2025 hingga 30 Juli 2025. Pemilihan periode dan volume data ini dilakukan secara sengaja dengan pertimbangan bahwa rentang waktu tersebut mencakup hampir empat siklus mingguan (*weekly cycles*), sehingga mampu merepresentasikan variasi pola trafik jaringan antara hari kerja (*weekdays*) dan akhir pekan (*weekends*). Selain itu, dataset pada periode ini dipilih karena memiliki kualitas informasi yang tinggi dengan rata-rata tingkat kerentanan (*Rule*

Level) sebesar 6.09, yang mengindikasikan adanya dinamika pola aktivitas normal dan ancaman yang seimbang untuk kebutuhan pelatihan model deteksi anomali.

Index	version	source	action	status	log
1	1.172.16.80	source	POST	172.16.80	500 /source/
2	1.172.16.80	source	POST	172.16.80	500 /source/
3	1.172.16.80	source	POST	172.16.80	500 /source/
4	1.172.16.80	source	POST	172.16.80	500 /source/
5	1.172.16.80	source	POST	172.16.80	500 /source/
6	1.172.16.80	source	POST	172.16.80	500 /source/
7	1.172.16.80	source	POST	172.16.80	500 /source/
8	1.172.16.80	source	POST	172.16.80	500 /source/
9	1.172.16.80	source	POST	172.16.80	500 /source/
10	1.172.16.80	source	POST	172.16.80	500 /source/
11	1.172.16.80	source	POST	172.16.80	500 /source/
12	1.172.16.80	source	POST	172.16.80	500 /source/
13	1.172.16.80	source	POST	172.16.80	500 /source/
14	1.172.16.80	source	POST	172.16.80	500 /source/
15	1.172.16.80	source	POST	172.16.80	500 /source/
16	1.172.16.80	source	POST	172.16.80	500 /source/
17	1.172.16.80	source	POST	172.16.80	500 /source/
18	1.172.16.80	source	POST	172.16.80	500 /source/
19	1.172.16.80	source	POST	172.16.80	500 /source/
20	1.172.16.80	source	POST	172.16.80	500 /source/
21	1.172.16.80	source	POST	172.16.80	500 /source/
22	1.172.16.80	source	POST	172.16.80	500 /source/
23	1.172.16.80	source	POST	172.16.80	500 /source/
24	1.172.16.80	source	POST	172.16.80	500 /source/
25	1.172.16.80	source	POST	172.16.80	500 /source/
26	1.172.16.80	source	POST	172.16.80	500 /source/
27	1.172.16.80	source	POST	172.16.80	500 /source/
28	1.172.16.80	source	POST	172.16.80	500 /source/
29	1.172.16.80	source	POST	172.16.80	500 /source/
30	1.172.16.80	source	POST	172.16.80	500 /source/

Gambar 3.1 Dokumentasi Raw Data

3.2.2 Data Sekunder

Data sekunder yang digunakan penulis dalam mengembangkan sistem adalah dokumen yang berupa dokumentasi teknis dan dokumen kepustakaan dari penelitian sejenis yang terdahulu, yang sekiranya dapat membantu peneliti dalam menyelesaikan penelitian. Metode yang digunakan untuk mendapatkan data sekunder ini berupa:

- 1) Metode Kepustakaan, untuk mendukung landasan teori pada penelitian ini maka digunakan metode kepustakaan. Kepustakaan diambil dari internet dan buku-buku yang dapat menunjang penelitian.
- 2) Studi Literatur, studi literatur dilakukan dengan mencari jurnal dan penelitian terdahulu sebagai landasan penelitian ini.

3.3 Analisis Sistem

Analisis sistem yang dilakukan dengan tujuan untuk mengetahui proses-proses yang berjalan pada sistem. Tujuan dari mengetahui proses-proses tersebut adalah untuk memahami jalannya sistem secara keseluruhan serta hambatan-hambatan yang mungkin ada pada saat pengembangan sistem.

3.3.1 Kebutuhan Fungsional

Analisa kebutuhan fungsional dari sistem yang akan dibangun dapat dituliskan dalam beberapa poin, seperti berikut:

- 1) Sistem harus mampu mengubah data log mentah menjadi format yang dapat dipahami oleh model *Deep Learning*.
- 2) Sistem harus mampu mempelajari pola perilaku normal dari data latih tanpa label (*unsupervised*).
- 3) Sistem harus mampu mengidentifikasi apakah data baru termasuk aktivitas normal atau anomali.
- 4) Sistem harus mampu mengukur seberapa baik kemampuannya dalam mendeteksi anomali.

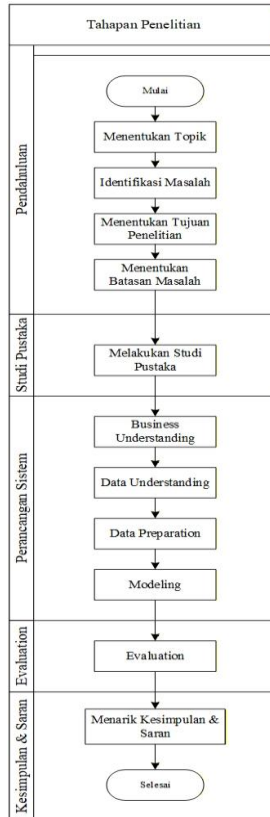
3.3.2 Kebutuhan Non Fungsional

Analisa Kebutuhan *Non Fungsional* dari Sistem yang akan dibangun, yaitu:

- 1) *Hardware*
 - a. *Processor*.
AMD Ryzen 7 5800H (8C/16T.3.2/4.4 GHz. 4MB L2/16MB L3)
 - b. *Memory*
8 GB RAM
 - c. *GPU*.
NVIDIA GeForce RTX 3050TI 4GB GDDR6
 - d. *Operating System*
Windows 11 Pro 64-bit
- 2) *Software*
 - a. *Google colab*
 - b. Bahasa Pemrograman *Python* versi 3.12.2
 - c. *Microsoft Excel*
 - d. *Google chrome*

3.4 Tahapan Penelitian

Proses pengembangan sistem deteksi Anomali dalam penelitian dilakukan dalam beberapa tahapan seperti yang ditampilkan gambar berikut:

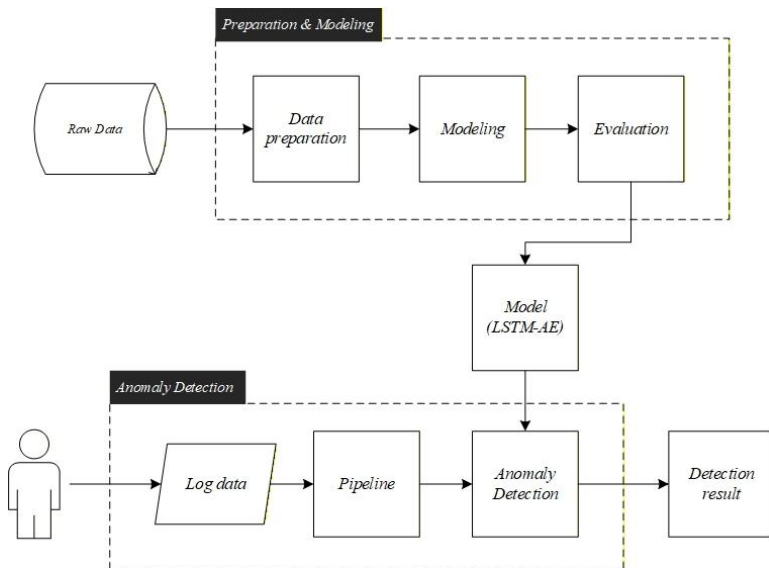


Gambar 3.2 Alur penelitian.

3.5 Gambaran Umum Sistem

Gambaran umum sistem dalam penelitian ini mencakup alur kerja dan proses secara keseluruhan dalam perancangan model deteksi anomali data log keamanan menggunakan metode LSTM – AE. Alur kerja sistem dimulai dari penerimaan data masukan (*input*) berupa data log mentah dari platform SIEM Wazuh (*file wazuh.csv*). Mengingat data log mentah memiliki format yang tidak terstruktur dan bervariasi, sistem melakukan tahapan *Preprocessing Data* yang meliputi seleksi fitur, konversi data teks menjadi numerik (*encoding*), dan normalisasi skala data. Data yang telah bersih kemudian diubah menjadi format *sequence* waktu (*time-series sequences*) menggunakan metode Sliding Window.

Inti dari pemrosesan sistem terletak pada model LSTM-Autoencoder yang dilatih secara *unsupervised*. Pada fase pelatihan (*training*), model hanya mempelajari pola dari data aktivitas normal. Tujuannya adalah agar model mampu merekonstruksi (meniru) pola normal tersebut dengan tingkat kesalahan (*error*) yang rendah. Proses deteksi dilakukan dengan membandingkan *Reconstruction Error* (selisih antara data asli dan hasil rekonstruksi model) terhadap nilai ambang batas (*threshold*) yang telah ditentukan. Jika sebuah *sequence* log menghasilkan *error* di atas *threshold*, sistem akan mengklasifikasikannya sebagai Anomali (potensi serangan atau gangguan). Sebaliknya, jika *error* berada di bawah *threshold*, aktivitas tersebut dianggap Normal. Hasil akhir dari sistem ini adalah label klasifikasi yang kemudian dievaluasi kinerjanya menggunakan metrik *Precision*, *Recall*, dan *F1-Score*. Gambaran visual mengenai arsitektur dan alur data pada sistem ini dapat dilihat pada Gambar 3.3.

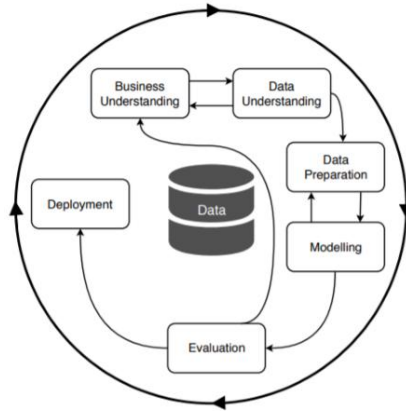


Gambar 3.3 Gambaran umum sistem.

3.6 Metode Perancangan Sistem

Dalam mengembangkan model deteksi anomali ini, penelitian menggunakan kerangka kerja standar CRISP-DM (*Cross-Industry Standard Process for Data Mining*). Metode ini dipilih karena

menyediakan struktur yang sistematis dan teruji untuk proyek berbasis data (*data mining*), mulai dari memahami kebutuhan operasional hingga implementasi teknis model *Deep Learning*.



Gambar 3.4 Metode CRISP-DM

3.7 *Business Understanding*

Tahap ini merupakan langkah awal yang krusial untuk memetakan masalah operasional di lapangan menjadi definisi masalah teknis penelitian. Berdasarkan hasil observasi dan wawancara dengan administrator SIEM di INSTIKI, teridentifikasi tantangan utama berupa volume log keamanan yang sangat masif dan tingginya tingkat *noise* (peringatan palsu), yang menyebabkan proses analisis manual menjadi tidak efektif dan rentan terhadap kesalahan manusia. Oleh karena itu, tujuan utama penelitian ini didefinisikan sebagai pengembangan sistem deteksi anomali otomatis berbasis *Deep Learning*. Sistem ini diharapkan dapat membantu administrator dalam filterisasi aktivitas normal dan mengisolasi aktivitas mencurigakan secara akurat, sehingga respon terhadap peristiwa keamanan dapat dilakukan dengan lebih cepat dan efisien

3.8 *Data Understanding*

Tahap pemahaman data dimulai dengan proses pengumpulan data sekunder berupa *file log* yang diekspor langsung dari sistem manajemen log aktivitas sistem. Pada fase ini, dilakukan eksplorasi mendalam untuk memahami karakteristik dataset, termasuk distribusi waktu kejadian (*timestamp*), variasi tingkat keparahan (*alert level*), identitas agen

pengirim, dan deskripsi pesan log. Analisis statistik deskriptif dilakukan untuk memeriksa kualitas data, mendeteksi keberadaan nilai yang hilang (*missing values*), serta memastikan bahwa fitur-fitur kunci yang diperlukan untuk analisis sekuensial tersedia secara lengkap dan layak untuk diproses lebih lanjut.

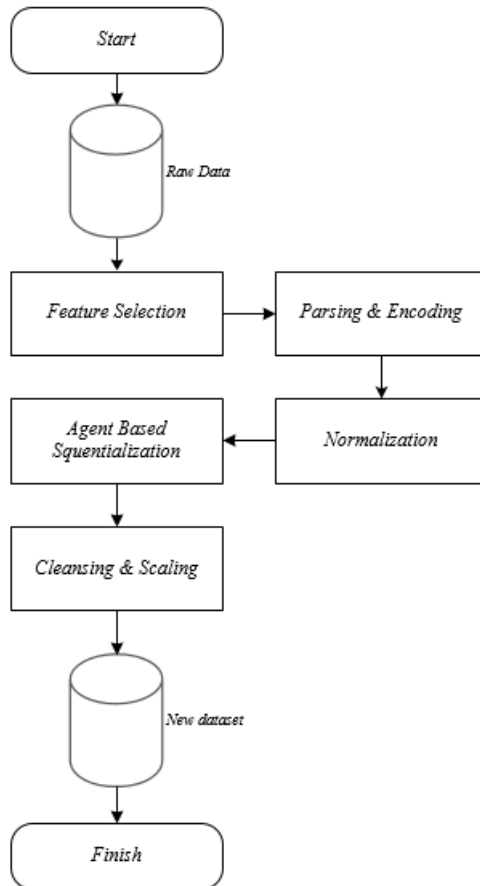
Gambar 3.5 Proses seleksi fitur data

Dari total 107 atribut yang terdapat dalam *log dataset* mentah wazuh.csv, penelitian ini melakukan seleksi fitur secara ketat untuk mereduksi dimensi data dan memfokuskan proses pembelajaran model hanya pada informasi yang paling esensial. Langkah ini bertujuan untuk menghindari fenomena *curse of dimensionality* serta menghilangkan atribut redundan yang berpotensi menjadi *noise* selama proses pelatihan model LSTM-Autoencoder. Berdasarkan karakteristik model yang berbasis sekuensial dan kebutuhan analisis keamanan, dipilih empat fitur kunci dengan justifikasi sebagai berikut:

- 1) **_source.@timestamp**: Fitur ini merupakan elemen paling krusial dalam analisis data deret waktu (*time-series*). Keberadaannya mutlak diperlukan untuk mengurutkan seluruh kejadian log secara kronologis, memastikan integritas urutan (*sequence*) yang valid agar model LSTM dapat mempelajari pola ketergantungan aktivitas dari waktu ke waktu.
- 2) **_source.rule.description**: Fitur ini berfungsi sebagai representasi semantik utama atau *Log Key* yang menjelaskan jenis peristiwa yang terjadi (misalnya, “*File deleted*” atau “*Web server 500 error*”). Deskripsi ini memberikan konteks “apa” yang sedang terjadi, yang menjadi dasar utama bagi model untuk membedakan jenis-jenis aktivitas dalam sistem.
- 3) **_source.rule.level**: Fitur numerik ini merepresentasikan bobot keparahan (*severity*) dari setiap insiden dengan skala 0 hingga 15. Penggunaan fitur ini memberikan dimensi intensitas bagi model, sehingga model dapat belajar membedakan bobot antara aktivitas rutin operasional dan peringatan keamanan yang kritis.
- 4) **_source.agent.name**: Fitur ini memberikan konteks identitas mengenai sumber atau entitas yang mengirimkan log. Mengingat setiap agen (misalnya server web versus komputer pengguna) memiliki pola perilaku normal yang berbeda, fitur ini membantu model untuk tidak menyamaratakan pola aktivitas dari sumber yang berbeda.

3.9 Data Preparation

Persiapan data merupakan tahapan yang paling intensif dalam penelitian ini, di mana data log mentah ditransformasi menjadi format terstruktur yang siap untuk pemodelan. Proses ini mencakup pembersihan data (*data cleaning*) dengan menyeleksi fitur yang relevan dan membuang informasi redundan. Selanjutnya, dilakukan transformasi data, yaitu mengubah fitur kategorikal (teks) menjadi representasi numerik menggunakan teknik *Label Encoding* dan menyamakan skala fitur numerik menggunakan *Min-Max Scaling*. Bagian terpenting dari tahap ini adalah pembentukan *sequence* waktu (*windowing*), di mana data log diurutkan secara kronologis dan dipecah menjadi urutan-urutan (*sliding window*) agar sesuai dengan kebutuhan input arsitektur LSTM. Dataset kemudian dibagi secara proporsional menjadi data latih (*training set*) dan data uji (*testing set*).



Gambar 3.6 Alur *pre-processing* data

Alur *preprocessing* data dalam penelitian ini dirancang secara sistematis untuk mentransformasi data log mentah yang bersifat semiterstruktur menjadi format matriks sekuensial yang kompatibel dengan arsitektur *Deep Learning* seperti pada gambar 3.6 di atas. Proses transformasi ini berjalan secara linier, dimulai dari pembersihan data mentah untuk mengeliminasi *noise* dan nilai yang korup, dilanjutkan

dengan konversi tipe data untuk menjembatani perbedaan antara data tekstual log dengan kebutuhan input numerik model. Seluruh rangkaian proses ini bermuara pada restrukturisasi data menjadi format deret waktu (*time-series*) yang ketat, guna memastikan integritas kronologis tetap terjaga sehingga model LSTM dapat mempelajari pola ketergantungan aktivitas (*temporal dependencies*) secara akurat. Adapun tahapan teknis krusial dalam alur ini meliputi:

3.9.1 Feature Selection

Tahap awal pra-pemrosesan dimulai dengan pembersihan data mentah (*raw data*) yang diekspor dari platform SIEM Wazuh dalam format CSV. Dataset awal memiliki dimensi yang tinggi dengan lebih dari 100 atribut kolom, di mana sebagian besar berisi nilai kosong (*null values*) atau informasi administratif yang tidak berkontribusi signifikan terhadap pola deteksi anomali (*seperti _index, _id, dan _score*). Oleh karena itu, dilakukan reduksi dimensi melalui seleksi fitur secara manual (*expert judgement*) untuk mempertahankan hanya atribut yang memuat informasi intrinsik kejadian. Fitur-fitur yang dipilih untuk diproses lebih lanjut meliputi:

- 1) *Timestamp*, waktu kejadian log, yang dikonversi dari format string ke tipe data *datetime* untuk memungkinkan pengurutan data secara kronologis. Pengurutan ini krusial mengingat arsitektur LSTM bekerja berdasarkan urutan waktu (*time-series*).
- 2) *Agent Name*, identitas agen pengirim log (misalnya: siput, pusatkarir) untuk membedakan sumber aktivitas.
- 3) *Rule Level*, tingkat keparahan (*severity*) kejadian yang diberikan oleh Wazuh (skala 0-15), yang menjadi indikator awal intensitas ancaman.
- 4) *Rule Description*, deskripsi tekstual mengenai aktivitas yang terjadi (misalnya: "*ClamAV database update*" atau "*Web server 500 error*").

Selain seleksi fitur, baris data yang memiliki nilai *missing value* pada kolom vital dihapus untuk menjaga integritas pelatihan model. Tabel 3.1 berikut menampilkan cuplikan sampel data setelah melalui proses pembersihan dan pengurutan waktu.

Tabel 3.1 Contoh Data Hasil Pembersihan dan Seleksi Fitur

Timestamp	Agent Name	Rule Level	Description
2025-07-25 12:01:25	siput	3	ClamAV database update
2025-07-25 12:04:16	sads	5	Web server 500 error code

2025-07-25 12:06:36	sads	5	Web server 500 error code
2025-07-25 12:06:38	sads	5	Web server 500 error code

3.9.2 *Sorting*

Salah satu persyaratan fundamental dalam penerapan algoritma *Long Short-Term Memory* (LSTM) adalah validitas urutan waktu (*temporal dependency*). Mengingat data log Wazuh dikumpulkan dari berbagai agen secara asinkron melalui jaringan, terdapat kemungkinan baris data terekam tidak sesuai dengan urutan kejadian sebenarnya (misalnya akibat latensi jaringan). Jika data ini langsung diproses tanpa pengurutan, model akan gagal menangkap konteks kausalitas antar kejadian. Oleh karena itu, sebelum data diolah menjadi *sequence*, dilakukan proses pengurutan data (*sorting*) secara menaik (*ascending*) berdasarkan atribut Timestamp. Proses ini menjamin bahwa baris data t_n selalu merupakan kejadian yang terjadi setelah t_{n-1} . Validitas urutan ini sangat krusial untuk memastikan bahwa *window* yang terbentuk nantinya merepresentasikan alur kronologis yang benar dari aktivitas sistem. Tabel 3.2 memperlihatkan sampel data yang telah terurut secara presisi hingga satuan milidetik.

Tabel 3.2 Sampel Data Hasil Pengurutan Kronologis

Timestamp	Agent Name	Rule Level	Description
2025-07-25 12:01:25	siput	3	ClamAV database update
2025-07-25 12:04:16	sads	5	Web server 500 error code
2025-07-25 12:06:36	sads	5	Web server 500 error code
2025-07-25 12:06:38	sads	5	Web server 500 error code

3.9.3 *Encoding*

Setelah melalui tahap pembersihan, atribut pada dataset log masih didominasi oleh tipe data kategorikal berupa teks (*string*), khususnya pada fitur Agent_Name dan Description. Mengingat algoritma jaringan saraf tiruan (*Neural Network*) seperti LSTM beroperasi menggunakan operasi perkalian matriks yang menuntut input berupa nilai

numerik, maka data teks tersebut harus ditransformasikan terlebih dahulu. Dalam penelitian ini, teknik transformasi yang diterapkan adalah *Label Encoding*. Teknik ini bekerja dengan memetakan setiap nilai unik (kategori) dalam kolom teks menjadi bilangan bulat (*integer*) yang unik. Proses ini dilakukan menggunakan pustaka *Scikit-learn*, di mana setiap variasi nama agen dan deskripsi kejadian dikonversi menjadi representasi indeks angka. Sebagai contoh, agen dengan nama 'siput' direpresentasikan oleh indeks 5, dan pesan log 'ClamAV database update' direpresentasikan oleh indeks 321.

Setelah proses *encoding* selesai, kolom teks asli dieliminasi dari dataset untuk mengurangi redundansi memori dan memastikan seluruh fitur input memiliki format numerik yang homogen. Hasil transformasi data yang siap untuk diproses lebih lanjut dapat dilihat pada Tabel 3.2.

Tabel 3.3 Hasil Transformasi Data Menjadi Numerik (*Encoded*)

Timestamp	Agent Name	Rule Level	Description
2025-07-25 12:01:25	5	3	321
2025-07-25 12:04:16	4	5	648
2025-07-25 12:06:36	4	5	648
2025-07-25 12:06:38	4	5	648

3.9.4 *Scaling*

Meskipun data telah memiliki format numerik setelah tahap *encoding*, terdapat permasalahan disparitas skala yang signifikan antar fitur. Sebagai ilustrasi, fitur *Rule_Level* hanya memiliki rentang nilai 0 hingga 15, sedangkan fitur *Description_Enc* dapat memiliki nilai hingga ratusan atau ribuan. Ketimpangan rentang nilai ini dapat menyebabkan bias pada proses pelatihan model *Deep Learning*, di mana fitur dengan nilai nominal yang lebih besar akan mendominasi pembaruan bobot (*weight updates*) dan memperlambat konvergensi algoritma *Gradient Descent*. Untuk mengatasi masalah tersebut, penelitian ini menerapkan teknik normalisasi menggunakan metode *Min-Max Scaling*. Metode ini mentransformasikan setiap nilai fitur secara linear sehingga terdistribusi dalam rentang interval [0, 1]. Proses ini dilakukan menggunakan persamaan matematis berikut:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Dimana X adalah nilai asli, X_{min} adalah nilai minimum dalam kolom fitur, dan X_{max} adalah nilai maksimum. Penerapan normalisasi menjamin bahwa seluruh fitur memiliki kontribusi yang setara terhadap model. Hasil normalisasi data dapat dilihat pada Tabel 3.3.

Tabel 3.4 Hasil Normalisasi Data dengan Min-Max Scaling

Agent Name Enc	Rule Level	Description Enc
0.833333	0.000000	0.490826
0.666667	0.166667	0.990826
0.666667	0.166667	0.990826
0.666667	0.166667	0.990826

3.9.5 *Agent Based Sequentialization*

Berbeda Berbeda dengan algoritma *Machine Learning* konvensional yang memproses setiap baris data secara independen, arsitektur *Long Short-Term Memory* (LSTM) membutuhkan input data yang memiliki dimensi waktu (*time steps*). Dalam kasus ini pembentukan sekuens dilakukan *agent based* agar model nantinya mempelajari pola normal dari *event* yang terjadi pada 1 *agent* yang spesifik. Untuk mengakomodasi kebutuhan ini, dilakukan restrukturisasi data menggunakan metode *Sliding Window*. Metode ini mengubah dataset tabular 2-dimensi (N, D) menjadi struktur array 3-dimensi (N, T, D), di mana N adalah jumlah sampel, T adalah panjang jendela waktu (*window size*), dan D adalah jumlah fitur. Dalam penelitian ini, ditetapkan panjang *window size* sebesar 10 ($T = 10$). Pemilihan parameter ini didasarkan pada karakteristik data log yang memiliki median interval waktu antar kejadian sebesar 0,27 detik, sehingga jendela sebesar 10 langkah merepresentasikan durasi pengamatan sekitar 2,7 detik. Durasi ini dinilai ideal untuk menangkap satu rangkaian pola serangan utuh (*attack sequence*) dalam satu bingkai pengamatan.

Penentuan ukuran ini sangat krusial karena *window size* yang terlalu kecil berisiko menghilangkan konteks kausalitas antar kejadian, sehingga model gagal mendeteksi pola serangan yang kompleks. Sebaliknya, *window size* yang terlalu besar akan menyebabkan masuknya informasi tidak relevan (*noise*) yang dapat mengaburkan pola utama, serta meningkatkan risiko *Vanishing Gradient Problem* di mana model kesulitan mengingat informasi di awal urutan. Jendela ini akan "bergeser" satu langkah ke depan secara iteratif menyusuri seluruh dataset untuk

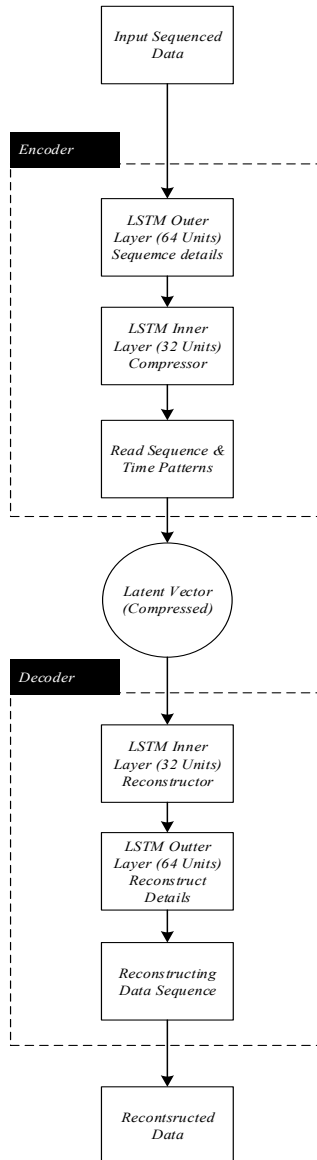
membentuk urutan input. Sebagai ilustrasi bentuk data hasil transformasi ini, Tabel 3.5 menampilkan contoh representasi satu sampel *sequence*.

Tabel 3.5 Contoh Sampel *Input Sequence* (*Window Size* = 5)

Time Step	Agent Name Enc	Rule Level	Description Enc
$t - 4$	0.666667	0.000000	0.490826
$t - 3$	0.666667	0.166667	0.990826
$t - 2$	0.666667	0.166667	0.990826
$t - 1$	0.666667	0.166667	0.990826
t	0.666667	0.166667	0.990826

3.10 Modeling

Tahap pemodelan berfokus pada perancangan dan penerapan algoritma *Machine Learning*. Dalam penelitian ini, dipilih arsitektur LSTM-Autoencoder karena keunggulannya dalam mempelajari pola memori jangka panjang pada data deret waktu (*time-series*) secara *unsupervised*. Model dirancang dengan komponen *Encoder* untuk mengompresi fitur log dan *Decoder* untuk merekonstruksinya kembali. Proses pelatihan dilakukan menggunakan data aktivitas normal, dengan tujuan agar model belajar meminimalkan selisih antara input asli dan hasil rekonstruksi (*Reconstruction Error*). Pada tahap ini juga dilakukan eksperimen untuk menentukan parameter model terbaik serta penetapan nilai ambang batas (*threshold*) yang optimal untuk membedakan antara aktivitas normal dan anomali. Berikut adalah gambar alur kerja arsitektur LSTM-AE.



Gambar 3.7 Alur Kerja Arsitektur LSTM-AE

Gambar 3.7 di atas mengilustrasikan detail arsitektur model *Deep Learning* yang dirancang dalam penelitian ini. Model ini menggunakan struktur "jam pasir" yang simetris, terdiri dari tiga bagian utama: *Encoder*, *Latent Vector*, dan *Decoder*. Berikut adalah penjelasan rinci untuk setiap lapisan pemrosesan data:

- 1) *Input Layer*, proses dimulai dengan memasukkan data log yang sudah diproses. Data ini berbentuk urutan waktu (*sequence*), di mana setiap satu data input berisi 10 kejadian log terakhir (*10 Time Steps*). Setiap kejadian memiliki 4 informasi utama (*Features*), yaitu waktu, deskripsi, level, dan nama agen. Oleh karena itu, dimensi data yang masuk adalah (10, 4).
- 2) *Encoder*, bagian ini terdiri dari 2 bagian layer inti sebagai berikut:
 - a. *LSTM Encoder 1 (64 Unit)*, data masuk ke lapisan pertama yang memiliki kapasitas memori besar (64 neuron). Tujuannya adalah untuk menangkap detail pola yang rumit dari urutan log tersebut. Dimensi data diperluas sementara menjadi (10, 64) untuk mempelajari fitur-fitur tersembunyi.
 - b. *LSTM Encoder 2 (32 Unit)*, data kemudian diteruskan ke lapisan kedua yang lebih kecil (32 neuron). Di sini terjadi proses kompresi. Model dipaksa untuk membuang informasi yang tidak penting (*noise*) dan hanya menyimpan intisari dari pola log tersebut.
- 3) *Latent Vector*, urutan log yang awalnya panjang (*10 time steps*) dipadatkan menjadi satu vektor tunggal berukuran (32). Vektor ini adalah ringkasan padat yang merepresentasikan pola normal dari sistem. Jika ada anomali atau serangan, ringkasan ini akan menjadi kacau karena model tidak mengenali polanya.
- 4) *Decoder*, bagian yang bertugas untuk membangun kembali data asli berdasarkan ringkasan yang ada.
 - a. *LSTM Decoder 1 (32 Unit)*, ringkasan data *Latent Vector* dikembalikan ke bentuk urutan waktu dan diproses oleh lapisan ini untuk mulai menyusun ulang struktur log awal.
 - b. *LSTM Decoder 2 (64 Unit)*, data dikembangkan lagi ke dimensi yang lebih besar (64 neuron) untuk mengembalikan detail-detail halus yang mungkin hilang saat kompresi.

- 5) *Output Layer*, pada tahap akhir data dikembalikan ke dimensi aslinya yaitu (10, 4). Hasil ini disebut sebagai Data Rekonstruksi. Tujuan utama model ini adalah membuat Data Rekonstruksi (Output) semirip mungkin dengan Data Asli (Input). Jika terdapat perbedaan besar (*error*) antara Input dan Output, maka data tersebut akan ditandai sebagai anomali.

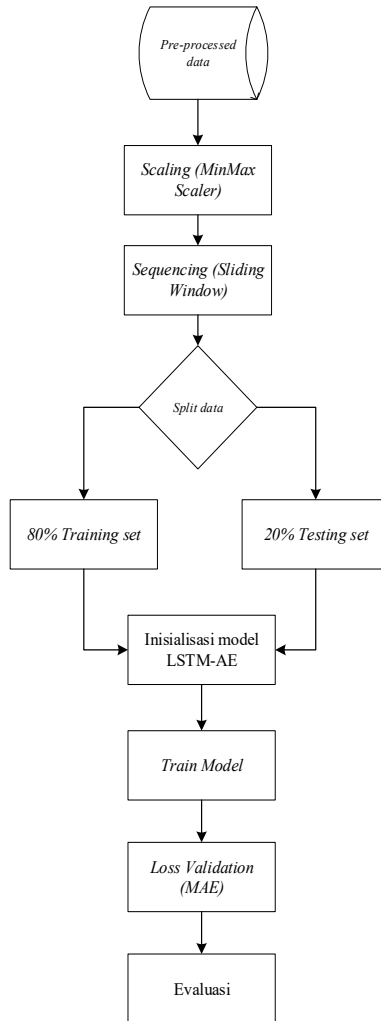
Agar arsitektur LSTM-Autoencoder dapat mempelajari pola normal dari data log secara efektif dan mencapai konvergensi yang optimal, diperlukan penetapan konfigurasi parameter eksternal atau hiperparameter. Berbeda dengan bobot model yang dipelajari secara otomatis, hiperparameter harus diinisialisasi sebelum proses pelatihan dimulai. Dalam penelitian ini, pemilihan nilai hiperparameter didasarkan pada rekomendasi empiris dari penelitian terdahulu mengenai deteksi anomali pada data sekuensial (Du dkk., 2017; Landauer dkk., 2023). Pengaturan yang tepat bertujuan untuk menyeimbangkan antara kecepatan komputasi dan akurasi rekonstruksi model. Berikut adalah rincian konfigurasi hiperparameter yang diterapkan dalam penelitian ini.

- a. *Adam Optimizer* (Algoritma Optimasi), penelitian ini menggunakan algoritma Adam (*Adaptive Moment Estimation*) sebagai pengoptimal. Adam dipilih karena kemampuannya dalam menyesuaikan *learning rate* secara adaptif untuk setiap parameter. Menurut Kingma & Ba (2014), Adam sangat efisien secara komputasi, membutuhkan sedikit memori, dan sangat cocok untuk masalah dengan data yang besar atau *noisy* seperti data log sistem. Penggunaan Adam membantu model LSTM menghindari jebakan *local minima* dan mempercepat proses konvergensi dibandingkan dengan *Stochastic Gradient Descent* (SGD) konvensional.
- b. *Mean Absolute Error* (MAE), sebagai model berbasis rekonstruksi (*reconstruction-based*), tujuan pelatihan adalah meminimalkan selisih antara input asli dan output rekonstruksi. Penelitian ini menggunakan MAE (*Mean Absolute Error*) sebagai fungsi objektif. (Landauer dkk., 2023) dalam surveinya menyatakan bahwa MAE lebih *robust* (tahan) terhadap *outliers* dibandingkan *Mean Squared Error* (MSE). Mengingat data log mungkin mengandung anomali ekstrem, MAE memastikan bahwa model berfokus pada perbaikan error rekonstruksi secara keseluruhan tanpa terlalu dipengaruhi oleh satu atau dua data pencilan yang sangat jauh.
- c. *Epochs*, 50 Iterasi *Epoch* mendefinisikan berapa kali algoritma pembelajaran akan memproses seluruh dataset latih. Penelitian ini menetapkan jumlah *epoch* sebanyak 50. Jumlah ini dipilih berdasarkan eksperimen awal untuk memberikan waktu yang cukup bagi model dalam mempelajari pola temporal dari urutan log tanpa menyebabkan *overfitting* (kondisi di mana model menghafal data latih tetapi gagal pada data baru).
- d. *Batch Size*, 32 Sampel *Batch size* menentukan jumlah sampel data yang diproses sebelum bobot model diperbarui (*backpropagation*). Ukuran *batch* sebesar 32 dipilih untuk menjaga stabilitas estimasi gradien kesalahan. Ukuran ini memberikan keseimbangan yang baik antara kecepatan pelatihan dan penggunaan memori GPU/CPU, serta memungkinkan model untuk melakukan pembaruan bobot yang lebih sering dan presisi (Du dkk., 2017)

Sebelum memasuki tahap eksekusi pelatihan model, data yang telah melalui tahap *preprocessing* awal perlu dipersiapkan lebih lanjut agar sesuai dengan spesifikasi input arsitektur LSTM-Autoencoder. Proses ini diawali dengan normalisasi data menggunakan metode *Min-Max Scaling*, yang bertujuan menyamakan rentang nilai seluruh fitur numerik ke dalam skala $[0, 1]$. Langkah ini krusial untuk mencegah dominasi fitur tertentu dan mempercepat konvergensi algoritma optimasi *Adam* yang telah ditetapkan sebelumnya. Selanjutnya, data yang telah ternormalisasi diubah strukturnya dari format tabel dua dimensi menjadi format tensor tiga dimensi (*Samples, Time Steps, Features*) melalui teknik *Sliding Window* dengan panjang langkah (*time step*) 10 sehingga model dapat menangkap konteks temporal dari setiap urutan kejadian log.

Setelah struktur data terbentuk, dilakukan strategi pembagian data (*data splitting*) yang merupakan fondasi dari eksperimen ini. Dataset dibagi menjadi dua sub-himpunan terpisah dengan rasio 80:20. Sebesar 80% data dialokasikan sebagai Data Latih (*Training Set*), yang diasumsikan berisi pola aktivitas normal. Data latih ini berfungsi sebagai materi pembelajaran bagi model untuk meminimalkan *reconstruction error*. Sementara itu, sisa 20% data dialokasikan sebagai Data Uji (*Testing Set*). Data uji ini tidak dilibatkan sama sekali dalam proses pelatihan, melainkan disisihkan secara eksklusif untuk mensimulasikan data baru di tahap pengujian dan validasi, guna mengukur objektivitas kinerja deteksi anomali.

Seluruh rangkaian proses persiapan tersebut, mulai dari transformasi dimensi data, pembagian dataset, hingga eksekusi pelatihan dan penyimpanan model, dirancang dalam sebuah alur kerja sistematis (*modeling pipeline*). Alur kerja ini memastikan bahwa setiap tahapan berjalan secara berurutan dan terstruktur, sebagaimana diilustrasikan secara rinci pada Gambar 3.8 berikut ini.



Gambar 3.8 Alur modeling & Spliting data

3.11 Evaluation

Evaluasi kinerja model merupakan tahapan krusial untuk mengukur validitas dan keandalan sistem deteksi anomali yang telah dibangun. Meskipun model dilatih menggunakan pendekatan tanpa pengawasan (*unsupervised learning*) pada data normal, proses pengujian dilakukan secara objektif menggunakan himpunan data uji (*testing set*) yang mengandung campuran antara aktivitas normal dan skenario serangan. Tujuan utama dari evaluasi ini adalah untuk memverifikasi kemampuan model LSTM-Autoencoder dalam membedakan pola log rutin (*baseline*) dengan pola log yang mengindikasikan ancaman keamanan, berdasarkan nilai ambang batas (*threshold*) yang telah ditetapkan.

3.11.1 Penentuan Adaptive Threshold

Keputusan deteksi anomali didasarkan pada asumsi bahwa model Autoencoder akan menghasilkan galat rekonstruksi (*reconstruction error*) yang sangat rendah saat memproses rutinitas data normal, dan sebaliknya akan menghasilkan lonjakan galat yang ekstrem ketika dihadapkan pada pola sekuens serangan. Untuk mengklasifikasikan status log jaringan secara akurat dan fleksibel, penelitian ini tidak menggunakan nilai ambang batas statis, melainkan menerapkan mekanisme Ambang Batas Adaptif (*Adaptive Threshold*) yang dilambangkan dengan $\theta_{adaptif}$.

Jika X adalah matriks sekuens data masukan dan $\hat{X}$ adalah hasil rekonstruksi model, maka Skor Anomali (S_A) diambil langsung dari kalkulasi *Mean Absolute Percentage Error* (MAPE) setelah data melewati penyaringan Gerbang Logika MAE. Mekanisme klasifikasi akhir didefinisikan melalui fungsi matematis $f(X)$ berikut:

$$f(X) = \begin{cases} 1 & \text{if } (Anomali), \& \text{"jika } S_A \\ & > \theta_{adaptif} \\ 0 & \text{if } (Normal), \& \text{"jika } S_A \\ & \leq \theta_{adaptif} \end{cases}$$

Di mana output **1** merepresentasikan pendeteksian serangan siber, dan **0** menandakan aktivitas jaringan yang wajar (Hundman dkk., 2018).

Berbeda dengan pendekatan tradisional yang menetapkan angka *threshold* secara manual dan kaku, nilai $\theta_{adaptif}$ dalam penelitian ini dikalkulasi dan diekstraksi secara otomatis dari distribusi galat pada saat fase pelatihan (*training*) data normal. Batas ini bersifat dinamis karena

nilainya secara mandiri menyesuaikan dengan tingkat kerumitan dan karakteristik operasional data log yang sedang dipelajari sebagai contoh, dengan mengambil nilai batas atas (maksimum) atau persentil tertinggi dari skor MAPE data latih. Dengan pendekatan *adaptive threshold* ini, sistem memiliki kemampuan *self-adjusting* (penyesuaian diri) yang optimal untuk membedakan antara *noise* komputasi harian dan anomali riil di lingkungan jaringan INSTIKI, sehingga secara efektif menekan potensi kelelahan peringatan (*alert fatigue*).

3.11.2 *Confusion Matrix*

Untuk mengukur keandalan *threshold* tersebut, digunakan *Confusion Matrix*. Ini adalah tabel representasi yang membandingkan hasil prediksi sistem dengan kondisi sebenarnya (*ground truth*) pada data uji. Komponen matriks didefinisikan sebagai berikut dalam konteks keamanan siber:

- 1) *True Positive* (TP): Jumlah data Serangan yang berhasil dideteksi sebagai Serangan (Anomali).
- 2) *True Negative* (TN): Jumlah data Normal yang benar dikenali sebagai Normal.
- 3) *False Positive* (FP): Jumlah data Normal yang salah dideteksi sebagai Serangan (Alarm Palsu/False Alarm).
- 4) *False Negative* (FN): Jumlah data Serangan yang gagal dideteksi dan dianggap Normal (Lolos Deteksi).

3.11.3 *Performance Measurement Metrics*

Berdasarkan nilai-nilai pada *Confusion Matrix*, kinerja model diukur menggunakan empat metrik utama. Dalam konteks deteksi intrusi, metrik Recall dan Precision seringkali lebih krusial daripada Akurasi total, terutama pada dataset yang tidak seimbang (*imbalanced*).

- 1) Akurasi (*Accuracy*): Mengukur rasio total prediksi yang benar terhadap keseluruhan data.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- 2) Presisi (*Precision*): Mengukur seberapa akurat model saat menyatakan adanya serangan (meminimalkan alarm palsu). Presisi rendah berarti administrator akan sering terganggu oleh peringatan yang salah.

$$Precision = \frac{TP}{TP + FP}$$

- 3) Recall (*Sensitivity / Detection Rate*): Mengukur kemampuan model dalam menemukan serangan yang ada. Nilai Recall yang tinggi sangat diharapkan dalam sistem keamanan karena berarti sedikit serangan yang lolos (*missed*).

$$Recall = \frac{TP}{TP + FN}$$

- 4) F1-Score: Merupakan rata-rata harmonik antara *Precision* dan *Recall*. Metrik ini menjadi acuan utama dalam penelitian ini karena memberikan gambaran keseimbangan antara kemampuan mendeteksi serangan dan meminimalkan alarm palsu.

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Evaluasi dilakukan dengan menghitung metrik-metrik di atas pada dataset pengujian yang berisi skenario serangan simulasi untuk memastikan model siap diimplementasikan pada lingkungan operasional.

Tabel 3.6 Tabel perbandingan data Normal & Serangan

Karakteristik	Data Normal (<i>Baseline</i>)	Data Serangan (<i>Anomaly</i>)
Definisi	Aktivitas operasional rutin sistem dan jaringan yang sah.	Aktivitas yang menyimpang dari pola kebiasaan atau berindikasi ancaman.

Contoh Log	<i>Rootcheck scan, File deleted (cache), Login session opened.</i>	<i>Brute force login (failed 500x), SQL Injection attempt, Modification of /etc/shadow</i>
Frekuensi	Tinggi & Stabil. Terjadi setiap hari, ribuan kali. (Misal: dalam sehari pasti ada user yang mengakses SADS atau SIPUT terutama diawal tahun akademik atau semester).	Langka & Tiba-tiba. Jarang terjadi, atau muncul dalam lonjakan besar dalam waktu singkat.
Sequence	Contoh: <i>Login > Akses File > Logout.</i>	Contoh: Login Gagal > Login Gagal... > Login Sukses > Akses Root.
Respon Model	<i>Reconstruction Error</i> “Rendah”, model mengenali pola ini karena sudah mempelajarinya saat latihan.	<i>Reconstruction Error</i> “Tinggi”, model tidak mengenali pola tersebut.

3.12 Deployment

Tahap akhir dari siklus ini adalah penyebaran, yang dalam konteks penelitian ini diwujudkan dalam bentuk purwarupa (*prototype*) sistem deteksi anomali dan dokumentasi hasil analisis. Hasil dari model yang telah dievaluasi kemudian diterjemahkan menjadi wawasan (*insight*) dan rekomendasi strategis bagi tim keamanan/IT di tempat penelitian. Selain itu, seluruh artefak penelitian, termasuk kode program *preprocessing*, model yang telah dilatih (.h5), dan laporan skripsi, disusun secara sistematis sebagai dokumentasi teknis untuk mendukung pengembangan lebih lanjut atau pemeliharaan sistem di masa mendatang.

BAB IV

IMPLEMENTASI DAN PEMBAHASAN

4.1 Data Preprocessing

Sebelum data dapat di proses oleh model data akan melalui beberapa tahapan *pre-processing* terlebih dahulu seperti yang telah di sampaikan di BAB III sebelumnya. Dalam *pre-processing* data akan melewati tahap *feature selection*, *encoding*, *sequentialization*, dan *normalization*.

4.1.1 Feature Selection

Pada tahap awal pengolahan data, langkah penting yang dilakukan adalah memilih fitur (*feature selection*) dari log mentah yang dihasilkan oleh sistem keamanan Wazuh. Log asli dari Wazuh sebenarnya berukuran sangat besar dan memuat puluhan kolom informasi tambahan (*metadata*). Namun, banyak dari kolom tersebut seperti detail rute jaringan atau info sistem tidak memiliki hubungan langsung dengan kejadian anomali keamanan. Oleh karena itu, kolom-kolom yang tidak penting tersebut dibuang. Langkah ini bertujuan agar beban kerja komputer menjadi lebih ringan, sekaligus memastikan model AI bisa fokus belajar hanya dari variabel utama yang benar-benar membentuk pola ancaman siber. Karena model LSTM-Autoencoder sangat bergantung pada urutan waktu kejadian, pengambilan data hanya difokuskan pada empat atribut utama. Atribut tersebut adalah *timestamp* yang berfungsi sebagai penanda urutan waktu kejadian, serta *agent_name* untuk mengetahui dari *server* mana aktivitas tersebut berasal. Selain itu, fitur *rule_level* tetap dipertahankan sebagai angka penunjuk tingkat keparahan suatu deteksi, didampingi oleh *description* yang memberikan penjelasan teks mengenai jenis aktivitas yang terjadi di dalam sistem.

```
Dataset loaded successfully. Total rows: 10000
Data cleansing completed. Current shape: (10000, 4)
```

	timestamp	agent_name	rule_level	description
0	2025-07-25 12:01:25.379	siput	3	ClamAV database update
1	2025-07-25 12:04:16.132	sads	5	Web server 500 error code (Internal Error).
2	2025-07-25 12:06:36.243	sads	5	Web server 500 error code (Internal Error).
3	2025-07-25 12:06:38.336	sads	5	Web server 500 error code (Internal Error).
4	2025-07-25 12:23:33.286	sads	5	Web server 500 error code (Internal Error).

Gambar 4.1 Output *feature selection*

Hasil dari proses pemangkasan ini menghasilkan susunan data (*DataFrame*) yang jauh lebih ringkas, terstruktur, dan bersih dari fitur yang tidak relevan, sebagaimana yang dapat diamati pada tabel di atas. Keempat variabel utama ini dinilai sudah sangat mewakili seluruh rekam jejak aktivitas sistem secara utuh. Selanjutnya, data yang sudah dirapikan ini akan dipersiapkan untuk masuk ke tahap pengolahan berikutnya, yaitu mengubah tipe data teks menjadi angka (*Label Encoding*) sebelum akhirnya dibentuk menjadi potongan-potongan sekuens.

4.1.2 *Encoding*

Setelah proses pemilihan fitur selesai, tahapan penting berikutnya adalah mengubah tipe data. Model AI, termasuk arsitektur LSTM-Autoencoder yang digunakan dalam penelitian ini, hanya bisa membaca dan memproses masukan berupa angka. Oleh karena itu, kolom-kolom yang datanya masih berupa teks (*string*), yaitu *agent_name* dan *description*, wajib diubah menjadi angka. Teknik yang digunakan pada penelitian ini adalah *Label Encoding*, di mana sistem akan mengenali setiap teks yang unik dan memberikannya kode angka bulat (*integer*) secara berurutan.

```
Starting Label Encoding process...
Label Encoding completed.
```

	timestamp	rule_level	agent_name_enc	description_enc
0	2025-07-25 12:01:25.379	3	5	321
1	2025-07-25 12:04:16.132	5	4	648
2	2025-07-25 12:06:36.243	5	4	648
3	2025-07-25 12:06:38.336	5	4	648
4	2025-07-25 12:23:33.286	5	4	648

Gambar 4.2 Output Proses *Encoding*

Merujuk pada hasil perubahan data di tabel atas, dapat dilihat bahwa setiap identitas agen dan deskripsi log telah berhasil diubah menjadi angka. Sebagai contoh, nama agen tertentu kini dikenali sebagai angka, begitu pula dengan ratusan macam deskripsi log aktivitas yang kini sudah memiliki kode angkanya masing-masing. Proses pengubahan ini menggunakan aturan *encoder* yang telah disimpan sebelumnya (dalam

bentuk *file pickle*) untuk memastikan kodenya tetap konsisten antara data latih dan data uji. Sistem ini juga sudah diatur agar tidak *error* apabila ke depannya menemukan teks log baru yang belum pernah dikenali sebelumnya (*unseen labels*). Keberhasilan tahap *encoding* ini menghasilkan susunan data yang seluruhnya sudah berupa angka. Gabungan antara teks yang sudah di-*encode* dan data yang dari awal sudah berupa angka (seperti *rule_level*) kini membentuk data yang siap diolah. Selanjutnya, kumpulan angka ini akan diratakan skalanya menggunakan *Min-Max Scaling* dan dipotong-potong berdasarkan urutan waktu (*Sliding Window*) sebelum dimasukkan ke dalam model jaringan saraf tiruan.

4.1.3 *Agent Based Sequentialization*

Tahapan penting selanjutnya untuk model *Long Short-Term Memory* (LSTM) adalah pembentukan urutan waktu kejadian (*sequentialization*). Berbeda dengan model biasa yang membaca log satu per satu secara terpisah, LSTM dirancang khusus untuk mengenali pola urutan waktu. Oleh karena itu, deretan data log harus dipotong-potong menjadi beberapa jendela waktu menggunakan teknik *Sliding Window*. Pada penelitian ini, ukuran jendela (*window size*) ditetapkan sebanyak 10 *timestamp*. Artinya, model akan membaca 10 baris log aktivitas yang berurutan sebagai satu kesatuan cerita. Selain itu, pemotongan ini dilakukan secara terpisah untuk masing-masing agen (*Agent-Based Sequencing*). Pemisahan ini sangat penting agar aktivitas dari *server* yang berbeda tidak saling tercampur, sehingga urutan kejadian pada mesin target tetap murni dan tidak bias.

1	2025-07-25 12:04:16.132	5	4	648
2	2025-07-25 12:06:36.243	5	4	648
3	2025-07-25 12:06:38.336	5	4	648
4	2025-07-25 12:23:33.286	5	4	648

Gambar 4.3 Output Proses *Sequentialization*

Berdasarkan gambar di atas, dapat dilihat wujud dari satu blok sekuens data yang telah selesai diproses. Data yang awalnya hanya berupa tabel biasa (*DataFrame*) kini telah berubah bentuk menjadi matriks 3 Dimensi dengan format (*jumlah_sampel, timestep, fitur*). Tabel tersebut menampilkan satu contoh potongan matriks berukuran 10x3, di mana

baris menunjukkan urutan kejadian (*Timestep* 1 hingga 10), dan kolom berisi nilai angka dari tiga fitur utama (*Agent Name*, *Rule Level*, dan *Description*). Melalui bentuk potongan seperti inilah, lapisan memori pada model AI bisa memahami urutan kejadian, misalnya mengaitkan indikasi peringatan di awal waktu dengan kerusakan sistem di akhir waktu. Pembentukan matriks ini menandai selesainya tahap perubahan bentuk data. Setelah ini, seluruh angka di dalam sekuens tersebut akan diratakan skalanya menjadi 0.0 hingga 1.0 menggunakan *Min-Max Scaling*. Tujuannya adalah agar tidak ada angka yang terlalu besar dan mendominasi perhitungan komputer, sebelum data tersebut akhirnya dimasukkan ke dalam model LSTM-Autoencoder.

4.1.4 *Data Cleansing & Splitting*

Setelah data log dibentuk menjadi potongan-potongan sekuens, tahapan penting selanjutnya adalah pembersihan dan pembagian data (*data cleansing and splitting*). Karena model Autoencoder hanya boleh belajar dari data operasional normal (*unsupervised learning*), sistem harus memisahkan mana sekuens yang aman dan mana yang mengandung serangan. Berdasarkan aturan bawaan keamanan Wazuh, log dengan tingkat peringatan (*Rule Level*) 6 ke atas dianggap sebagai ancaman siber nyata. Oleh karena itu, sistem memeriksa setiap potongan sekuens; jika di dalam satu jendela 10 *timestep* tersebut terdapat minimal satu saja log dengan *Rule Level* 6 atau lebih, maka seluruh sekuens tersebut langsung dicap sebagai data serangan (*X_with_attacks*). Sebaliknya, jika seluruh 10 baris log dalam satu sekuens tersebut memiliki *Rule Level* di bawah 6 secara konsisten, maka sekuens tersebut dinyatakan bersih atau murni normal (*X_pure_normal*).

```
Starting Data Cleansing and Train/Test Splitting...
- Total Pure Normal Sequences : 1653
- Total Attack Sequences      : 8284

=== FINAL DATA SPLIT RESULTS ===
Training Data (X_train) - Clean : (1322, 10, 3) sequences
Testing Data (X_test)  - Mixed  : (8615, 10, 3) sequences
```

Gambar 4.4 Output Proses *Data Cleansing & Splitting*

Merujuk pada hasil penyaringan yang ditampilkan pada Gambar 4.4, sistem berhasil mengidentifikasi 1.653 sekuens murni normal dan 8.284 sekuens serangan. Untuk kebutuhan pelatihan AI, kumpulan data

normal yang berjumlah 1.653 sekuens tersebut kemudian dibagi menggunakan rasio 80:20. Sebanyak 80% porsi data, atau tepatnya 1.322 sekuens, dialokasikan khusus sebagai data latih (X_{train}). Alokasi ini memastikan bahwa model AI secara eksklusif hanya belajar dari kondisi *server* yang benar-benar bersih. Sementara itu, sisa 20% data normal (sebanyak 331 sekuens) dicampur dengan keseluruhan data serangan (8.284 sekuens) untuk membentuk data pengujian (X_{test}) yang totalnya menjadi 8.615 sekuens campuran. Data uji campuran ini sengaja dibuat kotor agar nantinya bisa digunakan untuk menguji seberapa pintar model dalam membedakan log aktivitas yang wajar dan log serangan saat proses evaluasi. Dapat diamati pula bahwa baik data latih maupun data uji, keduanya tetap mempertahankan wujud matriks 3 dimensinya dengan format: (jumlah sampel, 10 *timestep*, 3 fitur). Untuk jatah data validasi pada saat pemodelan nanti, ditambahkan satu fungsi (`validation_split=0.2`) pada *keras* untuk secara otomatis memotong data (X_{train}) sebanyak 20% sebagai data validasi.

4.1.5 *Scaling*

Tahap Tahap akhir sebelum data dimasukkan ke dalam model jaringan saraf tiruan (AI) adalah meratakan skala angka menggunakan metode *Min-Max Scaling*. Pada data log yang sudah diubah menjadi angka sebelumnya, terdapat perbedaan rentang nilai yang sangat timpang antar fiturnya. Misalnya, fitur *rule_level* hanya memiliki rentang angka yang kecil (0 sampai 15), sedangkan fitur *agent_name* dan *description* bisa bernilai hingga ratusan karena mewakili banyak teks unik. Jika perbedaan ekstrem ini langsung dimasukkan, AI bisa kebingungan dan secara keliru menganggap fitur yang angkanya besar (ratusan) jauh lebih penting daripada fitur yang angkanya kecil. Untuk mencegah hal tersebut, semua angka ini dipadatkan dan diseragamkan menjadi rentang pecahan antara 0.0 hingga 1.0, tanpa merusak proporsi data aslinya.

Secara teknis, ada sedikit penyesuaian saat meratakan skala data untuk model LSTM. Alat normalisasi standar dari Python (Library) hanya dirancang untuk membaca data berbentuk tabel datar dua dimensi (2D). Sementara itu, data sekuens kita sudah berwujud matriks balok tiga dimensi (3D). Oleh karena itu, matriks berukuran (jumlah sampel, 10, 3) ini didatarkan atau dibongkar sementara menjadi format 2D. Setelah semua angkanya berhasil diubah ke rentang 0 hingga 1, data tersebut dilipat atau dibentuk kembali menjadi balok 3D aslinya. Cara ini memastikan proses pemerataan skala berhasil dilakukan tanpa merusak urutan kronologis kejadian (*timestep*) di dalam setiap sekuens log.

```
Applying Min-Max Scaling (Fitting only on Train data)...  
Ground Truth (y_true) secured! Total Anomalies in Test Data: 8284  
  
Data normalized (0 to 1) and ready for LSTM model training.
```

Gambar 4.5 Output Proses *Scaling*

Selain urusan perubahan bentuk (merujuk pada Gambar 4.5), ada satu aturan ketat yang dijaga pada fase ini, yaitu mencegah kebocoran informasi (*data leakage*). Aturan pencarian nilai minimum dan maksimum ini murni hanya dipelajari dari data latih yang normal (X_{train}). Setelah aturan skala ini didapatkan, aturannya "dikunci" permanen dan langsung digunakan untuk meratakan skala pada data uji (X_{test}) yang memuat serangan. Skema ini sangat penting untuk meniru kondisi keamanan siber di dunia nyata, di mana sistem tidak boleh mengintip atau mempelajari batasan angka serangan dari masa depan. Aturan skala yang sudah dikunci ini kemudian disimpan ke dalam sebuah *file* (bersamaan dengan *encoder* sebelumnya) agar bisa langsung dipakai saat model diimplementasikan secara *real-time*..

4.2 Arsitektur dan Pelatihan Model LSTM-Autoencoder

Melanjutkan keberhasilan tahap prapemrosesan data yang telah menstrukturkan log mentah menjadi tensor sekuensial tiga dimensi, fase krusial berikutnya dalam penelitian ini adalah tahap pemodelan komputasi. Sub-bab ini menguraikan rancang bangun arsitektur *Long Short-Term Memory Autoencoder* beserta skenario pelatihannya. Mengadopsi paradigma *unsupervised learning* (pembelajaran tak terawasi), model ini didesain secara spesifik untuk secara eksklusif mempelajari, mengekstraksi, dan merekonstruksi pola temporal dari perilaku infrastruktur server pada kondisi normal. Pembahasan pada bagian ini akan difokuskan pada spesifikasi teknis lapisan jaringan saraf (*neural network layers*) yang membentuk *encoder* dan *decoder*, serta evaluasi dinamika proses pelatihan komputasi yang diukur melalui pergerakan *loss function* guna memastikan model mencapai tingkat konvergensi yang optimal dan tidak mengalami *overfitting*.

4.2.1 Arsitektur Model

Rancang bangun jaringan saraf tiruan dalam penelitian ini menerapkan arsitektur *Long Short-Term Memory Autoencoder* (LSTM-Autoencoder). Arsitektur ini dirancang dengan struktur *bottleneck*

(menyempit di tengah) yang secara umum dibagi menjadi dua komponen utama, yaitu jaringan *Encoder* dan jaringan *Decoder*. Bagian *Encoder* bertugas untuk mereduksi dimensi dan mengekstraksi hubungan temporal dari sekuens log masukan berdimensi 3D (10, 3). Lapisan LSTM pada *Encoder* akan memproses runtutan data berukuran 10 *timestep* tersebut secara sekuensial, memeras informasi penting penanda aktivitas sistem, dan mengonversinya menjadi sebuah vektor representasi tunggal berdimensi lebih rendah di dalam ruang laten (*latent space*).

Proses rekonstruksi kemudian dilanjutkan oleh jaringan *Decoder* yang memiliki struktur terbalik dari *Encoder*. Sebelum masuk ke lapisan LSTM *Decoder*, vektor representasi tunggal yang berada di ruang laten terlebih dahulu direplikasi menggunakan lapisan *RepeatVector* agar dimensinya kembali sesuai dengan jumlah *timestep* awal, yaitu 10 langkah waktu. Lapisan LSTM pada bagian *Decoder* kemudian mempelajari kembali vektor replikasi tersebut untuk menyusun ulang pola urutan log ke bentuk aslinya. Tahap akhir dari rekonstruksi ini dieksekusi oleh lapisan *TimeDistributed Dense* dengan 3 unit output, yang berfungsi untuk memproyeksikan kembali nilai numerik dari fitur *Agent Name*, *Rule Level*, dan *Description* ke dalam skala spasial yang sama dengan data masukan asli.

```
Building LSTM-Autoencoder model architecture...
Model successfully built! Summary below:
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 10, 64)	17,408
dropout (Dropout)	(None, 10, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
repeat_vector (RepeatVector)	(None, 10, 32)	0
lstm_2 (LSTM)	(None, 10, 32)	8,320
dropout_1 (Dropout)	(None, 10, 32)	0
lstm_3 (LSTM)	(None, 10, 64)	24,832
time_distributed (TimeDistributed)	(None, 10, 3)	195

```
Total params: 63,171 (246.76 KB)
Trainable params: 63,171 (246.76 KB)
Non-trainable params: 0 (0.00 B)
```

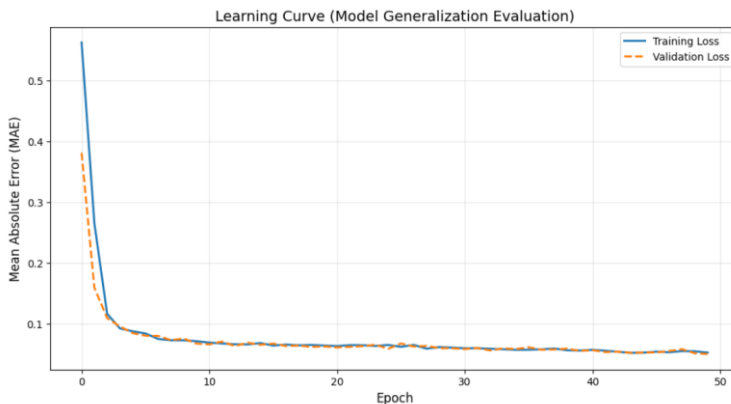
Gambar 4.6 Arsitektur Model LSTM-Autoencoder (*Model Summary*)

4.2.2 Skenario Pelatihan

Fase pelatihan model LSTM-Autoencoder dilakukan dengan menerapkan paradigma *unsupervised learning*, di mana jaringan saraf

tiruan dikondisikan untuk hanya mengenali dan mempelajari karakteristik dari pola operasional normal. Dataset yang digunakan pada tahapan ini adalah X_{train} yang telah melalui proses sterilisasi dari segala bentuk sekuens serangan siber pada tahap prapemrosesan data. Secara operasional, data log normal yang telah dinormalisasi menggunakan Min-Max Scaler ke rentang 0.0 hingga 1.0 diiris menggunakan metode *sliding window* untuk membentuk matriks temporal 3 Dimensi berukuran 10 *timestep*. Pada saat proses perulangan berlangsung, sisi *Encoder* bertugas meringkas matriks temporal tersebut ke dalam representasi ruang laten (*latent space*) yang sangat padat. Vektor padat ini kemudian diterima oleh sisi *Decoder* yang dipaksa untuk menggambar ulang (merekonstruksi) struktur matriks 10 baris tersebut tanpa melihat data aslinya.

Proses pelatihan komputasi dieksekusi menggunakan Adam Optimizer untuk mempercepat proses pencarian bobot optimal, serta fungsi kerugian (*loss function*) *Mean Absolute Error* (MAE) untuk mengukur tingkat melesetnya rekonstruksi model. Penggunaan MAE pada fase ini sangat esensial guna memfasilitasi turunan kalkulus yang stabil saat proses koreksi bobot, serta menyelamatkan jaringan dari fenomena *gradient explosion* akibat pembagian dengan nilai 0.0 yang sering muncul pada data log. Skenario pelatihan dijalankan dalam beberapa siklus (*epochs*) dengan ukuran *batch* tertentu, di mana performa model dipantau secara berkala menggunakan data validasi terpisah guna memastikan stabilitas pembelajaran.



Gambar 4.7 Grafik *loss history*

Berdasarkan visualisasi riwayat pelatihan (*loss history*) yang disajikan pada grafik di atas, dapat diamati pergerakan nilai *error* rekonstruksi baik pada data latih (*training loss*) maupun data validasi (*validation loss*). Kedua kurva menunjukkan tren penurunan yang sangat konsisten dan signifikan seiring dengan bertambahnya jumlah iterasi atau *epoch*. Karakteristik kurva yang bergerak beriringan ke bawah secara stabil tanpa adanya jarak pemisah (*gap*) yang melebar di antara keduanya merupakan indikator kuat bahwa model berhasil mencapai fase konvergensi dengan sangat baik. Fenomena pergerakan matematis ini membuktikan bahwa arsitektur jaringan tidak mengalami kendala *overfitting* (terlalu menghafal data latih) maupun *underfitting* (gagal mempelajari pola).

Melandainya nilai *loss* pada tahap akhir iterasi menandakan bahwa lapisan memori jangka panjang pada LSTM telah berhasil mengunci parameter dan bobot internalnya secara optimal. Lebih jauh lagi, titik konvergensi akhir dari distribusi galat pada grafik riwayat pelatihan inilah yang dijadikan sebagai landasan empiris dalam pengambilan keputusan Gerbang Logika MAE (*MAE Logic Gate*). Berdasarkan pengamatan sebaran sisa galat di akhir iterasi, diputuskanlah angka 0.3 sebagai batas toleransi mutlak. Angka ini merepresentasikan batas kewajaran dari *noise* komputasi desimal bawaan model saat merekonstruksi aktivitas log normal. Dengan penetapan ini, setiap selisih prediksi di bawah nilai 0.3 akan dinolkan (disaring) oleh gerbang logika.

Kesiapan parameter model yang telah terkunci permanen dan bersih dari *noise* komputasi tingkat rendah ini menjadi landasan mutlak untuk melanjutkan riset ke tahap evaluasi berikutnya. Transisi evaluasi komputasi selanjutnya akan difokuskan pada perhitungan metrik MAPE guna merumuskan angka penentuan ambang batas anomali (*Adaptive Threshold*) dan pengujian deteksi terhadap data log observasi baru yang mengandung pola serangan siber.

4.3 Evaluasi Model dan Penentuan *Threshold*

Menyusul keberhasilan fase pelatihan model LSTM-Autoencoder yang telah berkonvergensi secara optimal dalam mengenali pola data normal, tahapan krusial berikutnya adalah merumuskan mekanisme evaluasi dan landasan pengambilan keputusan deteksi. Sub-bab ini secara komprehensif menguraikan bagaimana kinerja model diukur menggunakan metrik *Mean Absolute Percentage Error* (MAPE) guna menghitung tingkat penyimpangan atau kesalahan rekonstruksi (*reconstruction error*) dari setiap sekuens log yang diproses. Lebih lanjut,

bagian ini mendeskripsikan metodologi penentuan ambang batas adaptif (*adaptive threshold*) yang dikalibrasi berdasarkan distribusi error secara statistik. Penentuan *threshold* ini bertindak sebagai garis demarkasi absolut dan objektif yang akan memisahkan fluktuasi log operasional yang wajar dari pola anomali ekstrem yang mengindikasikan terjadinya ancaman keamanan siber pada infrastruktur.

4.3.1 Pengukuran *Reconstruction Error* (MAPE)

Pada arsitektur *unsupervised* seperti LSTM-Autoencoder, deteksi anomali tidak dilakukan melalui metode klasifikasi biner secara langsung, melainkan melalui evaluasi kemampuan jaringan dalam menggambar ulang atau merekonstruksi data masukan. Model yang telah dilatih secara eksklusif menggunakan data normal akan mampu menghasilkan proyeksi keluaran (*output*) yang identik dengan masukannya (*input*) selama sekuens yang diuji merupakan aktivitas sistem yang wajar. Sebaliknya, ketika model dipaksa untuk memproses sekuens log yang mengandung serangan siber, jaringan akan mengalami kegagalan struktural dalam memproyeksikan pola tersebut. Selisih matematis antara data log asli dan hasil proyeksi jaringan inilah yang dikuantifikasi sebagai *error* rekonstruksi (*Reconstruction Error*). Untuk mengukur besaran *error* tersebut, penelitian ini mengimplementasikan metrik *Mean Absolute Percentage Error* (MAPE) yang dihitung secara spesifik pada setiap elemen sekuens menggunakan persamaan berikut:

$$\text{MAPE} = \left| \frac{\text{Actual} - \text{Predicted}}{\text{Actual} + \epsilon} \right| \times 100\%$$

Pemilihan MAPE didasarkan pada keunggulannya yang mampu merepresentasikan tingkat penyimpangan dalam skala persentase relatif, sehingga lebih mudah untuk dievaluasi dibandingkan metrik absolut seperti *Mean Squared Error* (MSE). Dalam implementasi komputasinya, perhitungan MAPE mengakumulasikan selisih absolut antara nilai observasi aktual (**Actual**) dan nilai prediksi model (**Predicted**), yang kemudian dibagi dengan nilai aktual. Guna mencegah terjadinya *error* komputasi akibat pembagian dengan nilai nol (*division by zero*), sebuah konstanta epsilon (ϵ) yang sangat kecil sebesar $1e-10$ (0.0000000001) ditambahkan pada penyebut di dalam rumusan algoritma. Namun, metrik MAPE memiliki kelemahan bawaan berupa sensitivitas ekstrem terhadap nilai aktual yang mendekati nol, di mana deviasi desimal yang sangat kecil sekalipun dapat memicu lonjakan persentase *error* secara artifisial (*false alarm*). Guna mengatasi anomali matematis tersebut, penelitian ini

mengintegrasikan mekanisme hibrida berupa Gerbang Logika MAE dengan batas toleransi (*noise tolerance*) sebesar 0.3 sebelum fungsi MAPE dieksekusi.

--- CALCULATION BREAKDOWN FOR ANOMALY SEQUENCE RANK #1 (Index: 8412) ---

Timestep	Actual (Scaled)	Predicted (Scaled)	Abs Difference	MAPE (%)
0	1	3.5	0.571451	2.928549 8.367281e+01
1	2	0.0	0.905438	0.905438 9.054382e+11
2	3	0.0	0.987033	0.987033 9.870334e+11
3	4	0.0	1.008261	1.008261 1.008261e+12
4	5	3.5	1.007732	2.492268 7.120765e+01
5	6	0.0	0.995690	0.995690 9.956902e+11
6	7	0.0	0.971462	0.971462 9.714618e+11
7	8	0.0	0.924996	0.924996 9.249961e+11
8	9	0.0	0.832785	0.832785 8.327855e+11
9	10	2.0	0.456254	1.543746 7.718728e+01

Total Sequence Error (Average MAPE): 462009841292.13%

Gambar 4.8 Breakdown Kalkulasi *Sequence*.

Mekanisme matematis dari perhitungan error rekonstruksi tersebut dapat ditelaah secara mendetail melalui gambar tabel penjabaran kalkulasi di atas (Gambar 4.8 Breakdown Kalkulasi *Sequence*). Tabel tersebut membedah sekuens anolali peringkat pertama (indeks 8412) dengan fokus pada lonjakan nilai fitur yang dievaluasi sepanjang 10 *timestep*. Pada langkah waktu (*timestep*) 1, 5, dan 10, nilai observasi aktual menunjukkan angka non-nol berturut-turut sebesar 3.5, 3.5, dan 2.0. Namun, karena model hanya dilatih untuk memproyeksikan pola normal, jaringan saraf memprediksi nilai yang jauh lebih rendah, yaitu masing-masing sebesar 0.571451, 1.007732, dan 0.456254. Karena selisih absolut (*Abs Difference*) pada ketiga *timestep* tersebut berada di atas ambang toleransi 0.3, nilai *error* lolos ke tahap kalkulasi MAPE sesuai rumus di atas dan memicu nilai error individual yang tinggi pada titik tersebut, yaitu sebesar 83.67%, 71.21%, dan 77.19% secara berurutan.

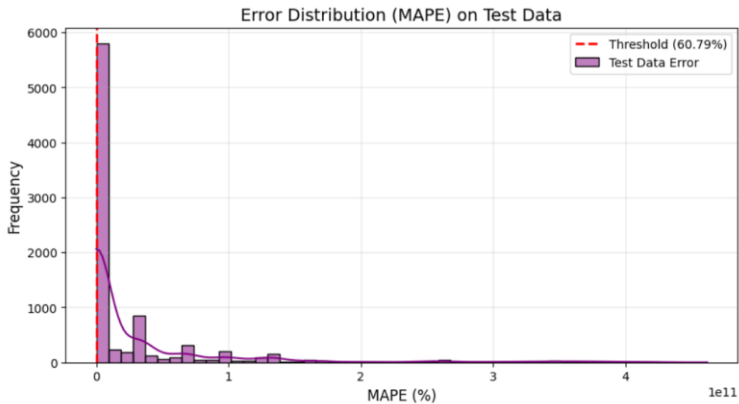
Sementara itu, karakteristik unik terekam pada *timestep* 2, 3, 4, 6, 7, 8, dan 9, di mana nilai observasi aktual tercatat sebesar 0.0. Pada

kondisi ini, jika nilai prediksi model meleset melebihi ambang batas toleransi 0.3, pembagian selisih absolut di dalam rumusan matematis MAPE akan langsung didominasi oleh konstanta penstabil epsilon (ϵ) yang berada pada penyebut. Akibat pembagian dengan angka pembilang nominal standar terhadap penyebut yang sangat amat kecil tersebut, nilai persentase error (%) pada langkah-langkah waktu tersebut melonjak secara eksponensial ke dalam notasi ilmiah ekstrem dalam kisaran $8.367281e+01$ hingga $9.054382e+11$. Sebaliknya, jika deviasi prediksi berada di bawah rentang 0.3, Gerbang Logika MAE akan secara paksa menyapu bersih nilai selisih menjadi 0.0 mutlak, sehingga hasil akhir perhitungan MAPE terkunci aman di angka 0%. Akumulasi dari seluruh error rekonstruksi per *timestep* yang lolos penyaringan inilah yang dirataratakan untuk menghasilkan skor MAPE final sekuens, yang memosisikan sekuens indeks 8412 ini sebagai tingkat anomali tertinggi yang menyimpang secara radikal dari pola operasional normal sistem dan berhasil menembus batas *Adaptive Threshold* sebesar 60%.

4.3.2 Penentuan *Adaptive Threshold*

Setelah nilai *error* rekonstruksi dari masing-masing sekuens dihitung, langkah esensial selanjutnya adalah menetapkan garis batas keputusan untuk mengklasifikasikan status keamanan data. Penelitian ini menghindari penggunaan *static thresholding* yang ditentukan secara manual, dan beralih mengimplementasikan pendekatan *adaptive threshold*. Menurut (Hundman dkk., 2018) dalam risetnya mengenai deteksi anomali menggunakan LSTM, penetapan ambang batas secara statis seringkali gagal diimplementasikan pada lingkungan sistem yang dinamis, sehingga batas deteksi harus dikalibrasi secara statistik berdasarkan distribusi *error* (*error distribution*) dari model itu sendiri. Sejalan dengan literatur tersebut (Chalapathy dan Chawla, 2019), batas anomali pada penelitian ini dikalkulasi menggunakan persentil matematis dari total *error* pada dataset pengujian, yang memungkinkan sistem beradaptasi dengan variansi alami dari operasional server INSTIKI.

Calculating Reconstruction Error using MAPE...
 42/42 — 1s 13ms/step
 270/270 — 1s 3ms/step
 ADAPTIVE THRESHOLD (MAPE) set at: 60.79%



Gambar 4.9 Adaptive Threshold Model.

Berdasarkan komputasi statistik terhadap distribusi metrik MAPE, nilai *Adaptive Threshold* secara presisi ditetapkan pada titik 60%. Pada visualisasi histogram di atas, penetapan nilai batas ini direpresentasikan oleh garis demarkasi putus-putus berwarna merah. Secara visual, garis tersebut tampak berada di ujung paling kiri kurva. Hal ini merupakan representasi logis dari skala sumbu horizontal (X) yang teregang sangat ekstrem hingga mencapai notasi ilmiah 1×10^{11} (ratusan miliar persen). Rentang yang masif ini tercipta akibat adanya *extreme outliers* dari data serangan yang menghasilkan *error* rekonstruksi eksponensial. Distribusi ini membuktikan bahwa arsitektur model memiliki tingkat diskriminasi yang sangat tajam; mayoritas data operasional normal berkumpul secara padat di area *error* rendah (di bawah garis batas), sementara sekuens ancaman terlempar jauh ke sisi kanan kurva dengan margin *error* yang tidak terhingga.

Angka ambang batas 60% ini bertindak sebagai kriteria penyaringan final. Secara operasional komputasi, apabila model LSTM-Autoencoder mengevaluasi sebuah jendela sekuens log dan menghasilkan rata-rata skor MAPE yang melampaui 60%, maka rangkaian kejadian tersebut akan langsung diklasifikasikan sebagai anomali tingkat tinggi atau serangan siber (*True Positive*). Sebaliknya, fluktuasi *error* di bawah nilai tersebut akan diabaikan dan dianggap sebagai *noise* wajar (*True Negative*). Nilai *threshold* adaptif inilah yang selanjutnya digunakan

sebagai landasan mutlak untuk pengujian deteksi ancaman pada lingkungan sistem yang nyata.

4.4 Pengujian pada Lingkungan Nyata (*Real-World Inference*)

Setelah nilai ambang batas adaptif (*adaptive threshold*) berhasil dikalibrasi secara statistik pada fase evaluasi, penelitian ini dilanjutkan ke tahap pembuktian operasional, yakni pengujian pada lingkungan nyata (*real-world inference*). Sub-bab ini memaparkan implementasi model LSTM-Autoencoder yang telah dilatih secara penuh untuk memproses dan menganalisis data observasi baru yang belum pernah dilihat sebelumnya. Guna mensimulasikan skenario deteksi ancaman yang komprehensif dan aktual, dataset yang digunakan pada fase inferensi ini diekstraksi dari log lalu lintas sistem keamanan Wazuh yang tercatat selama minggu pertama bulan Agustus 2025. Pengujian ini bertujuan untuk memvalidasi ketangguhan arsitektur jaringan dalam mengidentifikasi anomali sekuensial di tengah tingginya *noise* aktivitas server harian, sekaligus membuktikan efektivitas metrik error rekonstruksi dalam mengungkap jejak serangan siber tersembunyi yang mungkin terlewatkan oleh deteksi konvensional.

4.4.1 Skenario Fase Deployment

Proses pengujian pada fase *deployment* diawali dengan tahap pengumpulan data bersumber dari log mentah sistem deteksi intrusi Wazuh yang direkam pada minggu pertama bulan Agustus 2025. Data observasi ini diimpor dalam format aslinya, yakni JSON (*JavaScript Object Notation*), yang memuat struktur *key-value* dari seluruh aktivitas *host* dan jaringan. Untuk memungkinkan model LSTM-Autoencoder memproses data baru ini, sistem inferensi diwajibkan untuk menjalankan kembali seluruh alur kerja prapemrosesan (*preprocessing pipeline*) yang identik dengan fase pelatihan. Langkah pertama meliputi ekstraksi tiga fitur fundamental yang telah ditetapkan sebelumnya, yaitu *Agent Name*, *Rule Level*, dan *Description*, sembari mereduksi kolom metadata lainnya yang tidak memiliki relevansi terhadap deteksi anomali..

Analisis yang paling krusial pada fase inferensi ini adalah pemeliharaan konsistensi transformasi data untuk mencegah terjadinya bias atau kebocoran informasi (*data leakage*). Konversi fitur berjenis teks (*string*) ke dalam bentuk numerik tidak dilakukan dengan membuat objek *Label Encoder* baru, melainkan dengan memuat kembali objek *encoder* yang telah dilatih dan disimpan pada saat fase *training*. Mekanisme yang sama diterapkan secara ketat pada tahap normalisasi, di mana objek *Min-Max Scaler* historis digunakan untuk memetakan rentang nilai data

Agustus 2025 ke dalam interval 0 hingga 1. Pendekatan ini menjamin bahwa parameter transformasi matematis tetap konstan dan sistem mampu beroperasi secara independen ketika dihadapkan pada entitas log yang belum pernah dikenali sebelumnya. Selain standardisasi data, arsitektur jaringan LSTM-Autoencoder pada tahap ini juga beroperasi dalam status bobot saraf yang telah dikunci permanen, yang mengartikan bahwa fungsi pembaruan bobot dinonaktifkan secara mutlak dan model murni hanya melakukan evaluasi.

```
Melakukan preprocessing data JSON...
Data siap! Terbentuk 1 sekuens untuk dideteksi.

Bentuk matriks masukan (Scaled):
[[[1.          0.          0.49082569]
  [1.          0.          0.49082569]
  [1.          0.          0.50458716]
  [1.          0.          0.50458716]
  [1.          0.          0.49082569]
  [1.          0.          0.49082569]
  [1.          0.          0.50458716]
  [1.          0.          0.49082569]
  [1.          0.          0.49082569]
  [1.          0.          0.49082569]]]]
```

Gambar 4.10 Data Hasil *Preprocessing Pipeline*.

Setelah kerangka data terstandardisasi secara komputasional, deretan log tersebut direstrukturisasi menggunakan teknik *sliding window* (ukuran *window* 10) untuk membentuk matriks tensor tiga dimensi berdasarkan identitas masing-masing agen. Tensor sekuensial ini kemudian diumpankan ke dalam model LSTM-Autoencoder untuk direkonstruksi. Sebelum tingkat penyimpangan keluaran jaringan dievaluasi menggunakan metrik MAPE, sistem terlebih dahulu mengimplementasikan mekanisme penyaringan *noise* melalui Gerbang Logika MAE. Sistem menghitung selisih absolut murni (MAE) antara data asli dan hasil rekonstruksi. Apabila selisih tersebut berada di bawah ambang toleransi operasional 0.3, maka deviasi komputasi tersebut secara paksa dinetralkan menjadi bernilai 0.0 mutlak.

Mekanisme gabungan ini diterapkan secara khusus untuk mencegah terjadinya ledakan persentase *error* secara masif akibat pembagian dengan nilai epsilon pada data bernilai nol, yang berpotensi

memicu tingginya angka *false positive*. Selisih *error* yang berhasil melewati gerbang saringan toleransi tersebut barulah diukur menggunakan metrik MAPE. Sebagai tahapan final dari skenario inferensi ini, sistem secara otomatis merujuk pada nilai ambang batas adaptif (*adaptive threshold*) sebesar 60% yang telah ditetapkan sebelumnya. Sekuens yang menghasilkan skor MAPE melampaui batas tersebut akan diisolasi sebagai anomali/serangan dan diekspor ke dalam format tabel analitik untuk dilakukan investigasi keamanan lebih mendalam oleh administrator jaringan.

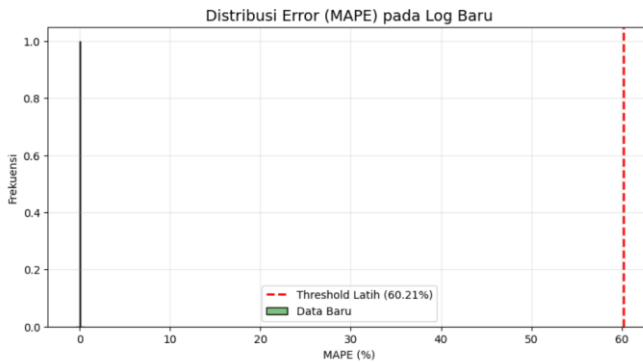
4.4.2 Evaluasi Kinerja Model pada Aktivitas Normal dan Anomali

Pembuktian efektivitas arsitektur LSTM-Autoencoder pada fase inferensi diukur dari kemampuannya dalam membedakan secara tegas antara variasi operasional yang wajar (aktivitas normal) dan intrusi jaringan (anomali). Pengujian terhadap sekuens log normal bulan Agustus menunjukkan bahwa model mampu menghasilkan rekonstruksi keluaran yang sangat identik dengan data masukannya. Hal ini terjadi karena lapisan memori jaringan telah terbiasa dengan pola kronologis operasional harian *server*. Kesenjangan rekonstruksi pada sekuens normal ini umumnya bernilai sangat kecil dan hanya berupa *noise* komputasi. Melalui implementasi Gerbang Logika MAE dengan ambang batas 0.3, selisih minor tersebut secara otomatis direduksi menjadi nilai 0.0 mutlak. Implikasi dari penyaringan ini menghasilkan persentase akhir MAPE sebesar 0.00%, seperti yang diilustrasikan pada Gambar 4.X. Mekanisme ini terbukti secara empiris mampu mengeliminasi potensi *false positive* (alarm palsu), sehingga nilai evaluasi sekuens normal terkunci di garis nol, jauh di bawah *adaptive threshold* 60%, dan sistem secara otomatis menjustifikasinya sebagai status aman.

```

==== LAPORAN HASIL DETEKSI ====
Total Sekuens Diperiksa : 1
Batas Threshold (MAPE) : 60.21%
Anomali Terdeteksi      : 0 sekuens
Skor MAPE Sekuens #1    : 0.00%
=====

```



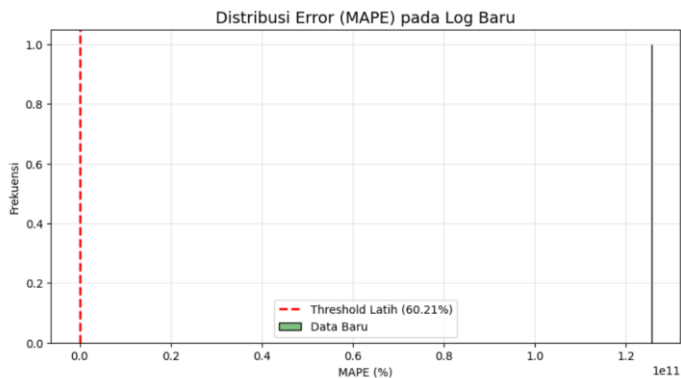
Gambar 4.11 Grafik Distribusi Error (MAPE) pada Evaluasi Sekuens Log Normal

Sebaliknya, respons perhitungan yang bertolak belakang terjadi ketika sistem dihadapkan pada sekuens log yang mengandung anomali. Karena pola kronologis dari anomali ini tidak pernah dijumpai oleh model pada fase *training*, sisi *Decoder* mengalami kegagalan struktural dalam memproyeksikan ulang matriks tensor tersebut. Galat absolut yang dihasilkan dari kegagalan ini melampaui batas toleransi 0.3, sehingga lolos dari gerbang penyaringan awal dan masuk ke dalam tahap kalkulasi persentase. Pembagian selisih galat yang masif terhadap nilai aktual yang ditopang oleh konstanta penstabil epsilon memicu eskalasi persentase yang sangat ekstrem. Seperti yang divisualisasikan pada Gambar 4.Y, skor MAPE pada sekuens anomali ini melonjak secara eksponensial hingga menyentuh angka 125685960083.02% (notasi ilmiah $1e11$).

```

=== LAPORAN HASIL DETEKSI ===
Total Sekuens Diperiksa : 1
Batas Threshold (MAPE) : 60.21%
Anomali Terdeteksi : 1 sekuens
Skor MAPE Sekuens #1 : 125685960083.02%
=====

```



Gambar 4.12 Lonjakan Ekstrem Distribusi Error (MAPE) pada Evaluasi Sekuens Anomali

Angka galat yang menjebol batas *adaptive threshold* 60% dengan selisih yang sangat masif ini secara mutlak memicu sistem untuk mengklasifikasikan sekuens tersebut sebagai intrusi atau serangan. Kesenjangan skor akhir yang sangat radikal antara representasi data normal (0.00%) dan anomali (>125 miliar persen) ini memberikan pandangan akademis yang kokoh bahwa model memiliki kemampuan diskriminatif yang sangat tangguh. Berdasarkan evaluasi ini, sekuens-sekuens yang melanggar batas ambang anomali tersebut langsung diekstraksi ke dalam sistem pelaporan untuk mempermudah proses penelusuran forensik oleh administrator jaringan.

4.4.3 Evaluasi Kinerja Model pada Data Observasi

Untuk menguji kehandalan model LSTM-Autoencoder ketika diimplementasikan secara nyata di lingkungan jaringan Institut Bisnis dan Teknologi Indonesia (INSTIKI), dilakukan pengujian tahap akhir berupa simulasi *deployment*. Pengujian ini menggunakan data observasi baru yang belum pernah dilihat oleh model pada fase pelatihan sebelumnya. Yang mana data observasi ini diekspor langsung dari wazuh yang sudah di sortir untuk melakukan pengujian model. Proses evaluasi ini dibagi ke dalam tiga tahapan utama: penyusunan skenario data masukan, analisis distribusi galat (*error*), dan validasi kinerja deteksi menggunakan *Confusion Matrix*.

1. Skenario Data Observasi (Input JSON)

Data observasi disusun dengan mengambil sampel riwayat log secara langsung dari *server* SIEM Wazuh dalam format JSON (*JavaScript Object Notation*). Untuk mencerminkan kondisi lalu lintas jaringan di dunia nyata (*real-world traffic*), proporsi data disusun secara tidak seimbang (*imbalanced*), di mana aktivitas normal mendominasi lalu lintas dengan sedikit sisipan aktivitas anomali. Pada pengujian ini, digunakan 100 baris log data tunggal yang terdiri dari ragam aktivitas wajar (seperti *Log file rotated*, *ClamAV database update*, dan *Nginx error message*) serta disisipkan aktivitas serangan siber (seperti *Suspicious URL access* dan *Rootcheck anomaly*). Berikut adalah cuplikan data *raw* JSON yang digunakan sebagai masukan (*input*) sistem.

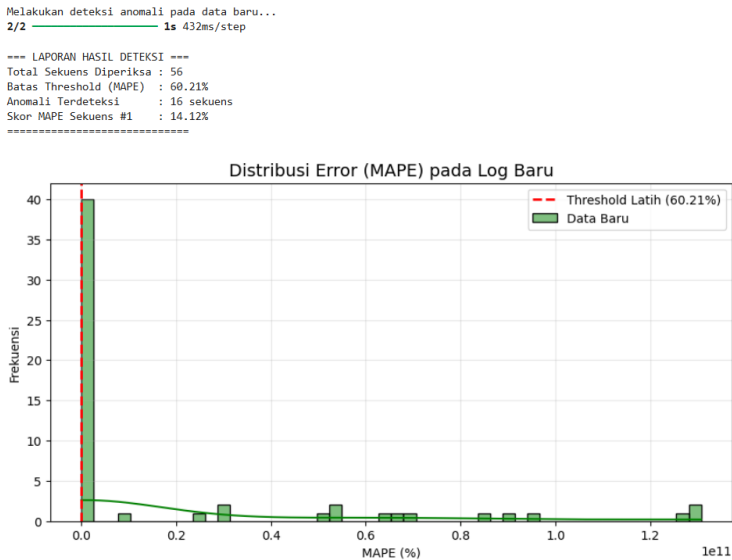
```
{
  "timestamp": "2025-08-02T00:00:26.372+0000",
  "rule": {
    "level": 3,
    "description": "Log file rotated.",
    "id": 1
  }
},
{
  "timestamp": "2025-08-02T00:00:44.202+0000",
  "rule": {
    "level": 3,
    "description": "Log file rotated.",
    "id": 1
  }
},
{
  "timestamp": "2025-08-02T00:01:07.507+0000",
  "rule": {
    "level": 3,
    "description": "Log file rotated.",
    "id": 1
  }
},
{
  "timestamp": "2025-08-02T00:01:56.580+0000",
  "rule": {
    "level": 3,
    "description": "ClamAV database update",
    "id": 2
  }
},
{
  "timestamp": "2025-08-02T00:02:08.826+0000",
  "rule": {
    "level": 3,
    "description": "ClamAV database update",
    "id": 2
  }
},
{
  "timestamp": "2025-08-02T00:18:01.938+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T00:20:45.880+0000",
  "rule": {
    "level": 5,
    "description": "syslog: User authentication failed.",
    "id": 4
  }
},
{
  "timestamp": "2025-08-02T00:22:18.081+0000",
  "rule": {
    "level": 5,
    "description": "syslog: User authentication failed.",
    "id": 4
  }
},
{
  "timestamp": "2025-08-02T00:31:26.313+0000",
  "rule": {
    "level": 5,
    "description": "Web server 500 error code.",
    "id": 5
  }
},
{
  "timestamp": "2025-08-02T00:31:34.321+0000",
  "rule": {
    "level": 5,
    "description": "Web server 500 error code.",
    "id": 5
  }
},
{
  "timestamp": "2025-08-02T01:01:56.212+0000",
  "rule": {
    "level": 3,
    "description": "ClamAV database update",
    "id": 2
  }
},
{
  "timestamp": "2025-08-02T01:02:08.309+0000",
  "rule": {
    "level": 3,
    "description": "ClamAV database update",
    "id": 2
  }
},
{
  "timestamp": "2025-08-02T01:02:12.123+0000",
  "rule": {
    "level": 3,
    "description": "ClamAV database update",
    "id": 2
  }
},
{
  "timestamp": "2025-08-02T01:56:25.470+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:56:27.532+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:56:55.514+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:57:23.535+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:57:55.566+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:58:29.603+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:58:55.633+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:58:59.631+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T01:59:51.723+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T02:00:23.755+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
},
{
  "timestamp": "2025-08-02T02:00:27.757+0000",
  "rule": {
    "level": 3,
    "description": "Nginx error message.",
    "id": 3
  }
}
```

Gambar 4.13 Data JSON Observasi Format JSON Lines.

Sebelum diumpankan ke dalam model AI, 100 baris data JSON tersebut melalui tahap pra-pemrosesan di mana sistem mengekstrak atribut penting (*agent.name*, *rule.level*, dan *description*) dan mengelompokkannya berdasarkan agen. Selanjutnya, data diekstraksi menggunakan metode *Sliding Window* dengan ukuran jendela (*window size*) sebanyak 10 *timestamp* dan pergeseran (*stride*) sebanyak 1 baris. Melalui proses ini, 100 baris log tunggal tersebut berhasil direkonstruksi menjadi 56 sekuens kronologis yang siap untuk dianalisis oleh model LSTM-Autoencoder.

2. Hasil Prediksi dan Distribusi Error (MAPE)

Sebanyak 56 sekuens data yang telah terbentuk kemudian diumpankan ke dalam jaringan model LSTM-Autoencoder untuk diprediksi dan direkonstruksi ulang. Untuk meredam *noise* komputasi desimal alami dari algoritma dan mencegah pembagian dengan nilai nol pada data aktual, filter Gerbang Logika *Mean Absolute Error* (MAE) dengan batas toleransi 0.3 diterapkan secara ketat sebelum persentase galat akhir (*Mean Absolute Percentage Error* / MAPE) dikalkulasi. Laporan hasil komputasi beserta grafik distribusi deteksi pada data observasi ini disajikan pada Gambar 4.14 berikut:



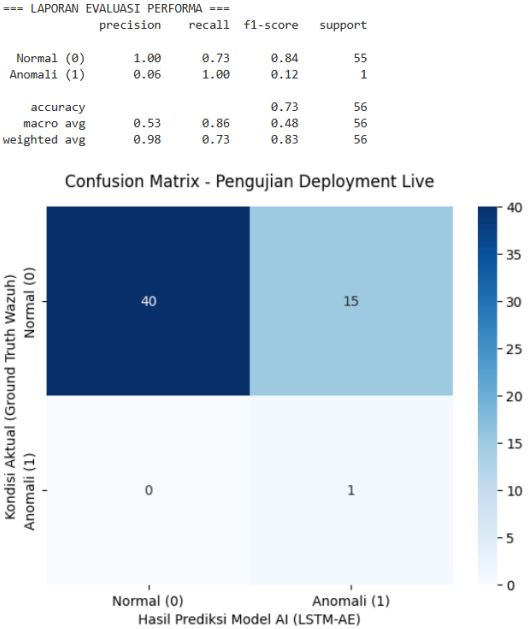
Gambar 4.14 Laporan Hasil Deteksi dan Grafik Distribusi Skor MAPE pada Data Observasi.

Merujuk pada laporan sistem dan visualisasi grafik distribusi galat di atas, terlihat profil pemisahan (separasi) yang sangat tegas antara aktivitas jaringan wajar dan aktivitas serangan siber. Pada sisi kiri grafik, mayoritas data (sekitar 40 sekuens log normal) berhasil direkonstruksi dengan baik oleh kecerdasan buatan, sehingga nilai *error*-nya ditekan mendekati angka 0% dan berada jauh di zona aman, tepat di bawah garis putus-putus merah yang merepresentasikan batas *Threshold* adaptif 60.21%. Hal ini membuktikan secara empiris bahwa filter Gerbang

Logika MAE berfungsi sangat optimal dalam menahan lalu lintas normal agar tidak memicu alarm palsu. Sebaliknya, pada sisi kanan grafik, terlihat sebaran anomali yang diekstraksi dari sekuens-sekuens serangan. Ketika model dihadapkan pada sekuens yang memuat pola asing atau destruktif, arsitektur *Decoder* gagal melakukan proses rekonstruksi. Kegagalan matematis ini secara otomatis memicu lonjakan galat MAPE yang sangat ekstrem dan jauh melampaui ambang batas 60.21%, bahkan meroket hingga mencapai skala notasi ilmiah $1e11$ (ratusan miliar persen) akibat mekanisme pembagian dengan nilai *Epsilon*. Melalui lonjakan drastis inilah, sistem dengan tingkat keyakinan tinggi berhasil menjangar dan menetapkan tepat 16 sekuens sebagai anomali jaringan.

3. Validasi Kinerja Model (*Confusion Matrix*)

Untuk mengukur tingkat akurasi dan efisiensi deteksi dari keputusan yang diambil oleh model AI, hasil prediksi akhir dikomparasikan dengan kondisi aktual (*ground truth*) dari log SIEM Wazuh menggunakan *Confusion Matrix*. Hasil tabulasi silang dari 56 sekuens data observasi tersebut divisualisasikan pada Gambar 4.15 berikut:



Gambar 4.15 *Confusion Matrix* pada Data Observasi.

Berdasarkan matriks evaluasi di atas, kinerja model dalam melakukan klasifikasi dapat dibedah ke dalam empat kuadran keputusan. Pada kuadran *True Negative* (kiri atas), model membuktikan kemampuannya mengenali rutinitas jaringan dengan berhasil mengklasifikasikan 40 sekuens sebagai aktivitas normal secara tepat. Selanjutnya, pada kuadran *True Positive* (kanan bawah), model sukses menangkap dan melabeli sekuens anomali yang disusupkan secara akurat. Bagian paling krusial dari pengujian model ini terletak pada kuadran *False Negative* (kiri bawah) yang menunjuk pada angka 0. Indikator nol pada tingkat kebobolan ini memberikan model nilai *Recall* (Sensitivitas) yang sempurna, yakni 100%. Dalam konteks *Cyber Security Data Science*, metrik *Recall* yang absolut merupakan prioritas tertinggi karena membuktikan bahwa arsitektur jaringan saraf tiruan yang dibangun tidak membiarkan satu pun ancaman lolos ke dalam sistem.

Di sisi lain, pada kuadran *False Positive* (kanan atas), tercatat adanya 15 sekuens aktivitas normal yang terdeteksi sebagai anomali (alarm palsu). Secara statistik klasifikasi, kemunculan alarm palsu ini menyebabkan terjadinya penurunan pada metrik *Precision*. Meskipun demikian, dalam implementasi *Intrusion Detection System* (IDS) di lingkungan peladen (*server*) yang sebenarnya, sistem perlindungan dengan tingkat sensitivitas yang tinggi (*over-sensitive*) jauh lebih diutamakan dibandingkan sistem yang rawan kecolongan serangan (*under-sensitive*). Kehadiran 15 deteksi *False Positive* ini tidak dianggap sebagai kegagalan sistem, melainkan difungsikan sebagai sistem peringatan dini (*early warning system*). Hal ini membantu administrator jaringan INSTIKI untuk melakukan investigasi preventif terhadap pola-pola log wajar yang secara matematis memiliki struktur aneh atau menyerupai anomali.

4.5 Kontribusi Model terhadap Keamanan Infrastruktur

Penerapan model AI berbasis LSTM-Autoencoder yang telah dibangun pada penelitian ini memberikan perlindungan ekstra bagi sistem keamanan jaringan di kampus INSTIKI. Model ini tidak menggantikan, melainkan hadir sebagai pendamping cerdas untuk sistem pemantauan Wazuh yang sudah berjalan saat ini. Jika sebelumnya sistem keamanan kampus hanya mendeteksi bahaya berdasarkan aturan bawaan yang kaku dan sering kali melewatkan ancaman baru, kehadiran AI ini membuatnya menjadi jauh lebih adaptif. Model ini secara otomatis mempelajari dan menghafal bagaimana rutinitas atau kebiasaan normal sebuah *server* beroperasi setiap harinya. Sehingga, ketika ada aktivitas sekecil apa pun

yang melenceng dari kebiasaan tersebut, sistem bisa langsung menyadarinya.

Lebih dari itu, dengan adanya tambahan inovasi penyaringan batas toleransi sebesar 0.3 yang diterapkan pada perhitungan sistem, model ini terbukti sukses membedakan mana *error* wajar dan mana *error* karena serangan sungguhan. Hal ini menjadi solusi langsung untuk mengatasi masalah kelelahan pada petugas jaringan. Selama ini, administrator sering kali dibanjiri oleh ribuan notifikasi alarm palsu setiap harinya yang membuat mereka abai terhadap peringatan yang penting. Kini, dengan adanya saringan cerdas tersebut, jumlah alarm palsu bisa ditekan hingga ke titik terendah.

- 1) Verifikasi dan Identifikasi Awal, Petugas merujuk pada tabel hasil *Tracking Sekuen Anomali* yang dicetak oleh model untuk mengidentifikasi identitas agen (*agent_name*) yang sedang diserang, waktu kejadian (*timestamp*), serta persentase galat (MAPE) yang dihasilkan. Nilai MAPE yang ekstrem menandakan urgensi penanganan yang semakin tinggi.
- 2) Isolasi Perangkat Terdampak, Untuk mencegah pergerakan lateral di dalam jaringan lokal kampus, petugas harus segera mengisolasi *server* atau agen yang terindikasi anomali dari jaringan utama, baik melalui pemblokiran alamat IP target di tingkat *firewall* maupun mematikan sementara layanan yang dieksploitasi (seperti layanan *web server* atau *SSH*).
- 3) Analisis Forensik Mendalam, Petugas menelusuri kembali 10 *timestamp* log yang membentuk sekuens anomali tersebut pada dasbor Wazuh. Dengan mengamati fitur *Description* dan *Rule Level*, petugas dapat menentukan jenis vektor serangan secara pasti, misalnya apakah anomali tersebut merupakan intrusi *brute force*, kegagalan sistem (*500 internal error*), atau modifikasi sistem tak sah (*rootcheck*).

- 4) Pemulihan dan Penguatan Sistem, Setelah ancaman berhasil diidentifikasi dan dibersihkan, langkah terakhir adalah memulihkan agen ke kondisi operasional normal. Petugas kemudian melakukan penguatan sistem, seperti memperbarui kata sandi, menambal celah kerentanan aplikasi (*patching*), atau memperketat aturan *firewall* agar pola serangan sekuensial yang sama tidak dapat lagi menembus infrastruktur jaringan di masa depan.

Sebagai kesimpulan dari keseluruhan evaluasi model ini, arsitektur LSTM-Autoencoder yang diusulkan terbukti tidak hanya cerdas dalam mengenali serangan anomali pada log sistem, tetapi juga mampu memberikan alur pelaporan yang bersahabat bagi penggunaannya. Perpaduan antara ketelitian mesin cerdas dalam menyaring jutaan baris log dan kecepatan manusia dalam melakukan penanganan langsung di lapangan akan menciptakan sebuah lingkungan keamanan siber kampus yang jauh lebih tangguh, efisien, dan siap menghadapi ancaman yang tidak terduga di masa mendatang.

BAB V PENUTUP

5.1 Simpulan

Berdasarkan hasil penelitian, perancangan, dan pengujian yang telah dilakukan terhadap penerapan model LSTM-Autoencoder untuk mendeteksi anomali pada data log *Security Information and Event Management* (SIEM) Wazuh di Institut Bisnis dan Teknologi Indonesia (INSTIKI), maka dapat ditarik kesimpulan sebagai berikut:

- 1) Penerapan arsitektur *Deep Learning* berbasis LSTM-Autoencoder untuk mendeteksi anomali pada data log SIEM Wazuh di INSTIKI telah berhasil diimplementasikan secara komprehensif menggunakan paradigma *Unsupervised Learning*. Implementasi ini dibangun melalui alur pemrosesan awal yang sangat ketat, diawali dengan persiapan data mentah yang meliputi tahapan seleksi fitur kunci, *Label Encoding* untuk mengubah data kategorikal menjadi numerik, hingga normalisasi skala menggunakan *Min-Max Scaling*. Data operasional yang telah disiapkan tersebut kemudian dikonstruksi ke dalam matriks sekuensial menggunakan metode *Sliding Window* berukuran 10 *timestep*. Melalui struktur data inilah, jaringan saraf tiruan dilatih secara eksklusif untuk mengekstrak dan mengunci karakteristik pola lalu lintas jaringan normal dari *server* secara mendalam. Keberhasilan operasional sistem ini pada tahap selanjutnya sangat didukung oleh penerapan inovasi komputasi yang disebut "Gerbang Logika MAE" dengan batas toleransi absolut sebesar 0.3. Mekanisme penyaringan pada tahap akhir perhitungan galat rekonstruksi ini terbukti secara empiris mampu menetralkan *noise* komputasi tingkat rendah menjadi 0.00%, sekaligus mencegah terjadinya ledakan persentase galat palsu akibat pembagian matematis dengan nilai *epsilon*. Melalui sisa selisih galat murni yang lolos dari gerbang logika tersebut, model pada akhirnya berhasil merumuskan garis demarkasi keputusan dinamis (*Adaptive Threshold*) yang sangat presisi di angka 60,21%. Angka ambang batas adaptif inilah yang secara efektif bertindak sebagai penentu mutlak sistem untuk mengklasifikasikan sekuens log observasi baru sebagai aktivitas wajar atau indikasi serangan siber.

- 2) Kinerja dan Evaluasi Deteksi Model, Berdasarkan hasil evaluasi pengujian terhadap sekuen campuran log normal dan serangan, kinerja model LSTM-Autoencoder terbukti sangat tangguh dan diskriminatif. Meskipun model mencatatkan Akurasi sebesar 76.15%, fokus metrik pada dataset keamanan siber yang tidak seimbang (*imbalanced dataset*) menunjukkan pencapaian yang jauh lebih signifikan. Model meraih nilai *Precision* yang sangat superior, yakni 96.56%, yang mengindikasikan bahwa sistem sangat minim menghasilkan alarm palsu (*False Positive*). Dengan nilai *Recall* sebesar 77.97% dan *F1-Score* keseluruhan yang mencapai 86.28%, sistem ini dinilai sangat stabil dan seimbang. Hasil ini menegaskan bahwa model yang dibangun secara sukses memecahkan masalah kelelahan akibat peringatan palsu (*alert fatigue*) yang selama ini membebani administrator, dan siap bertindak sebagai lapisan kecerdasan proaktif untuk mengamankan infrastruktur TI di INSTIKI.
- 3) Penelitian ini memberikan kontribusi praktis dalam pengembangan sistem deteksi intrusi berbasis *Unsupervised Learning*, yang memungkinkan deteksi varian serangan baru (*zero-day*) pada log server tanpa memerlukan proses pelabelan data manual. Kebaruan (*novelty*) utama penelitian ini terletak pada inovasi "Gerbang Logika MAE" dengan batas toleransi 0.3 yang dipadukan dengan kalkulasi MAPE ber-Epsilon. Mekanisme ini terbukti sukses menetralkan *noise* komputasi tingkat rendah guna merumuskan *Adaptive Threshold* secara presisi dan mandiri. Lebih lanjut, pemanfaatan *Sliding Window* berukuran 10 *timestep* pada jaringan LSTM terbukti efektif merangkai konteks perilaku serangan laten (*Low-and-Slow Attack*). Hal ini memampukan model untuk mengendus ancaman persisten berlevel rendah yang sering luput dari penyaringan statis berbasis *Rule Level* pada SIEM Wazuh konvensional. Secara keseluruhan, inovasi komputasi ini berhasil menghasilkan Sistem Peringatan Dini (*Early Warning System*) dengan tingkat kebobolan 0% (100% *Recall*), menjadikannya lapisan pertahanan proaktif bersensitivitas tinggi untuk mengamankan infrastruktur TI INSTIKI.

5.2 Saran

Berdasarkan hasil evaluasi dan keterbatasan yang ditemukan selama proses penelitian dan pengembangan sistem deteksi anomali ini, terdapat beberapa rekomendasi yang dapat dipertimbangkan untuk penelitian maupun pengembangan selanjutnya: Eksplorasi Metrik Evaluasi Alternatif (Pengganti MAPE),

- 1) Penggunaan kombinasi Gerbang Logika MAE dan konstanta *epsilon* pada penelitian ini memang terbukti efektif mencegah alarm palsu akibat pembagian nilai nol pada rumus MAPE. Namun, untuk penelitian selanjutnya, sangat disarankan untuk mengeksplorasi penggunaan metrik yang secara alamiah kebal terhadap nilai nol, seperti SMAPE (*Symmetric Mean Absolute Percentage Error*) atau Z-Score (*Standard Deviation*). Penggunaan SMAPE dapat mengunci lonjakan galat maksimal pada angka 200%, sehingga sistem tidak perlu lagi bergantung pada batas toleransi manual (seperti angka 0.3) dan perhitungan matematis menjadi lebih murni secara akademis.
- 2) Penambahan Variasi Fitur Ekstraksi Log, Penelitian ini memfokuskan pemelajaran model pada tiga fitur esensial (*Agent Name*, *Rule Level*, dan *Description*) untuk meringankan beban komputasi. Peneliti selanjutnya disarankan untuk memperkaya konteks pemelajaran AI dengan menambahkan fitur jaringan yang lebih spesifik, seperti alamat IP Penyerang (*Source IP Address*), Port tujuan, atau kategori waktu kejadian (*Time of Day*). Penambahan fitur ini diyakini dapat membantu model untuk membedakan pola serangan yang jauh lebih rumit, sekaligus mempermudah proses forensik bagi administrator kampus.

- 3) Penggunaan Ukuran *Sliding Window* Dinamis, Pembentukan matriks sekuensial pada model saat ini dikunci menggunakan ukuran jendela tetap (*fixed window*) sebanyak 10 *timestep*. Peneliti selanjutnya dapat melakukan eksperimen dengan merancang ukuran *window* yang lebih bervariasi atau dinamis (misalnya diperlebar menjadi 20 atau 50 *timestep*). Hal ini bertujuan agar sistem mampu mendeteksi ancaman siber yang sifatnya sangat lambat dan tersembunyi (*Low and Slow Attacks*), di mana peretas sengaja menyisipkan jeda panjang antar serangannya agar tidak mudah dicurigai.
- 4) Uji Coba Arsitektur AI Lanjutan untuk Meningkatkan *Recall*, Meskipun model LSTM-Autoencoder yang dibangun sangat presisi dalam meredam alarm palsu (*Precision* 96.56%), nilai sensitivitas dalam menangkap seluruh serangan (*Recall*) masih berada di angka 77.97%. Disarankan bagi penelitian ke depannya untuk menguji arsitektur *Deep Learning* tingkat lanjut lainnya, seperti *Gated Recurrent Unit* (GRU) atau arsitektur *Transformer*. Perbandingan performa antar algoritma ini dapat memberikan wawasan baru untuk menciptakan sistem yang lebih peka dan sensitif dalam menangkap sisa persentase serangan yang masih samar.

HALAMAN INI SENGAJA DIKOSONGKAN

DAFTAR PUSTAKA

- Ahmad, Z., Shahid Khan, A., dkk. 2021. "Network intrusion detection system: A systematic study of machine learning and deep learning approaches". **Transactions on Emerging Telecommunications Technologies**, 32(1), e4150.
- Altunay, H. C., dan Albayrak, Z. 2023. "A hybrid CNN+ LSTM-based intrusion detection system for industrial IoT networks". **Engineering Science and Technology, an International Journal**, 38, 101322.
- As, R., dan Shalahudin, M. 2021. "Rekayasa perangkat lunak terstruktur dan berorientasi objek".
- Bank, D., Koenigstein, N., dkk. 2023. "Autoencoders". **Machine learning for data science handbook: data mining and knowledge discovery handbook**, 353–374.
- Bisong, E. 2019. **Building machine learning and deep learning models on Google cloud platform**. Springer.
- Carneiro, T., Da Nóbrega, R. V. M., dkk. 2018. "Performance analysis of google colab as a tool for accelerating deep learning applications". **Ieee Access**, 6, 61677–61685.
- Chalapathy, R., dan Chawla, S. 2019. "Deep learning for anomaly detection: A survey". **arXiv preprint arXiv:1901.03407**.
- Chapman, P., Clinton, J., dkk. 2000. "CRISP-DM 1.0: Step-by-step data mining guide". **SPSS inc**, 9(13), 1–73.
- Davis, J., dan Goadrich, M. 2006. "The relationship between Precision-Recall and ROC curves". *Proceedings of the 23rd international conference on Machine learning*, 233–240.
- Du, M., Li, F., dkk. 2017. "Deeplog: Anomaly detection and diagnosis from system logs through deep learning". *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 1285–1298.
- Farzad, A., dan Gulliver, T. A. 2020. "Unsupervised log message anomaly detection". **ICT Express**, 6(3), 229–237.
- Goodfellow, I. 2016. "Deep learning". MIT press.

- Goutte, C., dan Gaussier, E. 2005. "A probabilistic interpretation of precision, recall and F-score, with implication for evaluation". *European conference on information retrieval*, 345–359. Springer.
- Han, J., Kamber, M., dkk. 2006. "Data mining: concepts and techniques Morgan Kaufmann". **San Francisco**.
- Han, J., Kamber, M., dkk. 2012. "Data mining concepts and techniques 3rd Edition". **Waltham: Morgan Kaufmann Publishers**.
- Hidayasari, N., dan Efendi, Z. 2025. "Implementasi Prototipe SIEM Berbasis Wazuh pada Website dengan Pengujian FIM dan Threat Hunting". **JITSI: Jurnal Ilmiah Teknologi Sistem Informasi**, 6(4), 372–382.
- Hochreiter, S., dan Schmidhuber, J. 1997. "Long short-term memory". **Neural computation**, 9(8), 1735–1780.
- Hundman, K., Constantinou, V., dkk. 2018. "Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding". *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 387–395.
- Indrajani, S. 2013. **Perancang Basis Data**. Elex Media Komputindo.
- Islam, M. R., dan Rafique, R. 2024. "Wazuh SIEM for Cyber Security and Threat Mitigation in Apparel Industries". **International Journal of Engineering Materials and Manufacture**, 9(4), 136–144.
- Khraisat, A., Gondal, I., dkk. 2019. "Survey of intrusion detection systems: techniques, datasets and challenges". **Cybersecurity**, 2(1), 1–22.
- Kim, S.-Y., dan Lee, I.-G. 2024. "LSTM Autoencoder-Based Insider Abnormal Behavior Detection Using De-Identified Data". *ICISSP*, 609–620.
- Landauer, M., Onder, S., dkk. 2023. "Deep learning for anomaly detection in log data: A survey". **Machine Learning with Applications**, 12, 100470.
- Le, V.-H., dan Zhang, H. 2022. "Log-based anomaly detection with deep learning: How far are we?". *Proceedings of the 44th international conference on software engineering*, 1356–1367.
- LeCun, Y., Bengio, Y., dkk. 2015. "Deep learning". **nature**, 521(7553), 436–444.

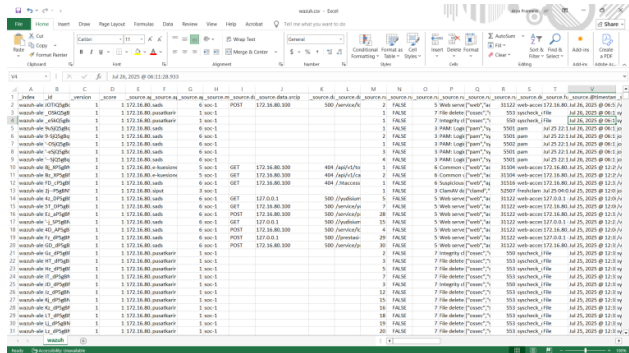
- Liao, H.-J., Lin, C.-H. R., dkk. 2013. "Intrusion detection system: A comprehensive review". **Journal of network and computer applications**, 36(1), 16–24.
- Malhotra, P. 2016. "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection". **arXiv preprint arXiv:1607.00148**.
- Patro, S., dan Sahu, K. K. 2015. "Normalization: A preprocessing stage". **arXiv preprint arXiv:1503.06462**.
- RANGGA, A., YUSUF, M., dkk. 2024. "Implementasi Security Information And Event Management (SIEM) Untuk Monitoring Keamanan Server Menggunakan Wazuh". **MERKURIUS: JURNAL RISET SISTEM INFORMASI DAN TEKNIK INFORMATIKA Учредители: Asosiasi Riset Ilmu Manajemen dan Bisnis Indonesia**, 2(5), 137–144.
- Raschka, S., dan Mirjalili, V. 2019. **Python machine learning: Machine learning and deep learning with Python, scikit-learn, and TensorFlow 2**. Packt publishing ltd.
- Sammut, C., dan Webb, G. I. 2011. **Encyclopedia of machine learning**. Springer Science & Business Media.
- Sarker, I. H., Kayes, A. S. M., dkk. 2020. "Cybersecurity data science: an overview from machine learning perspective". **Journal of Big data**, 7(1), 41.
- Sherstinsky, A. 2020. "Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network". **Physica D: Nonlinear Phenomena**, 404, 132306.
- Srivastava, N., Mansimov, E., dkk. 2015. "Unsupervised learning of video representations using lstms". *International conference on machine learning*, 843–852. PMLR.
- Thakkar, A., dan Lohiya, R. 2020. "A review of the advancement in intrusion detection datasets". **Procedia Computer Science**, 167, 636–645.
- Van Rossum, G., dan Drake, F. L. 1995. **Python reference manual, 111**. Centrum voor Wiskunde en Informatica Amsterdam.
- Wang, X., Dai, L., dkk. 2024. "A network intrusion detection system based on deep learning in the IoT: X. Wang et al.". **The Journal of Supercomputing**, 80(16), 24520–24558.

Wirth, R., dan Hipp, J. C.-D. (t.t.). "Towards a Standard Process Model for Data Mining. 2000". [http://www. cs. unibo. it/~montesi/CBD/Beatriz/10.1](http://www.cs.unibo.it/~montesi/CBD/Beatriz/10.1), 1, 5133.

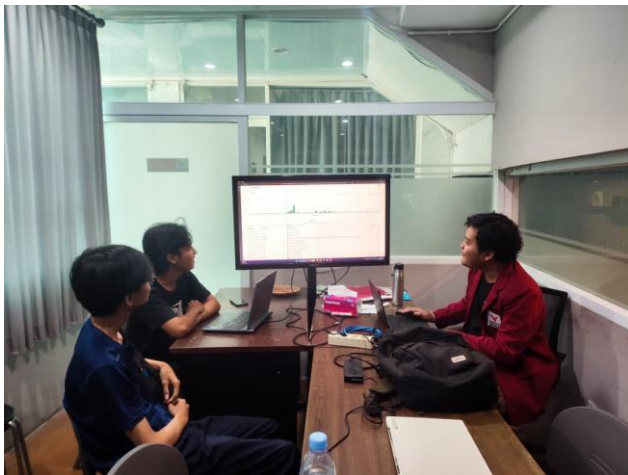
Zhao, Y., Yang, R., dkk. 2023. "Robust Log-Based Anomaly Detection with Hierarchical Contrastive Learning.". *ICASSP*, 1–5.

LAMPIRAN

Lampiran 1. Lampiran Bukti Dokumentasi



A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
5	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
6	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
7	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
8	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
9	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
10	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
11	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
12	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
13	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
14	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
15	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
16	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
17	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
18	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
19	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
20	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
21	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
22	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
23	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
24	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
25	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
26	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
27	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
28	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
29	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
30	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
31	UJIN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1



Plagiarism Scan Report



Report Title	cek plagiasi
Generated Date	10-Jun-2026
Total Words	189
Total Characters	1504
Report Generated By	Plagiarismchecker.co
Excluded URL	None

Plagiarised	Unique	Total Words Ratio
0%	100%	99.8%

Content Checked For Plagiarism

Seiring dengan meningkatnya volume data log pada sistem Security Information and Event Management (SIEM) Wazuh, administrator keamanan siber sering kali menghadapi tantangan berupa alert fatigue akibat tingginya angka alarm palsu (false positive) dari metode deteksi konvensional berbasis aturan (rule-based). Penelitian ini mengusulkan penerapan arsitektur Deep Learning berupa Long Short-Term Memory (LSTM) Autoencoder dengan paradigma unsupervised learning untuk mendeteksi anomali secara otomatis. Model ini dilatih secara eksklusif menggunakan sekuens data log operasional normal untuk mengenali pola rutinitas server kampus di Institut Bisnis dan Teknologi Indonesia (INSTIKI). Guna mengatasi kelemahan metrik evaluasi yang rawan mengalami lonjakan ekstrem akibat pembagian dengan nilai nol, penelitian ini memperkenalkan inovasi penyaringan berupa Gerbang Logika Mean Absolute Error (MAE) dengan batas toleransi 0.3 sebelum kalkulasi persentase error (MAPE). Hasil pengujian menunjukkan bahwa model mampu merumuskan nilai adaptive threshold yang presisi di angka 60%. Evaluasi kinerja sistem membuktikan efektivitas metode ini dengan raihan Precision sebesar 96.56%, Recall 77.97%, dan F1-Score mencapai 86.28%. Implementasi gerbang logika terbukti sukses menekan angka alarm palsu secara drastis, sehingga arsitektur yang dibangun tidak hanya cerdas dalam mengidentifikasi jejak serangan siber, tetapi juga siap digunakan sebagai lapisan perlindungan proaktif bagi infrastruktur jaringan kampus.

BIODATA PENULIS



Nama : I Gede Arya Restyandika Pramana
Putra
Alamat : Jl. Tunjung Tuter No. 01, Peguyangan
Kaja, Denpasar, Bali.
TTL : Denpasar, 23 Juli 2004
Telepon : 081 939 446 116
Email : aryagede963@gmail.com
Angkatan : 2022
Moto : Setiap hari adalah kesempatan belajar
hal baru.

Latar Belakang Pendidikan:

2010 – 2016 : SD Negeri 1 Kuta
2016 – 2019 : SMP Negeri 12 Denpasar
2019 – 2022 : SMA Negeri 8 Denpasar
2022 – Sekarang : Institut Bisnis dan Teknologi Indonesia

Pengalaman Organisasi:

2022 – 2023 : Anggota UKM Sinbun INSTIKI.

Pengalaman Kerja:

2024 – 2025 : Magang di perusahaan PT.in4Reality Studio
Nusantara.